

KUBERNETES

© Copyright 2024 Zenika. All rights reserved

KUBERNETES (ADMIN)



© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



1. Architecture
2. Installation
3. Configuration and operational maintenance
4. Cluster Upgrade
5. Day to day actions, make your Kubernetes users happy and aware
6. Extensibility: Operators, CRD and API Servers
7. Service Mesh, Security

© Copyright 2024 Zenika. All rights reserved

LOGISTICS

- Schedule
- Lunch
- Other questions?

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



ARCHITECTURE

© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



- *Architecture*
- Installation
- Configuration and operational maintenance
- Cluster Upgrade
- Day to day actions
- Extensibility: Operators, CRD and API Servers
- Service Mesh, Security

© Copyright 2024 Zenika. All rights reserved

CHAPTER CONTENT

- Introduction
- Control Plane
- Workers

© Copyright 2024 Zenika. All rights reserved

HOW TO..

- How to deal with container errors/unexpected stops?
- How to deal with node crash?
- How to deal with node maintenance?
- How to deal with scale up/down?
- How to deal with the multiplication of components to manage/deploy?
- How to deal with your apps updates?

© Copyright 2024 Zenika. All rights reserved

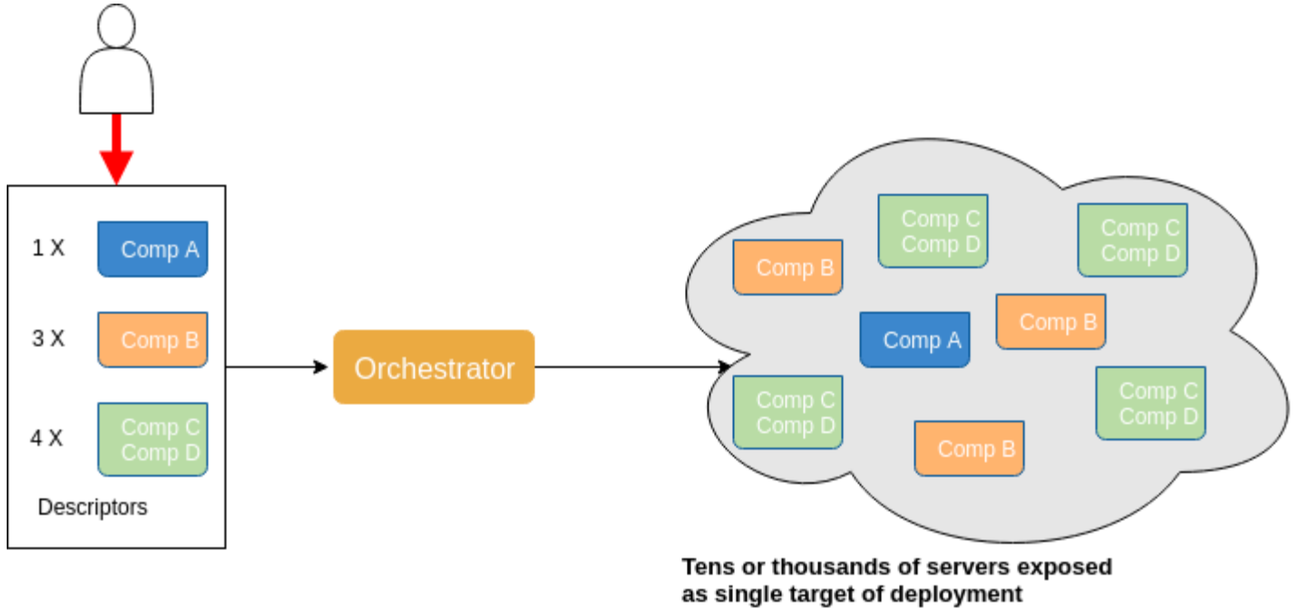
WHY USE AN ORCHESTRATOR?



- Orchestrators have been built to answer all these questions by design
- Orchestrators are often called "Datacenter Operating Systems"
- They are tools built to ease the management of containerized workloads

© Copyright 2024 Zenika. All rights reserved

THE ORCHESTRATOR FROM A FINAL USER PERSPECTIVE



© Copyright 2024 Zenika. All rights reserved

PLACEMENT AND CONTAINER LIFECYCLES



The orchestrator will choose where to host your containers according to constraints:

- technical constraints: network, hardware (GPU, HDD/SSD)
- co-localisation (affinity, anti-affinity, ...)
- number of replicas/copies
- CPU/memory/storage resources availability

© Copyright 2024 Zenika. All rights reserved

FAILOVER MANAGEMENT



- The orchestrator will restart components that failed:
  - containers
  - nodes/hosts
- The orchestrator offers native high availability for control plane components

© Copyright 2024 Zenika. All rights reserved

NETWORKING FEATURES



- Load Balancing
- Service Discovery
- Networks shared by all containers accross nodes (most of the time *overlays*) so that containers don't need anymore to be connected using node ports

© Copyright 2024 Zenika. All rights reserved

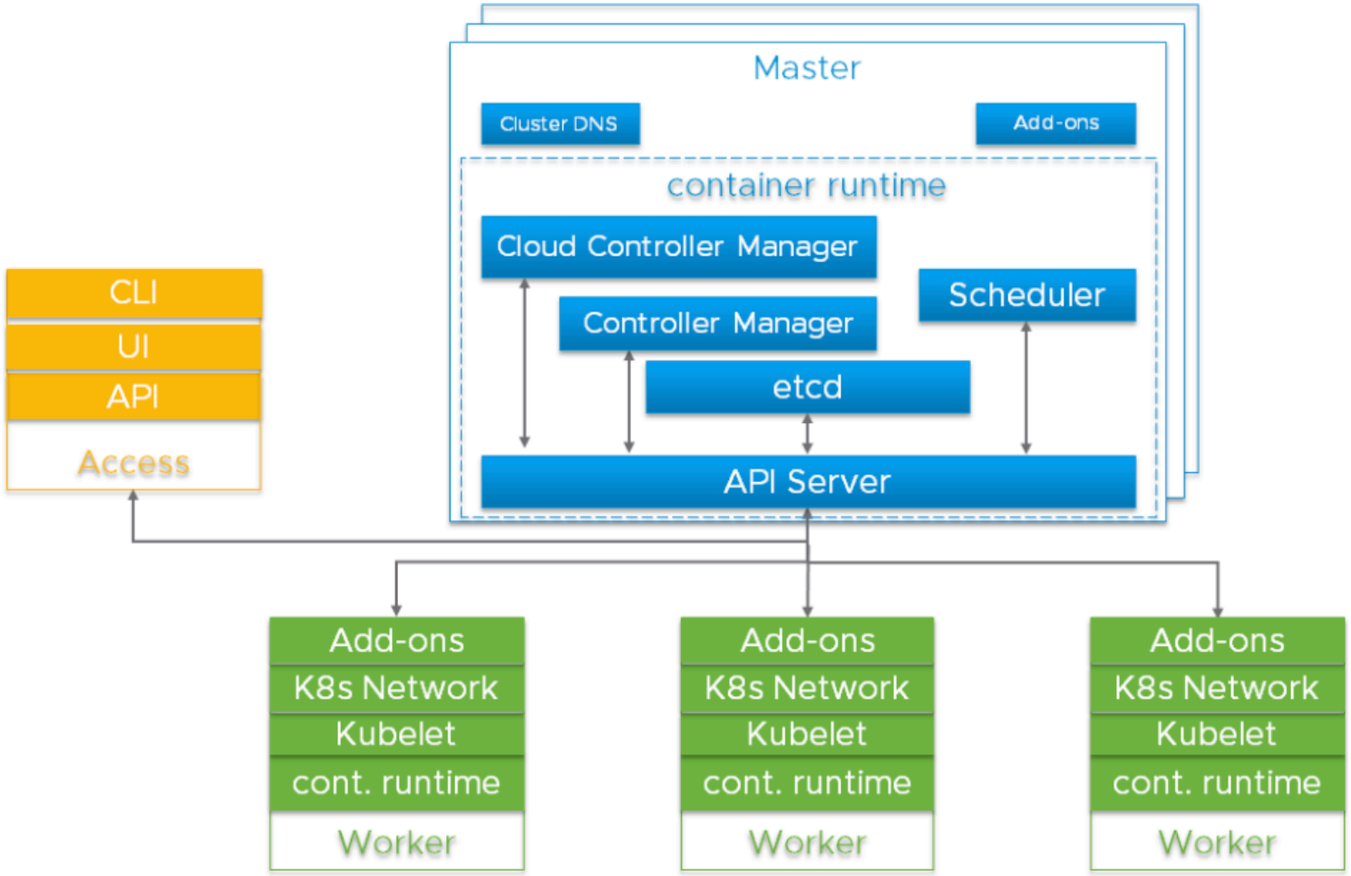
ORCHESTRATION FEATURES



- Distributed storage management
- Configuration and Secrets
- Manual and auto scaling
- Interaction through a command line and a REST API
- Text decriptors that can be managed like the rest of the application code
- *Role Based Access Control*: management of who can do what

© Copyright 2024 Zenika. All rights reserved

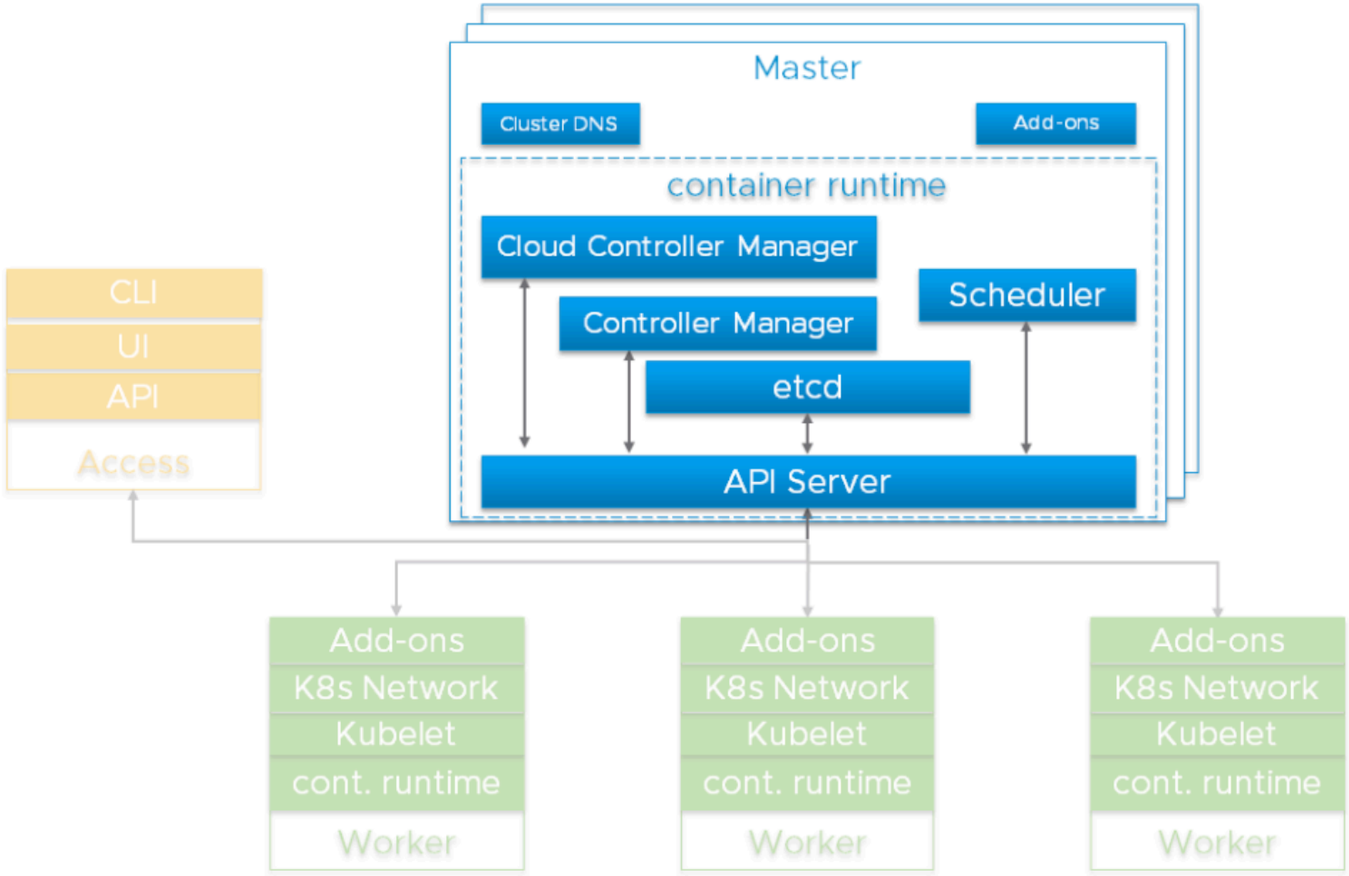
KUBERNETES ARCHITECTURE



Source: kubeadm Cluster Creation Internals - Lucas Käldström

© Copyright 2024 Zenika. All rights reserved

CONTROL PLANE



© Copyright 2024 Zenika. All rights reserved

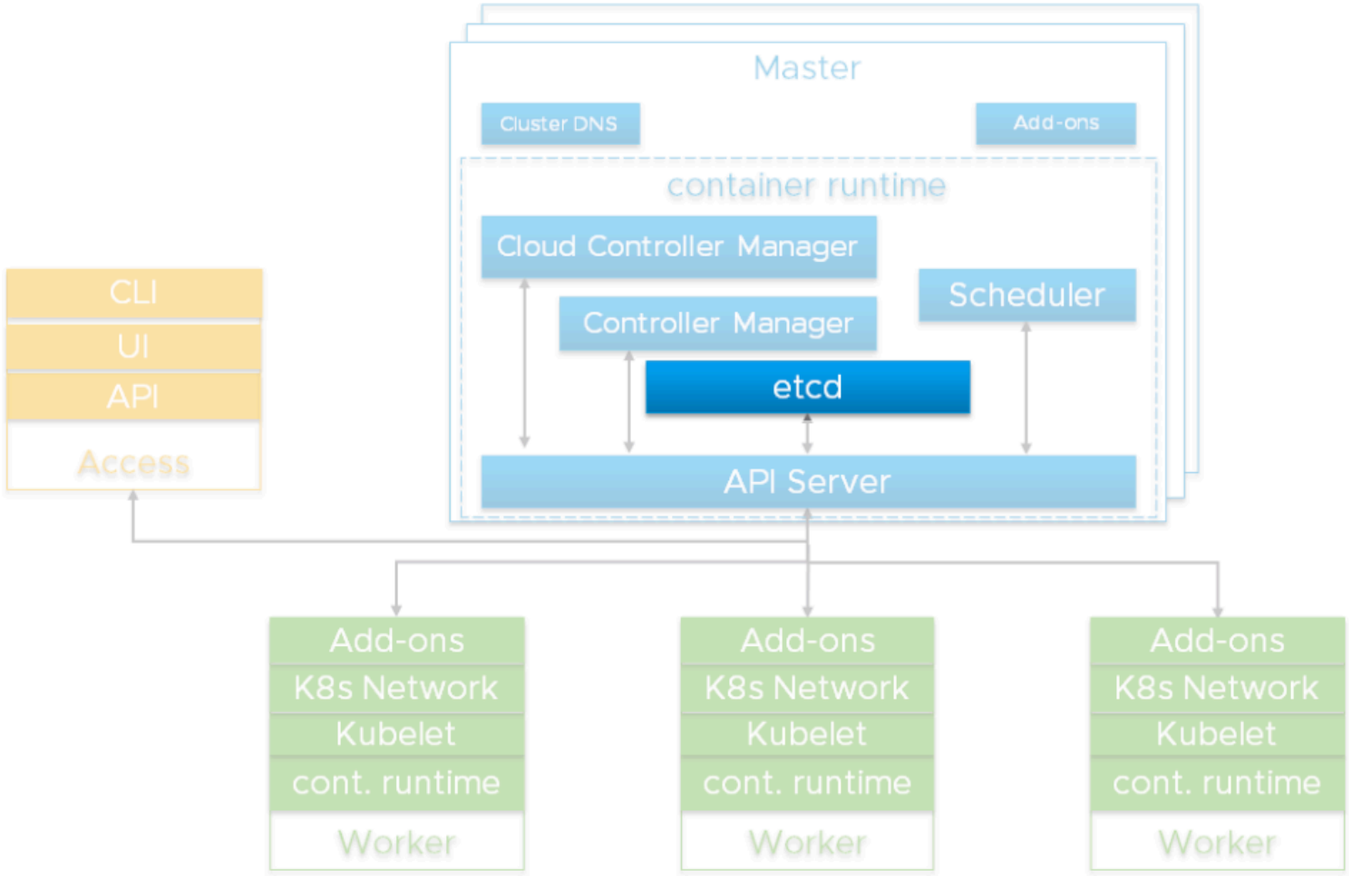
CONTROL PLANE



- Number of control-plane nodes defines your fault tolerance
  - $2*n + 1$ control-plane nodes are needed to handle n failures
- Etcd can be hosted inside or outside the cluster
- If the control plane fails:
  - No updates/modifications can be applied to existing workloads and configurations
  - Workloads are not moved when a worker node fails
- But:
  - Workloads stays online
  - Containers can be restarted by the kubelet on the same node if required because it is not managed by a controller

© Copyright 2024 Zenika. All rights reserved

ETCD



© Copyright 2024 Zenika. All rights reserved

ETCD - OVERVIEW



- A *distributed hierarchical key/value store* from CoreOS/Red Hat
- Now a CNCF Project
- Programmed in Golang
- Node fault tolerance implemented with the Raft protocol

© Copyright 2024 Zenika. All rights reserved

ETCD - RAFT PROTOCOL



- Raft: an *understandable* distributed consensus algorithm
- Node roles: *leader, candidate or follower*
- Logs replication
  - Followers write to their log on leader request
  - Leader commits writes to log when follower majority acknowledges
  - Leader notifies followers of the new consensus
  - The leader *never* erases committed logs

ℹ️ If you want to further understand raft you can check Raft or
`docker container run -p 80:80 aaabbb000/nginx-raftscope`

© Copyright 2024 Zenika. All rights reserved

ETCD IN KUBERNETES



Etcd versions:

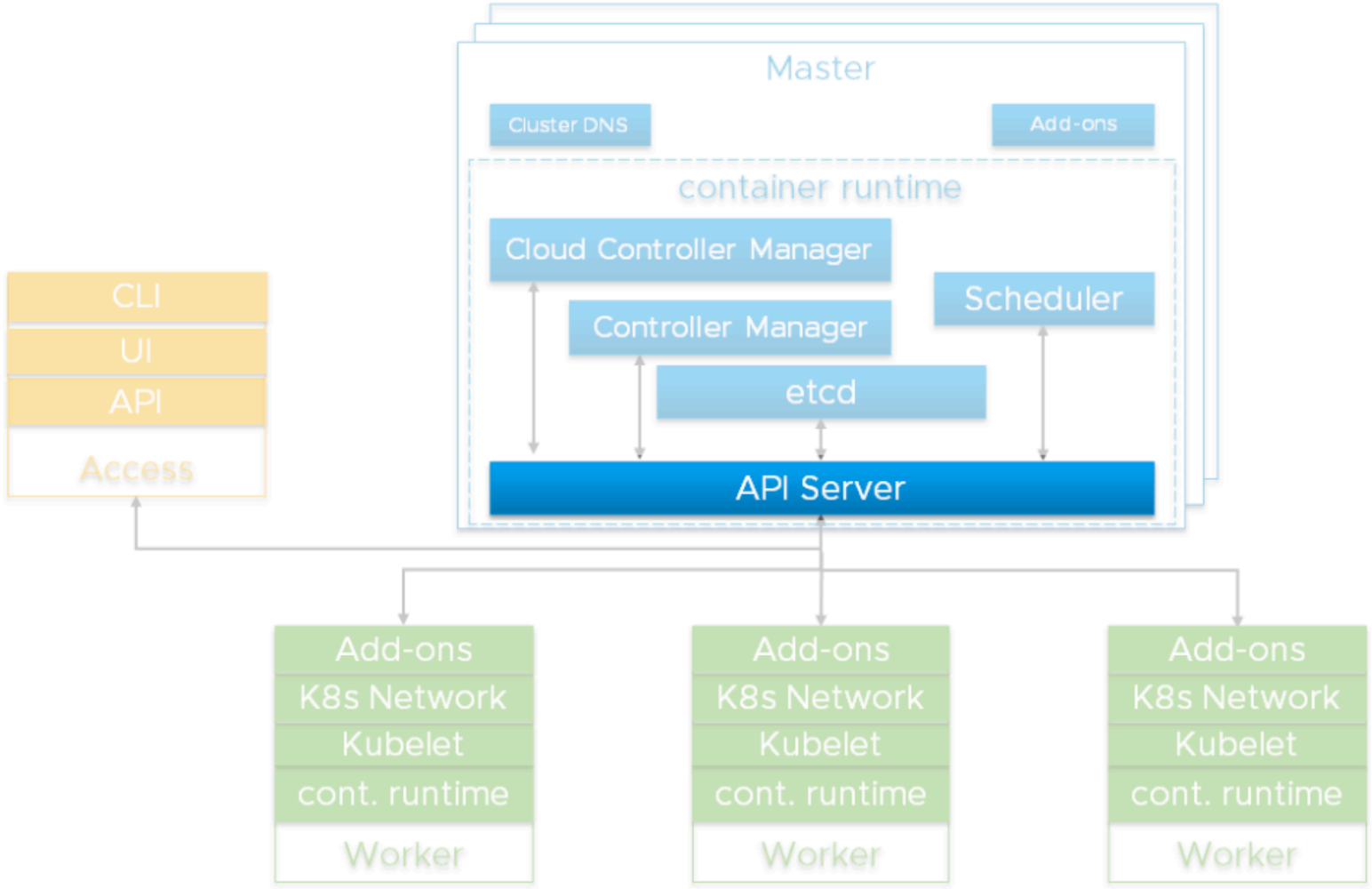
- Kubernetes Changelog points out which etcd version you should use
- etcd API v3 (with gRPC/protobuf)

With APIv3 and gRPC, api-server can register watchers on etcd resources

⚠️ Only the API Server should interact with etcd (except for backup/restore)

© Copyright 2024 Zenika. All rights reserved

API SERVER



© Copyright 2024 Zenika. All rights reserved

API SERVER



- Handles *all interactions* with the cluster
  - both external and internal
- Main front to the etcd database and so to the cluster desired state
  - authorizes and validates all requests
- Exposes the API
  - HTTPS/JSON

© Copyright 2024 Zenika. All rights reserved

API SERVER - REQUEST FLOW



1. API Groups and version negotiation
2. Authentication
3. Authorization
4. Admission control
5. Storage to etcd

ℹ️ API servers are running active-active and always query etcd with quorum read making sure they see the last consistent state

© Copyright 2024 Zenika. All rights reserved

API SERVER - REMINDERS API GROUPS



The Kubernetes API is divided in API groups specified in a *REST path* and in the *apiVersion* field of a serialized object

- Core group (or legacy) at the REST path `/api/v1` and uses `apiVersion: v1`
- The named groups are at REST path `/apis/GROUP_NAME/VERSION`, and use `apiVersion: GROUP_NAME/VERSION` (e.g. `apiVersion: batch/v1`)
  - Full list available here

© Copyright 2024 Zenika. All rights reserved

API SERVER - ADMISSION CONTROLLERS (1/3)



- Intercept requests to the API Server
- Two kinds of Admission controllers executed in order
  - Mutating: Make changes to a request/resource prior persistence
  - Validating: Validate the request

© Copyright 2024 Zenika. All rights reserved

API SERVER - ADMISSION CONTROLLERS (2/3)



Admission controllers are enabled/disabled with parameters passed to the kube-apiserver command line.

- Enabling admission controller `NamespaceLifecycle`

```
kube-apiserver --enable-admission-plugins=NamespaceLifecycle ...
```

- Disabling admission controller `PodNodeSelector`

```
kube-apiserver --disable-admission-plugins=PodNodeSelector ...
```


© Copyright 2024 Zenika. All rights reserved

API SERVER - ADMISSION CONTROLLERS (3/3)



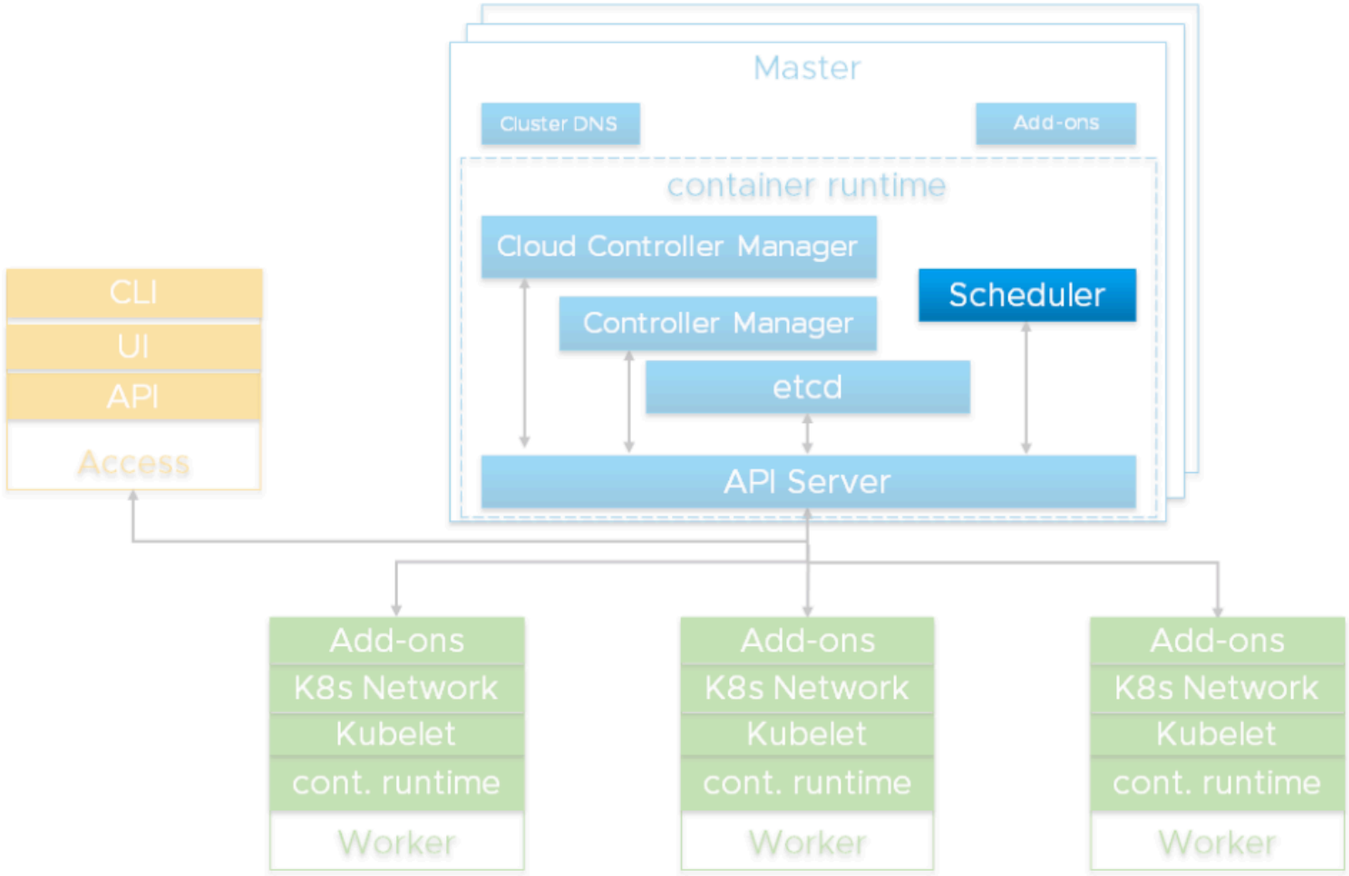
Examples of included admission controllers:

- `DefaultStorageClass` applies a default storage class on pvc which don't have one
- `LimitRanger` ensures respect of the `LimitRange` constraint in the target namespace
- `NamespaceLifecycle` rejects creation in non-existent or being deleted namespace and deletion of `default`, `kube-system`, `kube-public` namespaces

ℹ️ Full list available here.

© Copyright 2024 Zenika. All rights reserved

SCHEDULER



© Copyright 2024 Zenika. All rights reserved

SCHEDULER



- Defines where to run workloads based on available resources
- Runs in active/passive mode
  - Current active scheduler is flagged in `etcd`
- Two steps
  - Filtering by predicates
  - Scoring by priorities
  - If two nodes get the same score, choose random

© Copyright 2024 Zenika. All rights reserved

SCHEDULER - FILTERING (1/2)



Filter Predicates in Kubernetes

- `NodeResourcesFit`: Checks if the node has all the resources that the Pod is requesting.
- `NodePorts`: Checks if a node has free ports for the requested Pod ports.
- `PodTopologySpread`: Implements Pod topology spread.
- `InterPodAffinity`: Implements inter-Pod affinity and anti-affinity.
- `NodeAffinity`: Implements node selectors and node affinity.
- `NodeUnschedulable`: Filters out nodes that have `.spec.unschedulable` set to true.
- `NodeName`: Checks if a Pod spec node name matches the current node.
- `TaintToleration`: Implements taints and tolerations.

© Copyright 2024 Zenika. All rights reserved

SCHEDULER - FILTERING (2/2)



- `VolumeRestrictions`: Checks that volumes mounted in the node satisfy restrictions that are specific to the volume provider.
- `VolumeBinding`: Checks if the node has or if it can bind the requested volumes.
- `EBSLimits`: Checks that AWS EBS volume limits can be satisfied for the node.
- `GCEPDLimits`: Checks that GCP-PD volume limits can be satisfied for the node.
- `NodeVolumeLimits`: Checks that CSI volume limits can be satisfied for the node
- `AzureDiskLimits`: Checks that Azure disk volume limits can be satisfied for the node
- `VolumeZone`: Checks that volumes requested satisfy any zone requirements they might have

© Copyright 2024 Zenika. All rights reserved

SCHEDULER - RANKING (1/2)



For example, suppose there are two priority functions, `priorityFunc1()` and `priorityFunc2()` with weighting factors `weight1` and `weight2` respectively, the final score of some `NodeA` will be:

```
finalScoreNodeA = (weight1 * priorityFunc1) + (weight2 * priorityFunc2)
```

After the scores of all nodes are calculated, the node with highest score is chosen as the host of the Pod. If there are more than one node with equal highest scores, a random one among them is chosen.

© Copyright 2024 Zenika. All rights reserved

SCHEDULER - RANKING (2/2)



Default ranking strategies:

- `NodeResourcesBalancedAllocation`: Favors nodes that would obtain a more balanced resource usage if the Pod is scheduled there.
- `ImageLocality`: Favors nodes that already have the container images that the Pod runs.
- `InterPodAffinity`: Implements inter-Pod affinity and anti-affinity.
- `NodeResourcesFit`: Checks if the node has all the resources that the Pod is requesting.
- `NodeAffinity`: Implements node selectors and node affinity.
- `PodTopologySpread`: Implements Pod topology spread. Weight is doubled because this is a score coming from user preference.
- `TaintToleration`: Implements taints and tolerations.

© Copyright 2024 Zenika. All rights reserved

SCHEDULER - SERVER CONFIGURATION



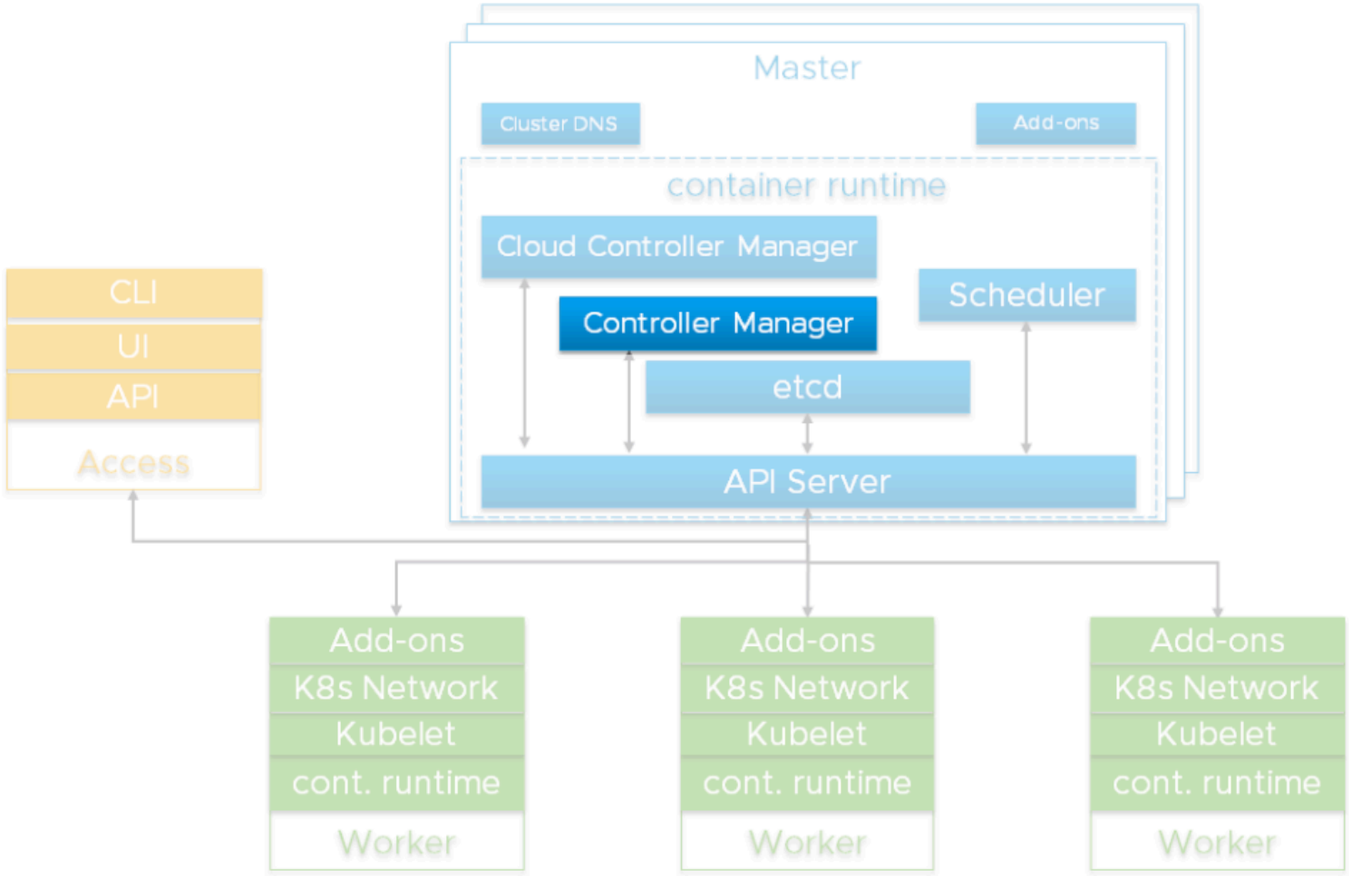
You can change the default scheduler policy by defining a custom profile in a `KubeSchedulerConfiguration`:

`k8s-scheduler-config.yaml`

```yaml
apiVersion: kubescheduler.config.k8s.io/v1beta2
kind: KubeSchedulerConfiguration
clientConnection:
  kubeconfig: kubeconfig
leaderElection:
  leaderElect: true
profiles:
  pluginConfig:
    - name: PodTopologySpread
    [...]
```


© Copyright 2024 Zenika. All rights reserved

CONTROLLER MANAGER



© Copyright 2024 Zenika. All rights reserved

CONTROLLER MANAGER



- Implements the control loop for workloads (`ReplicationController`, `ReplicaSet`, `Deployment`, ...) and other resources
- Runs in active/passive mode
  - Current active controller is flagged in `etcd`

© Copyright 2024 Zenika. All rights reserved

CLOUD CONTROLLER MANAGER

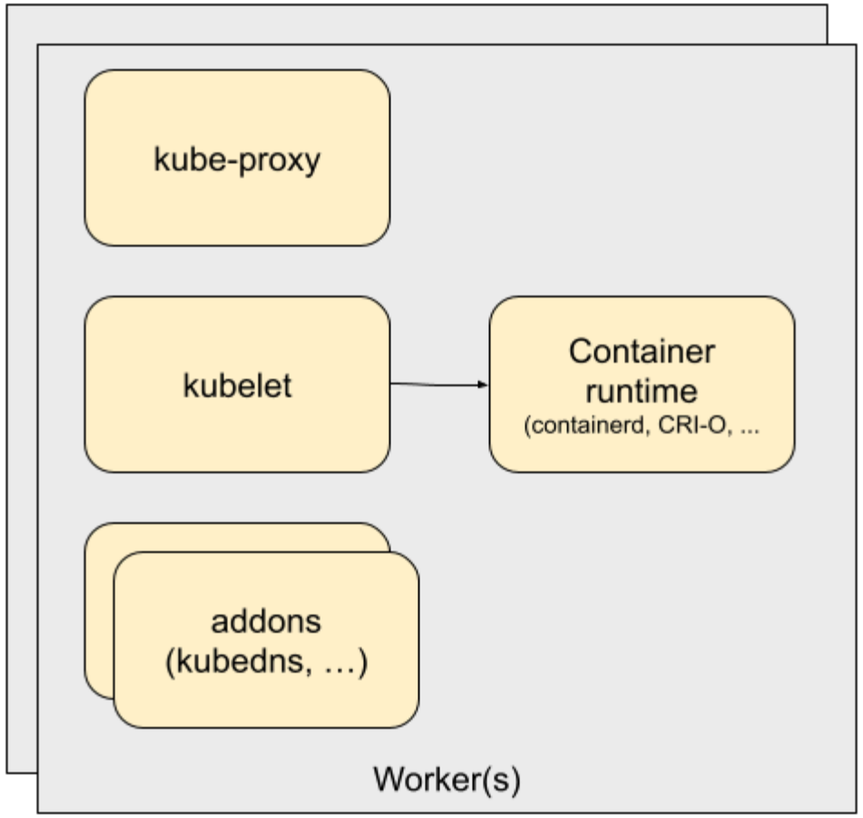


Implementation depends on the Cloud provider:

- Network
- VM Pool
- Storage...

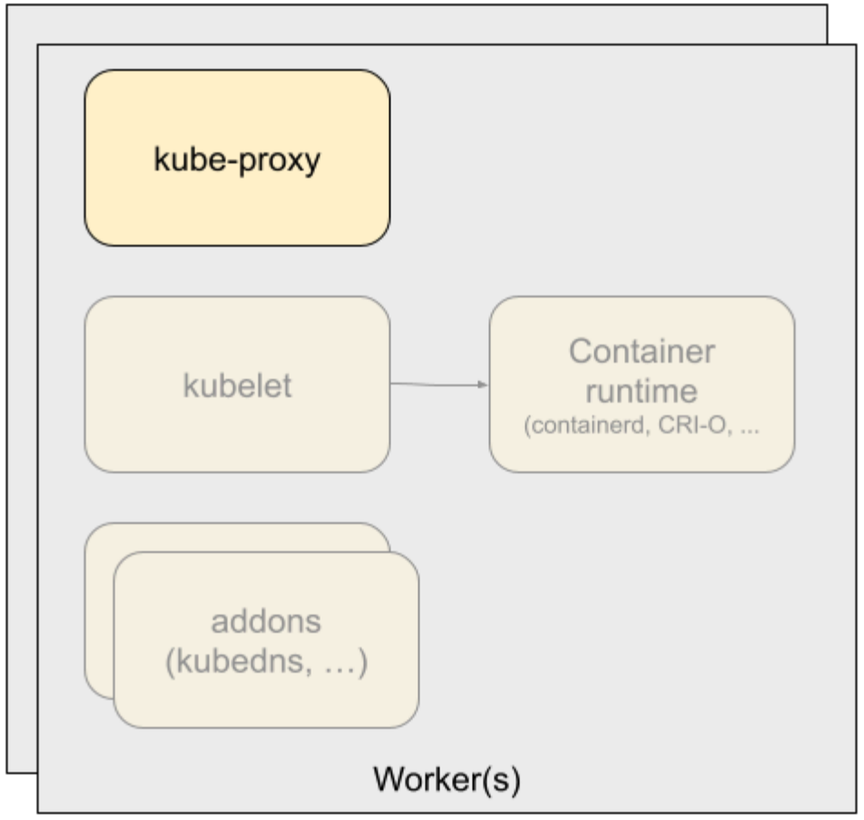
© Copyright 2024 Zenika. All rights reserved

WORKERS



© Copyright 2024 Zenika. All rights reserved

KUBE-PROXY



© Copyright 2024 Zenika. All rights reserved

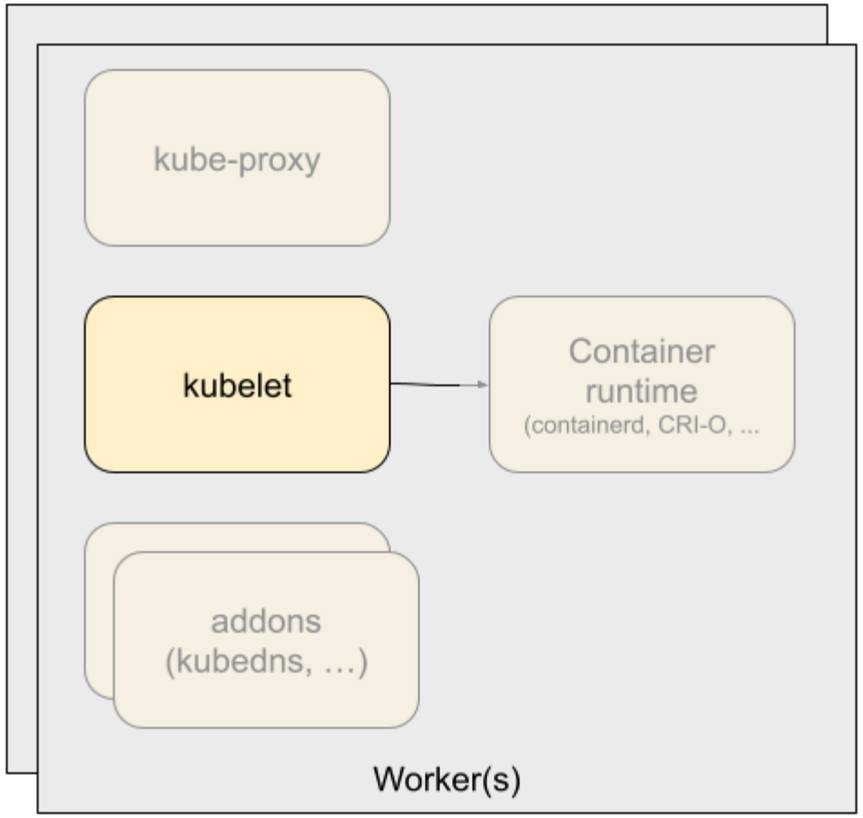
KUBE-PROXY



- Network traffic routing accross the nodes depends on the network addon. But egress, ingress of services are taken care of by the `kube-proxy` component.
- `kube-proxy` configures node network for Kubernetes services
- It uses:
  - `iptables`
  - IPVS - GA since v1.11
  - `nftables`: the successor to the iptables only on Linux nodes - Alpha since v1.29

© Copyright 2024 Zenika. All rights reserved

KUBELET



© Copyright 2024 Zenika. All rights reserved

KUBELET (1/2)



- It is the Node agent, it runs on a Node and make sure that containers included in assigned PodSpecs are running and healthy.
- PodSpecs are provided to the `kubelet` by querying the API Server
- These communications use a `kubeconfig` like any API Server client

© Copyright 2024 Zenika. All rights reserved

KUBELET (2/2)



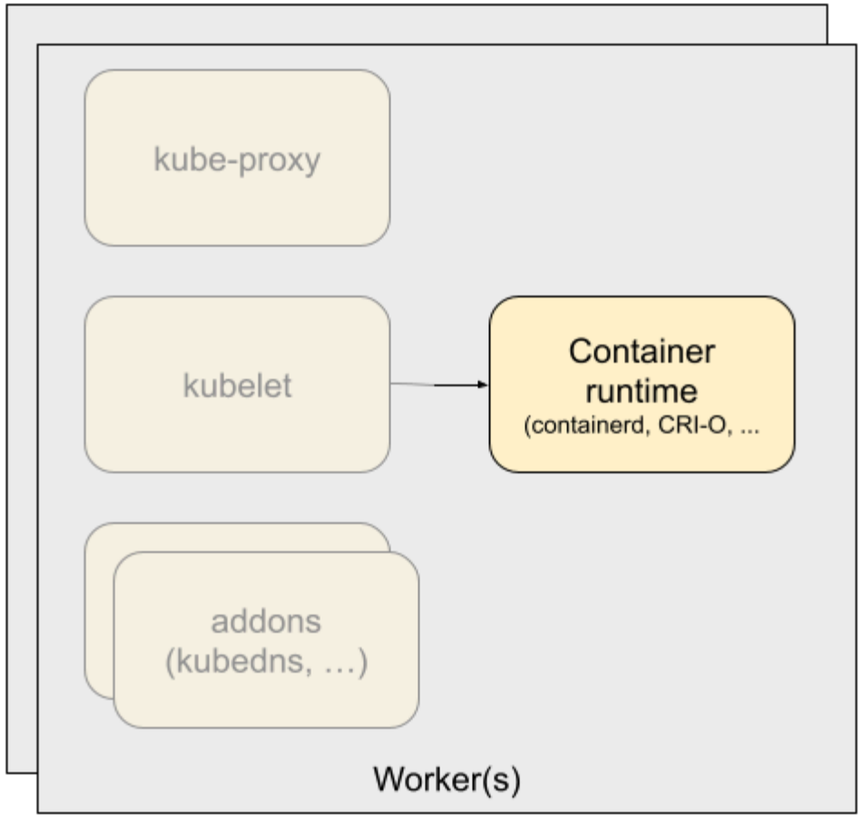
There are also other ways to provide PodSpecs to `kubelet`:

- File: `pod-manifest-path` passed to the command line is monitored every 20s by default
- HTTP Endpoint: `manifest-url` passed to the command line and checked by the `kubelet` every 20s by default
- HTTP Server: the `kubelet` expose an API use to interact with Pods which can be used to create new Pods

⚠️ The HTTP Server on the `kubelet` allows creation of pods *without authentication*. It should be **disabled** or **at least secured**

© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIME



© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIME



The `kubelet` handles container runtimes which implements the Container Runtime Interface (CRI)

It supports:

- docker (deprecated since 1.20, 12/2020)
- containerd
- cri-o

It's also possible to use more than one container runtime in parallel. See Chapter 2 - CRI

⚠️ Check the compatible/endorsed version of your runtime

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



Lab 1: Container runtime & Kubelet

© Copyright 2024 Zenika. All rights reserved

NETWORK COMMUNICATIONS SUMMARY (1/2)



Control plane node(s)

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	6443	Kubernetes API server	All
TCP	Inbound	2379	etcd client API	kube-apiserver
TCP	Inbound	2380	etcd server peer	etcd
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	10251	kube-scheduler	Self
TCP	Inbound	10252	kube-controller-manager	Self

© Copyright 2024 Zenika. All rights reserved

NETWORK COMMUNICATIONS SUMMARY (2/2)

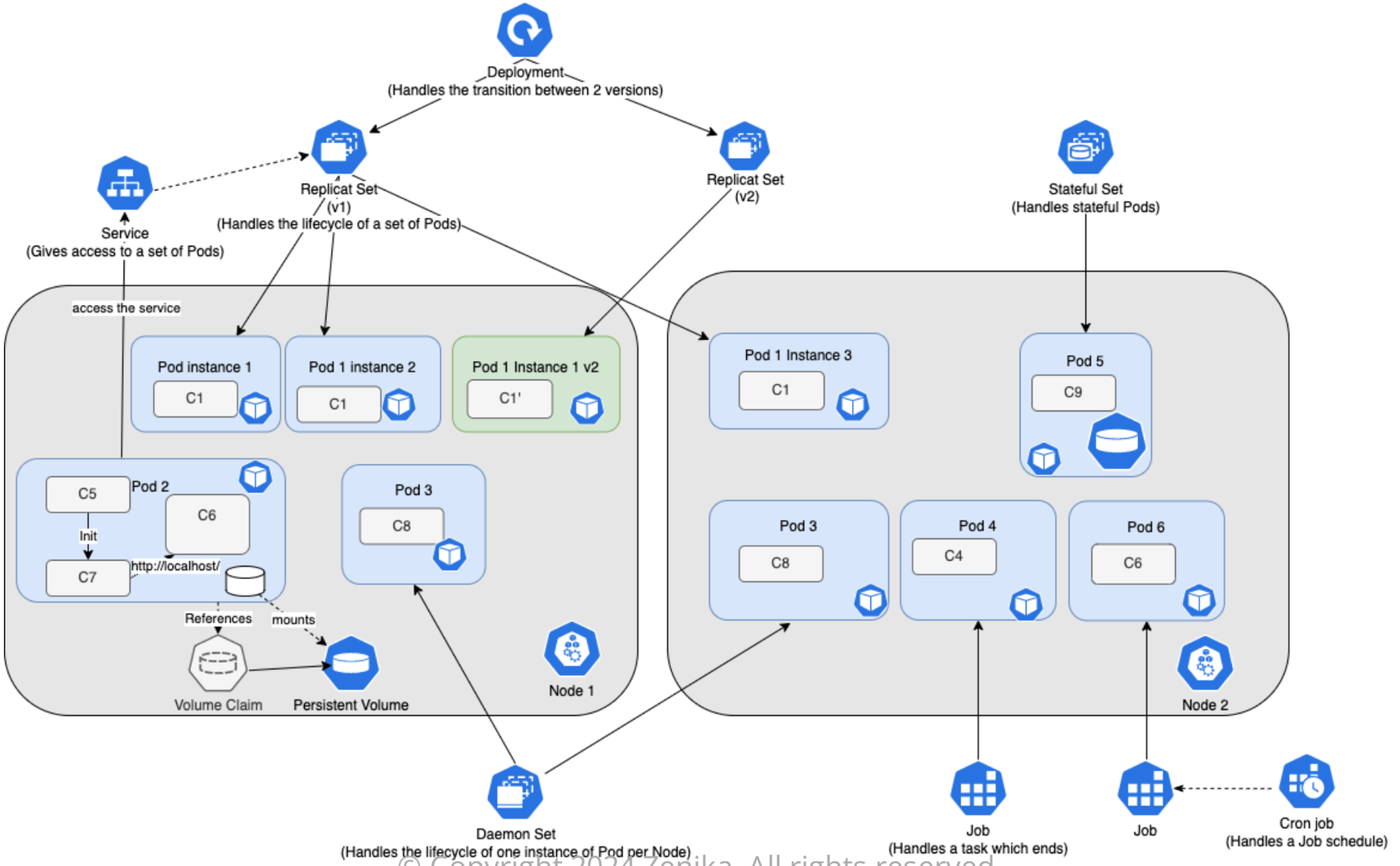


Worker node(s)

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	30000-32767	NodePort Services	All

© Copyright 2024 Zenika. All rights reserved

RESOURCES SUMMARY USED BY OUR USERS



© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



INSTALLATION

© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



- Architecture
- *Installation*
- Configuration and operational maintenance
- Cluster Upgrade
- Day to day actions
- Extensibility: Operators, CRD and API Servers
- Service Mesh, Security

© Copyright 2024 Zenika. All rights reserved

CHAPTER CONTENT

- Installation steps
- Distributions
- Network solutions
- Container Runtimes
- DNS
- Storage

© Copyright 2024 Zenika. All rights reserved

INSTALLATION STEPS - SUMMARY



1. Infrastructure provisioning
2. Certificates creation
3. Kubernetes configuration files
4. Etcd installation
5. Control plane members installation
6. Worker members installation
7. kubectl configuration
8. Addons installation (network, dns, ...)

© Copyright 2024 Zenika. All rights reserved

INSTALLATION STEPS - PREPARATION



- How do I manage the certificates?
- Local or distant etcd?
- Which specs and operating system for my nodes?
- What size? How many containers?
- Which distribution should I use?
- Which network solution?
- Which container runtime?
- Which dns addon?
- How do I manage the storage provisioning?

© Copyright 2024 Zenika. All rights reserved

INSTALLATION STEPS - CERTIFICATE MANAGEMENT



- All communications throughout the clusters are encrypted
- Every components should have its own key pair and certificate
- All these certificates are not signed by the same CA

Default CN	Parent CA	kind
kube-etcd	etcd-ca	server, client
kube-etcd-peer	etcd-ca	server, client
kube-etcd-healthcheck-client	etcd-ca	client
kube-apiserver-etcd-client	etcd-ca	client
kube-apiserver	kubernetes-ca	server
kube-apiserver-kubelet-client	kubernetes-ca	client
front-proxy-client	kubernetes-front-proxy-client	client

© Copyright 2024 Zenika. All rights reserved

INSTALLATION STEPS - CERTIFICATE RENEWAL



Certificates generated by `kubeadm` will expire 1 year after generation.

- Kubelet certificates auto-renewal is in beta
- All certificates are renewed when doing an upgrade
- You can run `kubeadm certs renew` to renew a specific cert
- You can run `kubeadm certs check-expiration` to check certificate expiration dates

© Copyright 2024 Zenika. All rights reserved



Lab 2.1: Certificates

© Copyright 2024 Zenika. All rights reserved

INSTALLATION STEPS - ETCD



- Kubernetes can use an external `etcd` cluster
  - `etcd` nodes must be installed and managed
- Or use an embedded one, hosted in the control plane nodes
  - `etcd` nodes are tied to control plane nodes
  - run as static `Pod`
- `etcd` node count is not meant to be changed
  - Think ahead before choosing how many `etcd` node you use
  - Usually 3 or 5

© Copyright 2024 Zenika. All rights reserved

INSTALLATION STEPS - NODE SPECS



- Virtual or physical machines
- x86_64, arm64 architecture

Control-Plane:

- `api-server` and `etcd` need at least 1 core and 1GB memory for 10s of nodes
- Using VMs allows to migrate `control-plane` more easily

Workers:

- Nodes do not have to be identical (CPU, Memory, ...)
- Choose the right Node size:
  - More nodes means more infrastructure overhead (daemonsets, agents, ...)
  - Less nodes means less flexibility and resilience
  - See: https://learnk8s.io/kubernetes-instance-calculator

© Copyright 2024 Zenika. All rights reserved

INSTALLATION STEPS - NODE OS



- Compatible with chosen container runtime
- With a well defined and applied upgrade strategy (especially for the kernel which is shared)
- A compatible Linux host. The Kubernetes project provides generic instructions for Linux distributions based on Debian and Red Hat and distributions without a package manager.

© Copyright 2024 Zenika. All rights reserved

CLUSTER SIZING - HA



Component	Role	Effect of loss	Recommended # instances
etcd	Maintains state of all Kubernetes objects	Loss of storage = catastrophic Loss of majority = Kubernetes loses control plane, API servers. Read only API calls not requiring a quorum may work and existing workloads may continue to run"	odd number, 3+
API Server	Provides API used internally and extenally	Unable to stop, start, udate pods, services, replication controller. Scheduler and Controllers Manager depend on it. Workloads continue if they are not dependant on API calls (operators, custom controllers, CRDs, etc...)"	2+
kube-scheduler	Places pods on nodes	No pod placements, priority and preemption	2+
kube-controller-manager	Runs many controllers	Core control loops that regulate state, in-tree cloud provider integration breaks	2+
cloud-controller-manager	Integration for out-of-tree cloud providers	Cloud provider integration breaks	1 (you don't manage it)
Add-ons (eg DNS)	Varies	Varies	Depends on add-on 2+ for DNS

© Copyright 2024 Zenika. All rights reserved

CLUSTER SIZING - NODES NUMBER



When creating a cluster, you should respect these constraints:

- No more than 5000 nodes
- No more than 150000 total pods
- No more than 300000 total containers
- No more than 110 pods per node

© Copyright 2024 Zenika. All rights reserved

CLUSTER SIZING - OTHER LIMITATIONS



There is a SIG (Special Interest Group) focused on cluster scalability limits.

It's still a work in progress but there are already some limits documented:

Quantity	Threshold scope=namespace	Threshold: scope=cluster
#Nodes	n/a	5000
#Namespaces	n/a	10000
#Pods	3000	150000
#Pods per node	min(110, 10*#cores)	min(110, 10*#cores)
#Services	5000	10000
#Endpoints per service	250	n/a
#Deployments	2000	TBD
#AccessTokens	2000	2000
#AccessTokens verifications	5000 QPS	5000 QPS

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS



More than 50 Kubernetes platforms and distributions

- On premises:
  - OpenShift (ocp/okd)
  - kubeadm, kube-spray, kops
  - Rancher 2
  - Tanzu
- Managed:
  - GKE (Google)
  - AKS (Azure)
  - EKS (Amazon)
  - OVHcloud Kubernetes-as-a-service

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - KUBEADM (1/2)



- Distributed as a single go binary
- Maintained by the Kubernetes community
- Can be used to deploy a High Availability cluster
- Follows the Kubernetes release cycle

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - KUBEADM (2/2)



- `kubeadm` will:
  - Generate certificate chains
  - Deploy `kubelet` on control plane/worker nodes
  - Deploy `etcd` cluster member and control plane components as static `Pods`
  - Provide a simple way to join worker nodes to the cluster
- `kubeadm` won't:
  - Create VMs
  - Configure network infrastructure
  - Install container runtime on machines

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - KUBESPRAY (1/2)



- A set of ansible playbooks
- In kubernetes-sigs
- Not synchronized with Kubernetes release cycle but up to date

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - KUBESPRAY (2/2)



- `kubespray` will:
  - Generate certificate chains
  - Deploy `etcd` on control plane nodes
  - Deploy `kubelet` on control plane/worker nodes
  - Deploy control plane components as static `Pods`
  - Provide some playbook to add nodes later
  - Use `kubeadm` underneath
  - Support multiple OS
- `kubespray` won't:
  - Create VMs
  - Configure network infrastructure

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - KOPS (1/2)



- A tool distributed as a single binary
- Maintained by the Kubernetes community
- Not synchronized with Kubernetes release cycle
- Create a cluster on AWS or GCE
  - Openstack, Hetzner, or DigitalOcean (beta)
  - SpotOcean, Azure (alpha)

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - KOPS (2/2)



- `kops` will:
  - Create VMs
  - Configure network infrastructure
  - Install container runtime on machines
  - Generate certificate chains
  - Deploy `etcd` on control plane nodes
  - Deploy `kubelet` on control plane/worker nodes
  - Deploy control plane components as static pods
  - Provide some playbook to add nodes later
- `kops` won't:
  - Work on every on-premises configurations

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - OPENSHIFT



- A complete and highly integrated Kubernetes distribution created by Red Hat
- Available:
  - with Red Hat support: OpenShift Container Platform
  - as Opensource: OKD
- Includes:
  - Web UI
  - Application catalog
  - a container registry
  - metrics with Prometheus
  - CI/CD with Tekton
  - a kind of PaaS with S2I

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - RANCHER 2



- Rancher now exclusively deploys and manages multiple Kubernetes
- Kubernetes deployments on bare-metal and cloud with `rke`
- Existing cluster management

- Includes:
  - complete UI
  - RBAC and multi tenancy
  - Monitoring and alerting
  - Application catalog (helm)
  - (simple) Pipeline engine

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - TANZU

VMWare Tanzu is an integration of Kubernetes cluster creation and deployment with the vSphere hypervisor.

- Includes:
  - complete UI with Octant
  - automatic cluster creation on your vSphere hypervisor
  - RBAC and LDAP integration with Tanzu Grid
  - default integration for log centralization and cluster monitoring

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - SUMMARY (1/2)



Distribution	HA	Supported	Supported network addons	Container Runtime
kubeadm	✅[1]	Community	Calico, Canal, Flannel, Kube-router, Romana, JuniperContrail/TungstenFabric	containerd, cri-o, docker
kubespray	✅	Community	Calico, Canal, Flannel, Cilium, Contiv	containerd, cri-o, docker
kops	✅	Community	Calico, Canal, Flannel, kopeio-vxlan, kube-router, Cilium, romana, amazon-vpc-routed-eni	containerd
OpenShift	✅	Red Hat	OpenShift *, Flannel, Nuage Networks	cri-o
Rancher 2	✅	Rancher Labs	Calico, Canal, Flannel	containerd, cri-o, docker
Tanzu	✅	VMWare	Antrea, Calico	containerd

[1]: Introduced in kubeadm 1.11, relies on an external LB

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - SUMMARY (2/2)



Decision helpers:

- Do you need enterprise support?
- Do you need UI for your end users?
- Do you want to manage your control plane?
- Is it easy to setup? to maintain?
- Do you want to run Kubernetes on premise?
- Which cloud provider is the best for your needs? (costs / auto upgrade / geographical area...)

© Copyright 2024 Zenika. All rights reserved

DISTRIBUTIONS - CNCF CONFORMANCE TEST



The CNCF provides a *conformance test suite* : https://sonobuoy.io/ . This test suite is developped by vmware, the source code and docs are available on github.

It is provided as a binary which deploys resources on the cluster testing expected behaviour of controllers, scheduler, etc.

```
$ sonobuoy run
```

View tests progress and logs

```
$ sonobuoy status
$ sonobuoy logs
```

It produces a report available through this command:

```
$ sonobuoy retrieve .
```


© Copyright 2024 Zenika. All rights reserved



Lab 2.2: Installation

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS (1/2)



As stated in the official Kubernetes documentation:

> Kubernetes imposes the following fundamental requirements on any networking implementation (barring any intentional network segmentation policies):
>
> - all containers can communicate with all other containers without NAT
> - all nodes can communicate with all containers (and vice-versa) without NAT
> - the IP that a container sees itself as is the same IP that others see it as

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS (1/2)



To fulfill these requirements two kinds of network plugins can be used:

- kubenet is the default and doesn't itself implements cross-node networking
- cni plugin and deploy a network addon on the cluster

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - REMINDERS (1/2)

Layer		Protocol Data Unit (PDU)	Protocol Examples	Function
L7	Application	Data	HTTP	High-level APIs, including resource sharing, remote file access
L6	Presentation			Translation of data between a networking service and an application; includingcharacter encoding, data compression and encryption/decryption
L5	Session			Managing communication sessions, i.e. continuous exchange of information in the form of multiple back-and-forth transmissions between two nodes
L4	Transport	Segment, Datagram	TCP/UDP	Reliable transmission of data segments between points on a network, includingsegmentation, acknowledgement and multiplexing
L3	Network	Packet	IP	Structuring and managing a multi-node network, including addressing, routingand traffic control
L2	Data link	Frame	Ethernet	Reliable transmission of data frames between two nodes connected by a physical layer
L1	Physical	Symbol		Transmission and reception of raw bit streams over a physical medium

© Copyright 2024 Zenika. All rights reserved

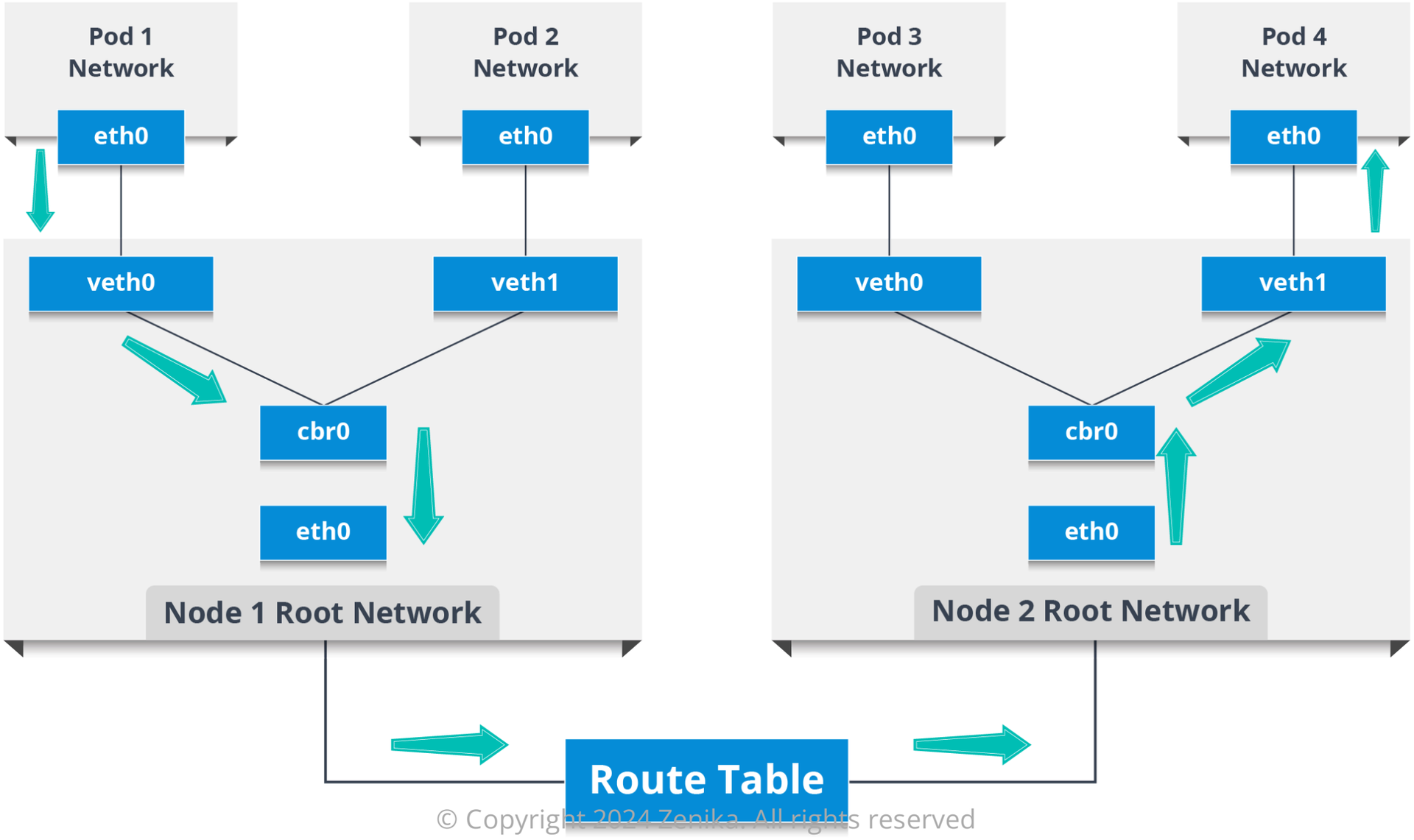
NETWORK SOLUTIONS - REMINDERS (2/2)



- Overlay network: An overlay network decouples network services from the underlying infrastructure by encapsulating one packet inside another packet
- BGP: Border Gateway Protocol is a protocol used to exchange routing and reachability information
- Vxlan: Virtual Extensible LAN encapsulates L2 frames within L4 UDP datagrams
- eBPF: extended Berkeley Packet Filter provides advanced rules to filter traffic without leaving kernel space

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - NATIVE ROUTING



© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - KUBENET



- Kubenet creates a `cbr0` network bridge interface
- All created pods are then affected a `veth` pair and the Pod end of the pair is assigned an IP address allocated from a configured range

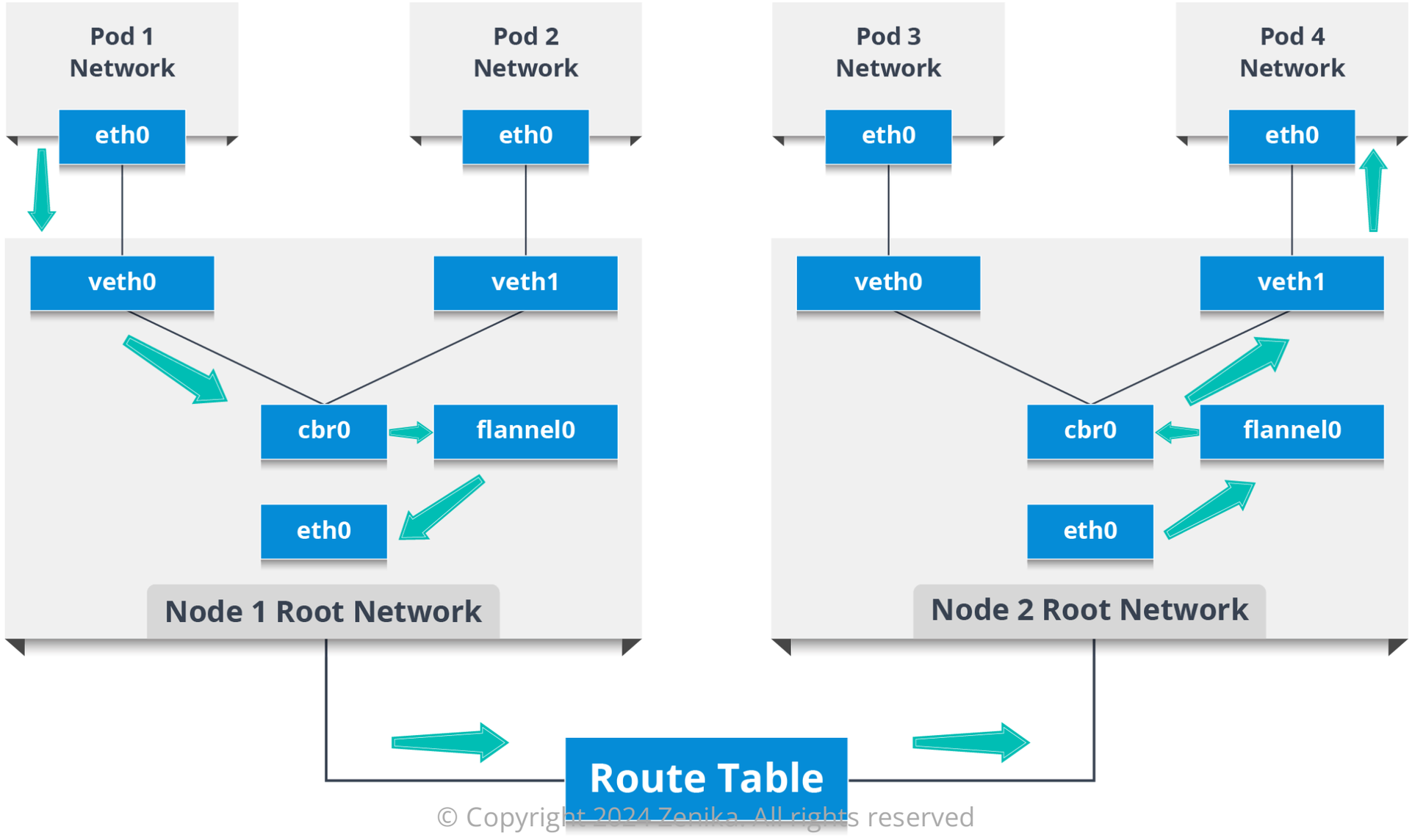
But kubenet comes with some requirements. Either:

1. A route declaration exists for each node pods CIDR range (GKE is implemented this way)
2. All interfaces are on the same ethernet switching infrastructure (L2) but:

> "it seems to work, but has not been thoroughly tested"

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - OVERLAY NETWORK



© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - CNI (1/3)



There are many addons supporting CNI and providing the needed network connectivity to your cluster

- Flannel
- Project Calico
- Canal
- Cilium
- ...

All of these use some mechanic (initContainers, ...) to put the CNI plugin binary on the kubelet host.

Each of these comes with a different approach, features and performances...

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - CNI (2/3)



A CNI plugin is a simple command invoked by the kubelet which should handle 4 event types:

- `ADD`: sent when a new container is created, the plugin has to set up the network namespace, interface, routes, ...
- `DEL`: sent when a container is deleted
- `GET`: used to get information on a previously created container (currently not used by kubelet)
- `VERSION`: used to get the supported CNI versions

The event type and payloads are passed through environment variables

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - CNI (3/3)



To install a CNI Plugin, you must:

1. Put its config file in `/etc/cni/net.d/`
2. Put its executable command in `/opt/cni/bin`

Configuration example for Flannel:

```
{
  "name": "cbr0",
  "plugins": [{
    "type": "flannel",
    "delegate": {
      "hairpinMode": true,
      "isDefaultGateway": true
    }
  }]
}
```


© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - FLANNEL



Flannel is a simple network overlay from CoreOS.

- It supports different backends:
  - to encapsulate traffic between nodes (vxlan, udp)
  - to route traffic between nodes on the same L2 network (host-gw)

Flannel stores its state in etcd

It doesn't support Network Policies

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - PROJECT CALICO



Project Calico provides network connectivity for various platforms (Kubernetes, Openstack, Mesos, ...)

- Uses BGP to enable routing between Pods
- Supports IP in IP encapsulation if the network drops traffic to/from unknown addresses
- Supports Network Policies

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - CILIUM



Cilium uses eBPF to leverage a high performance network solution

- Stores its state in etcd
- Supports overlay network (default) or native routing
- Default Kubernetes Network Policies + cilium specifics available

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - CANAL



Canal combines Flannel and Calico

- Flannel provides the overlay or non-overlay network connectivity options
- Calico provides Network Policies enforcement

© Copyright 2024 Zenika. All rights reserved

NETWORK SOLUTIONS - CNI SUMMARY



Solution	Encapsulation	Network Policies	IPv6	State
Flannel	vxlan	❌	❌	etcd
Calico	IP in IP, vxlan	✅	✅	etcd
Cilium	vxlan, geneve	✅ + specifics	✅	etcd
Canal	vxlan	✅	✅	etcd

Decision helpers:

- Do you need Network Policies?
- Is the backend solution compatible with your existing network?
- Will the solution scale with your cluster?
- Is the addons supported in your Kubernetes distribution?

© Copyright 2024 Zenika. All rights reserved



Lab 2.3: Network solution

© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIMES



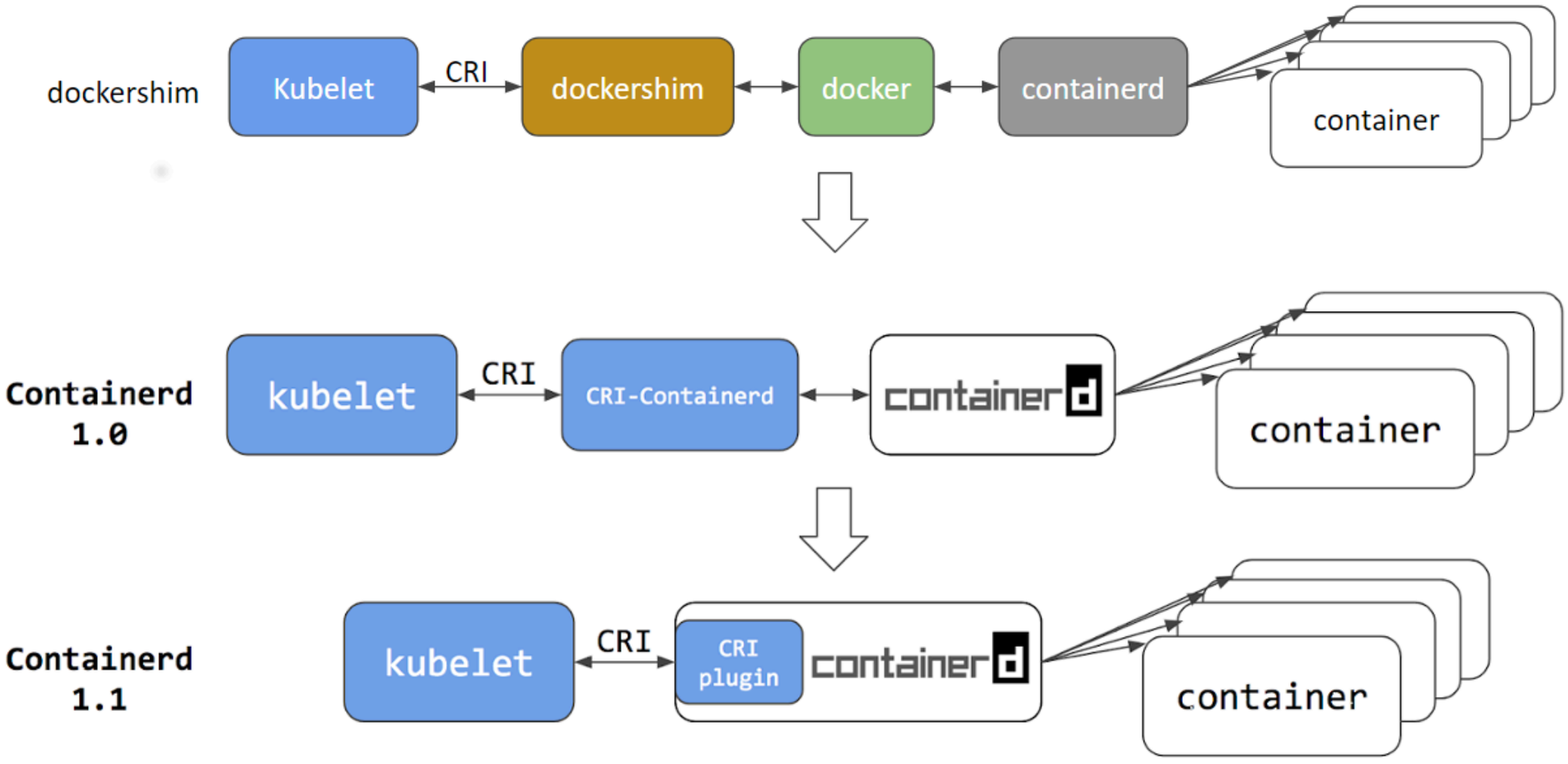
At the beginning of the Kubernetes project, Docker was the standard and only supported container runtime. With the OCI standards and other container runtimes being created, the community decided to create the Container Runtime Interface (CRI).

This allowed new compatible runtimes to be created and used with Kubernetes:

- Containerd
- CRI-O

© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIMES - CONTAINERD



© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIMES - CRI-O



cri-o was designed specifically to be used as the Kubernetes Container Runtime.

The project began at Red Hat in 2016.

cri-o has been used to run OpenShift Online Platform since August 2018.

It can use any OCI runtime that implements the OCI runtime spec and defaults to using runc.

© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIMES - LOW LEVEL



There are a few alternate low level runtimes available for Kubernetes

- gvisor: a container runtime using a userspace implementation of syscalls to isolate untrusted/unsafe workload from the kernel
- KataContainers: an hypervisor based runtime

© Copyright 2024 Zenika. All rights reserved

RUNTIMECLASSES



To use one of these custom low level runtimes:

- A k8s *RuntimeClass* should be defined
- The CRI must be configured to use it along the regular `runc`

Example *RuntimeClass* for gvisor:

```yaml
apiVersion: node.k8s.io/v1beta1
kind: RuntimeClass
metadata:
  name: gvisor
handler: runsc
---
apiVersion: v1
kind: Pod
metadata:
  name: gvisor-pod
spec:
  runtimeClassName: gvisor
  [...]
```


© Copyright 2024 Zenika. All rights reserved

RUNTIMECLASSES - CONTAINERD CONFIGURATION



Declaration of the custom handler (`runsc` for gvisor) in the containerd configuration.

`/etc/containerd/config.toml`

```
version = 2
[plugins."io.containerd.runtime.v1.linux"]
  shim_debug = true
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc]
  runtime_type = "io.containerd.runc.v2"
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runsc]
  runtime_type = "io.containerd.runsc.v1"
```


© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIMES - KUBELET (1/2)



kubelet is the agent that runs on each nodes and manage Pods and container lifecycle. It comes with a default configuration from the Kubernetes distribution, but may be configured.

```
$ kubelet -h
Usage:
  kubelet [flags]

Flags:
      --alsologtostderr                  log to standard error as well as files
      --bootstrap-kubeconfig string      Path to a kubeconfig file that will be used to get
client certificate for kubelet. If the file specified by --kubeconfig does not exist, the
bootstrap kubeconfig is used to request a client certificate from the API server. On success, a
kubeconfig file referencing the generated client certificate and key is written to the path
specified by --kubeconfig. The client certificate and key file will be stored in the directory
pointed by --cert-dir.
      [...]
```


© Copyright 2024 Zenika. All rights reserved

CONTAINER RUNTIMES - KUBELET CRI-O



kubeadm will try to automatically detect the container runtime on Linux nodes by scanning through a list of well known domain sockets.

The detectable runtimes and the socket paths, that are used, can be found in the table below:

Runtime	Domain Socket
containerd	/var/run/containerd/containerd.sock
CRI-O	/var/run/crio/crio.sock

If there are more than one installed on the provisioned node, specify the `--cri-socket` argument to `kubeadm init`.

© Copyright 2024 Zenika. All rights reserved

DNS (1/2)



kubedns is a custom component used to provide service name resolution inside the cluster.

As of Kubernetes 1.10 this component was made pluggable and CoreDNS introduced as beta.

Started in Kubernetes 1.11, CoreDNS became the default for kubeadm replacing kubedns.
kubedns is still used on GKE.

It's still possible to use kubedns prior to v1.30

e.g with kubeadm:

```
kubeadm init --feature-gates=CoreDNS=false
```


© Copyright 2024 Zenika. All rights reserved

DNS (2/2)



CoreDNS provides new features:

- Making an alias for an external name
- Adding an arbitrary entry inside the cluster domain

© Copyright 2024 Zenika. All rights reserved



Lab 2.4: DNS

© Copyright 2024 Zenika. All rights reserved

STORAGE



With Kubernetes, storage is provided to pods by PV/PVC.

To satisfy the PVCs, at least one StorageClass is needed.

A StorageClass definition must contain at least these informations:

- `provisioner`: the plugin used for provisioning PVs. This field is mandatory.
- `parameters`: used to pass custom parameters to the underlying plugin. These are provisioner specific.

© Copyright 2024 Zenika. All rights reserved

STORAGE - PROVISIONERS



There are many options when choosing a storage provisioner

- On premise:
  - Cinder (OpenStack block storage)
  - GlusterFS / CephFS / Rook
  - Portworx / StorageOS
  - NetApp Trident
  - NFS
  - ...

© Copyright 2024 Zenika. All rights reserved

STORAGE - PROVISIONERS CLOUD



- Cloud solutions:
  - AWS EBS, EFS
  - AzureDisk, AzureFile
  - GCEPersistentDisk

© Copyright 2024 Zenika. All rights reserved

STORAGE - STORAGECLASS EXAMPLE



GlusterFS StorageClass example:

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: slow
provisioner: kubernetes.io/glusterfs
parameters:
  resturl: "http://127.0.0.1:8081"
  clusterid: "630372ccdc720a92c681fb928f27b53f"
  restauthenabled: "true"
  restuser: "admin"
  secretNamespace: "default"
  secretName: "heketi-secret"
  gidMin: "40000"
  gidMax: "50000"
  volumetype: "replicate:3"
```


© Copyright 2024 Zenika. All rights reserved

STORAGE - RECLAIM POLICY



- When a volume is no longer needed, the associated PVC can be deleted
- The ReclaimPolicy of the PersistentVolume define what happens to the PersistentVolume once freed:
- Freed PVs can be:
  - Retained: the PV is no longer used but can't be bound to another PersistentVolumeClaim until an operator release it
  - Recycled (deprecated): the PersistentVolume is cleaned, i.e. datas are deleted. Once cleaned it can be associated to a new PVC
  - Deleted: the PersistentVolume is deleted (e.g. AzureDisk, GC Disk, AWS EBS...)
- Default is `delete`

© Copyright 2024 Zenika. All rights reserved

STORAGE - ACCESS MODES



- ReadWriteOnce (RWO): the volume can be mounted as read-write by a single node
- ReadOnlyMany (ROX): the volume can be mounted read-only by many nodes
- ReadWriteMany (RWX): the volume can be mounted as read-write by many nodes
- ReadWriteOncePod (RWOP): the volume can be mounted as read-write by a single pod (stable since v1.29)

⚠️ A volume can only be mounted using one access mode at a time, even if it supports multiple modes

© Copyright 2024 Zenika. All rights reserved

CSI DRIVERS



`StorageClass` plugins were previously developed in Kubernetes tree. This is not sustainable and acceptable regarding vendors multiplicity.

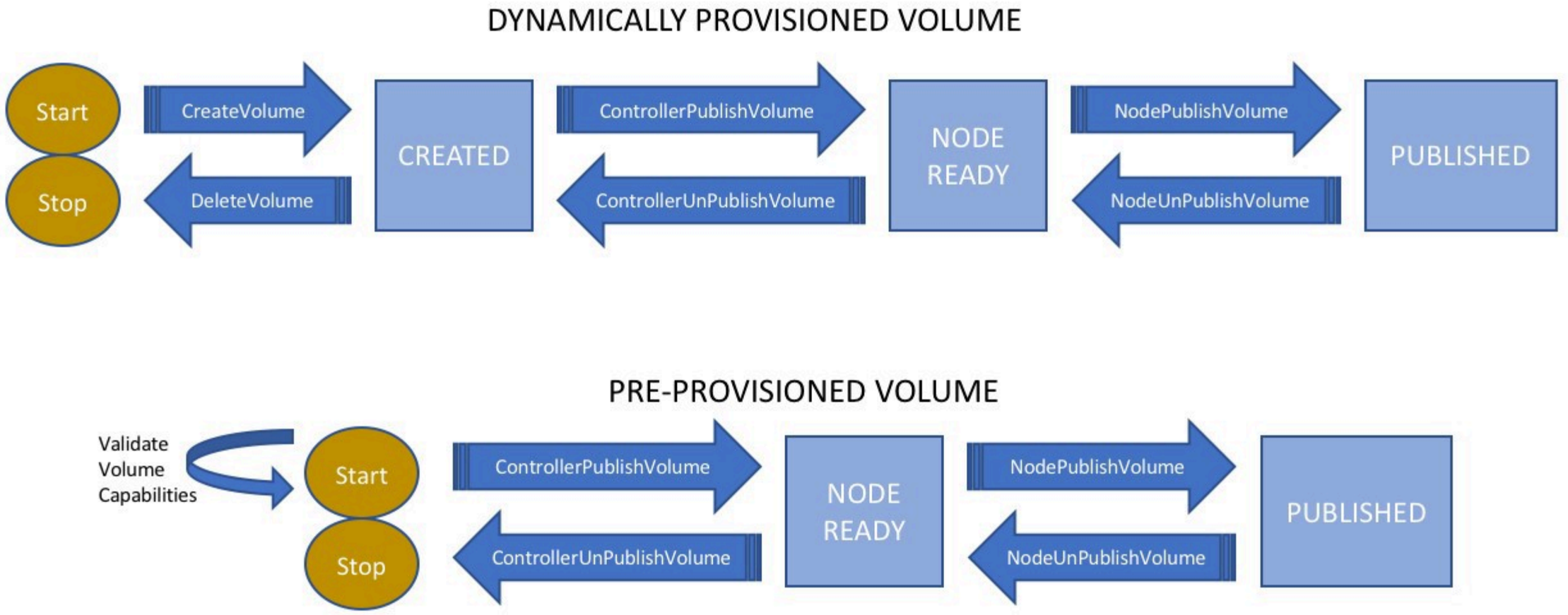
CSI defines an API to interact with volume providers:

- Create/Delete volumes
- Mount/Unmount volumes
- Create snapshots
- Expand volumes

> Full specification available here

© Copyright 2024 Zenika. All rights reserved

CSI VOLUMES LIFECYCLE



-- See https://medium.com/@click2cloud.contact/container-storage-interface-27282289dbc4

© Copyright 2024 Zenika. All rights reserved

CSI DRIVER ARCHITECTURE



- Controller Plugin: interacts with the storage provider to create, delete and gather information regarding volumes
- Node Plugin: handles the logic needed to make the volume accessible from the Node where the consuming workload is deployed

© Copyright 2024 Zenika. All rights reserved

CSI DRIVERS EXAMPLES



Name	Access modes	Dynamic	Other features
CephFS	Read/Write Multiple Pods	✅	Expansion, Snapshot, Clone
Cinder	Depends on the storage backend used	✅, if storage backend supports it	Raw Block, Snapshot, Expansion
GlusterFS	Read/Write Multiple Pods	✅	Snapshot
NFS	Read/Write Multiple Pods	✅	Expansion
AWS EBS	Read/Write One Pod	✅	Expansion, Snapshot, Clone
...	...	...	...

CSI Drivers list: https://kubernetes-csi.github.io/docs/drivers.html

© Copyright 2024 Zenika. All rights reserved



Lab 2.5: Storage

© Copyright 2024 Zenika. All rights reserved

CSI DRIVER DEVELOPMENT



To help with the development of CSI Drivers, the SIG (Special Interest Group) Storage provides standard side-car containers to handle k8s in-cluster communications.

The CSI Driver developper just need to implement the contract defined in the CSI Spec. The underlying code will be called by the side-car containers.

-- See: https://kubernetes-csi.github.io/docs/sidecar-containers.html#kubernetes-csi-sidecar-containers

© Copyright 2024 Zenika. All rights reserved

CSI DRIVER CAPABILITIES



CSI drivers may publish a list of capabilities to Kubernetes in order to expose their supported operations

- PluginCapability:
  - CONTROLLER_SERVICE
  - VolumeExpansion
- ControllerServiceCapability:
  - CREATE_DELETE_VOLUME
  - PUBLISH_UNPUBLISH_VOLUME
  - CREATE_DELETE_SNAPSHOT
  - CLONE_VOLUME
- NodeServiceCapability:
  - STAGE_UNSTAGE_VOLUME

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



CONFIGURATION AND OPERATIONAL MAINTENANCE

© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



- Architecture
- Installation
- *Configuration and operational maintenance*
- Cluster Upgrade
- Day to day actions
- Extensibility: Operators, CRD and API Servers
- Service Mesh, Security

© Copyright 2024 Zenika. All rights reserved

CHAPTER CONTENT



- Logs centralization
- Metrics centralization
- Backup / Restore
- Tools for capacity planning (node resources consumption, namespaces, requests, limits)
- Garbage Collection
- Connectivity of a Kubernetes cluster with the rest of your infrastructure
- LoadBalancers
- Ingress
- Schedulers
- Troubleshooting the cluster

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - WHY



Why should we centralize logs?

Kubernetes deploy pods an available nodes:

- There is no way to predict where your application will be running
- ssh into the machine or use a network share to access the logs are not viable options

> Instead, each running process writes its event stream, unbuffered, to stdout [...] each process' stream will be captured by the execution environment, collated together with all other streams from the app, and routed to one or more final destinations for viewing and long-term archival.
>
> -- https://12factor.net/logs

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - HOW



Kubernetes doesn't provide this out of the box, but:

- Kubernetes hosted by cloud providers usually come with their own solution:
  - Cloud Logging for Google Cloud
  - Log Analytics for Azure
- Some distributions provide it:
  - OpenShift provides an integrated EFK stack
  - Rancher 2.0 provides a pluggable architecture and supports many solutions (embedded ElasticSearch, external ElasticSearch, Splunk, Kafka, Syslog)
- On-premises you are on your own (nearly):
  - There are many docs and articles explaining how to use EFK with Kubernetes or even ECK, an Elastic Operator for Kubernetes.

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - WHAT

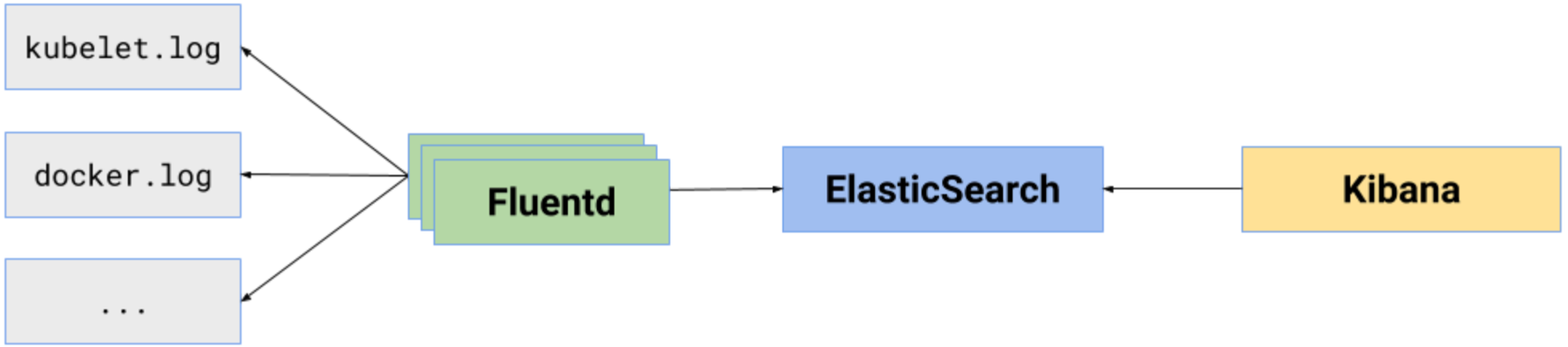


There are multiple logs producers to address when you run Kubernetes:

- Kubernetes components:
  - Kubelet/Node
  - Kube API Server
  - Kube Controller manager
  - Kube Scheduler
- Kubernetes addons (deployed as usual workloads most of the time):
  - Network plugins
  - Ingress controllers
- Hosted workloads
  - All user deployed pods

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - EFK



These components are used to:

- Fluentd: Collect logs data from files or other sources...
- ElasticSearch: Store, index and expose logs
- Kibana: Display logs, search, create specific dashboards

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - EFK DEPLOYMENT



- Fluentd should be deployed as a DaemonSet
- ElasticSearch can be deployed:
  - inside the cluster as a StatefulSet or as an Operator (covered in chapter 6)
  - outside the cluster, using an externally managed instance
- Kibana can be deployed as a standard Deployment

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - FLUENTD CONFIGURATION



Fluentd configuration uses mainly three directives:

- source: determines the input sources
- match: determines the output destinations
- filter: determines the event processing pipelines

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - FLUENTD SOURCE



Example source configuration for container logs:

```
# CRI Log Example:
# 2016-02-17T00:04:05.931087621Z stdout F [info:2016-02-16T16:04:05.930-08:00] Some log text
here
<source>
    @id fluentd-containers.log
    @type tail
    path /var/log/containers/*.log
    pos_file /var/log/es-containers.log.pos
    tag raw.kubernetes.*
    read_from_head true
    <parse>
        @type multi_format
        <pattern>
            format /^(?<time>.+) (?<stream>stdout|stderr) [^ ]* (?<log>.*)$/
            time_format %Y-%m-%dT%H:%M:%S.%N%:z
        </pattern>
    </parse>
</source>
```

Source: https://github.com/kubernetes-sigs/instrumentation-addons/tree/master/fluentd-elasticsearch

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - FLUENTD FILTER



Example filter configuration:

```
# Enriches records with Kubernetes metadata
<filter kubernetes.**>
    @type kubernetes_metadata
</filter>
```

The Kubernetes fluentd plugin is used to extract the namespace, pod name & container name which are added to the log message as a Kubernetes field object & the Docker container ID is also added under the docker field object.

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - FLUENTBIT



As an alternative deployment pattern, you could use Fluentbit along with Fluentd Fluentbit:

- Designed to be a very lightweight forwarder
- Deployed on each nodes
- Collects and forwards logs to a Fluentd instance
- Adds pods/namespace/container metadata with the right plugin

The Fluentd instance then acts as an aggregator.

© Copyright 2024 Zenika. All rights reserved

LOGS CENTRALIZATION - CLUSTER



Additional tasks for configuring fluentd:

- Check the container runtime log config
- Check kubelet log config
- Check control plane components log config
- Write a ConfigMap with all needed files

© Copyright 2024 Zenika. All rights reserved



Lab 3.1: EFK Deployment

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION



Metrics are split into two pipelines for monitoring Kubernetes:

- A core metrics pipeline:
  - Used by core components (e.g. scheduler and horizontal pod autoscaling)
  - Not meant to be integrated with third-party monitoring systems
  - Implemented by Kubernetes components (Kubelet/cAdvisor, metrics-server)
  - Served by the API Server
- A monitoring pipeline:
  - Used for collecting various metrics and exposing them to end-users
  - Provided by third-party components
  - Opensource Kubernetes does not ship with a monitoring pipeline
  - Can be served by the API Server

© Copyright 2024 Zenika. All rights reserved

METRICS SERVER



Metrics server goals:

- Provide access to cluster metrics the same way as other API objects
- Storing metrics as their lifecycle differs from other objects and shouldn't be stored in etcd
- The metrics server adds the `metrics.k8s.io/v1beta1` API to your cluster using the aggregation layer (covered in Chapter 6)

© Copyright 2024 Zenika. All rights reserved

METRICS SERVER



The `kubectl` command uses this service to gather Pod and Node metrics.

```
kubectl top nodes
kubectl top pods
```

Other considerations:

- Metrics are required for some Kubernetes components (HPA, scheduler, kubectl top), so it will be run by default in all clusters.
- Only one instance running on the cluster, vertically autoscaled by addon-resizer
- No long term storage of metrics, such use case should be addressed with a Monitoring pipeline

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - MONITORING PIPELINE - WHY?



Why do you need a separated monitoring pipeline?

- Flexible monitoring pipeline, as core Kubernetes components do not need to rely on it
- Data collected by the monitoring pipeline may differ from those used by Kubernetes cluster
- Expose custom metrics to Horizontal Pod Autoscaler

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - COLLECTED DATAS



Example collected datas:

- core system metrics: CPU / Mem
- non-core system metrics: IO / NetIO / FS / Processus
- service metrics from user application containers
- service metrics from Kubernetes infrastructure containers; these metrics are exposed using Prometheus instrumentation

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS OVERVIEW



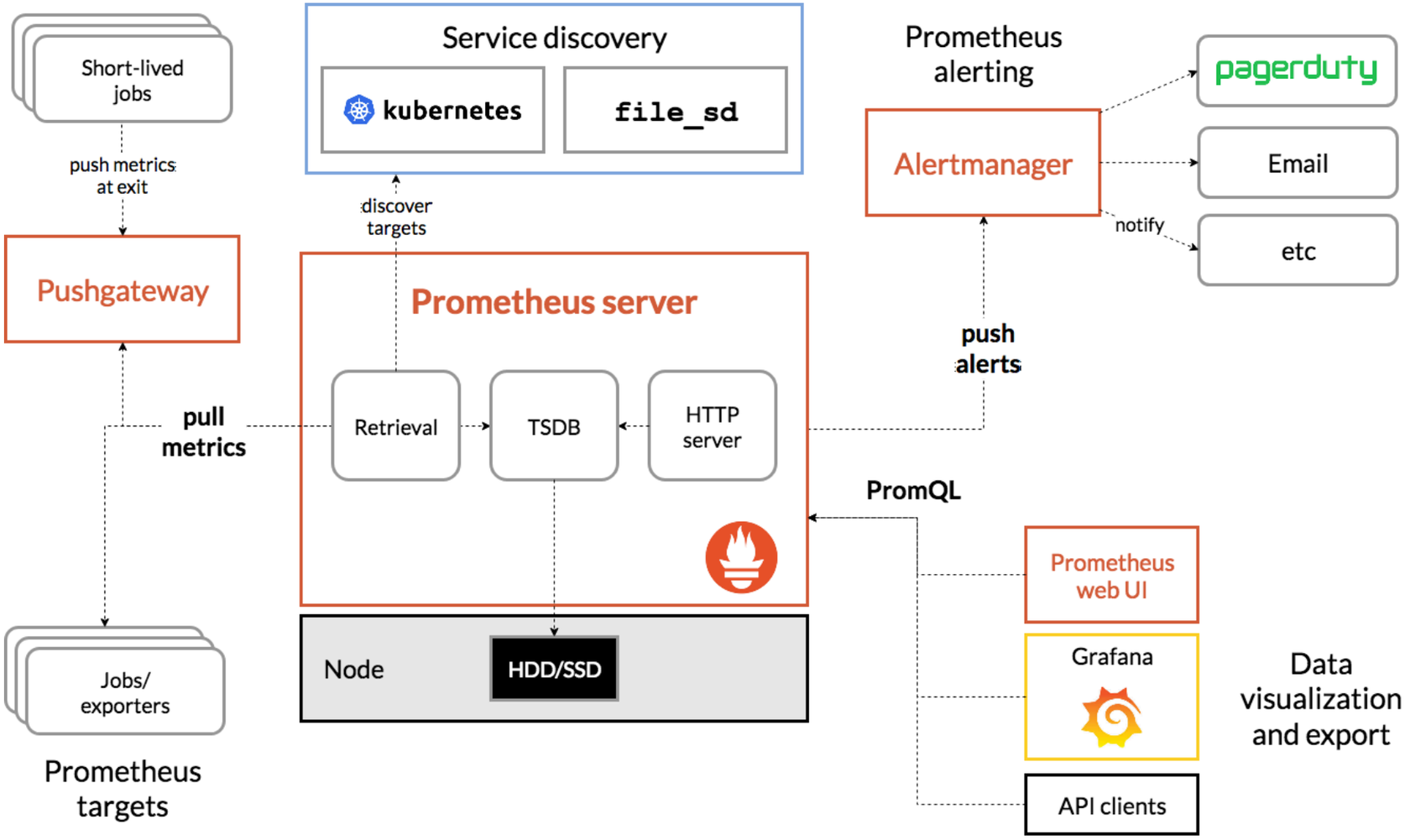
Prometheus is an Opensource project comprising many subprojects:

- A timeseries database
- A dedicated query language: PromQL
- A "standard" metric format: OpenMetrics
- Exporters and libraries to expose metrics from many languages/frameworks/applications
- An Alertmanager

Metrics are scraped (or pulled) from a http(s) endpoint

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS



Source: https://prometheus.io/docs/introduction/overview/

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS METRIC TYPES

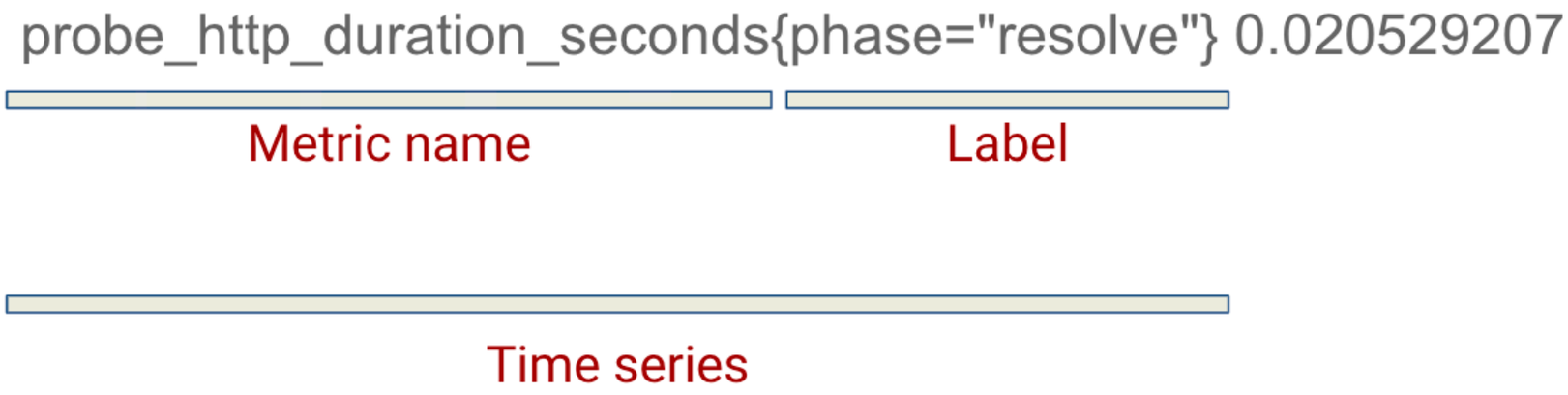
Basic counters	Sampling counters
Counter	Histogram
Gauge	Summary

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS METRIC EXAMPLE



```
# HELP http_requests_total Total number of HTTP requests made.
# TYPE http_requests_total counter
http_requests_total{code="200",handler="graph",method="get"} 2
http_requests_total{code="200",handler="label_values",method="get"} 2
http_requests_total{code="200",handler="prometheus",method="get"} 1072
http_requests_total{code="200",handler="query",method="get"} 2
http_requests_total{code="200",handler="static",method="get"} 46
```



© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS SOURCES

- Exporters:
  - Hardware/Host: Node exporter, IBM Z HMC exporter, ...
  - Webserver: Apache, nginx, ...
  - Messaging systems: Kafka, RabbitMQ, ...
  - Storage: Ceph, Gluster, Hadoop HDFS, ...
  - Databases: Mongodb, MSSQL, MySQL, Redis, PostgreSQL, Oracle DB, ...
- Software exposing Prometheus metrics:
  - Kubernetes components: Kube-apiserver, kube-state-metrics, kubelet, kube-controller-manager, kube-scheduler
  - Others: etcd, Minio, Traefik, FluxCD, ...

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS DEPLOYMENT



The recommended way to deploy Prometheus on Kubernetes is the Prometheus Operator. It uses Custom Resource Definitions (CRD) to declare desired state:

- Prometheus: defines a desired Prometheus setup to run in the cluster
- ServiceMonitor: defines how a dynamic set of services should be monitored using a selector
- AlertManager: defines a desired Alertmanager setup to run in the cluster
- PrometheusRule: defines a desired Prometheus rule to be consumed by one or more Prometheus instances.

We'll cover Operators and CRD on Chapter 6

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS CRD



Example Prometheus CRD:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: Prometheus
metadata:
  name: prometheus
spec:
  serviceMonitorSelector:
    matchLabels:
      team: frontend
  version: v2.5.0
```

serviceMonitorSelector lets you specify which ServiceMonitor will be scraped by this instance

© Copyright 2024 Zenika. All rights reserved

METRICS CENTRALIZATION - PROMETHEUS SERVICE MONITOR



Example Service Monitor CRD:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: example-app
  labels:
    team: frontend
spec:
  selector:
    matchLabels:
      app: example-app
  endpoints:
  - port: web
```

- The label selector is used to select Services and their underlying Endpoint objects
- The endpoint port specifies the port on which the metrics are exposed

© Copyright 2024 Zenika. All rights reserved



Lab 3.2: Prometheus Deployment

© Copyright 2024 Zenika. All rights reserved

BACKUP / RESTORE (1/2)



Why?

- Disaster recovery: to be able to restore the cluster state if the etcd cluster gets corrupted or lost
- Upgrade tests: to create full duplicated environments to test cluster or critical components upgrade or restore a partial state to test
- Environment replication: restore full or partial state to replicate some your setup on another cluster
- Scheduled migration: to migrate from one cluster to another

© Copyright 2024 Zenika. All rights reserved

BACKUP / RESTORE (2/2)



What?

- Root certificate and key
- Etcd datas
- Volume datas

How?

- With stateless workloads:
  - the common way is to snapshot etcd state
  - another possibility is to export all kube resources (pods, deploy, ...)
- However, with stateful workloads, volume datas should also be backed up.

© Copyright 2024 Zenika. All rights reserved

BACKUP WITH ETCD (1/2)



Make sure you're able to connect to etcd:

```
$ etcdctl --cacert=ca.crt --cert=client.crt --key=client.key \
   --endpoints https://ETCD_HOSTNAME:2379 endpoint status
```

- `--cacert=ca.crt`: CA certificate used to trust etcd https endpoint
- `--cert=client.crt`, `--key=client.key`: client key pair, signed by the CA
- `--endpoints https://ETCD_HOSTNAME:2379`: etcd is reachable at `ETCD_HOSTNAME:2379`
- `endpoint status`: Get a status of the cluster (members, state, ...)

© Copyright 2024 Zenika. All rights reserved

BACKUP WITH ETCD (2/2)



Then create a snapshot of etcd state:

```
$ etcdctl --endpoints ${ENDPOINT} snapshot save snapshot.db
```

And verify it:

```
$ etcdutl --write-out=table snapshot status snapshot.db
+----------+----------+------------+------------+
|   HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+----------+----------+------------+------------+
| fe01cf57 |       10 |          7 | 2.1 MB     |
+----------+----------+------------+------------+
```

ℹ️ Certificate parameters are omitted for clarity

© Copyright 2024 Zenika. All rights reserved

RESTORE WITH ETCD



To restore, you should use the `etcdutl` command which operates on directory, and not with a running etcd instance.

The following command initializes a data directory for a new with `snapshot.db`, it must be run on each host

```
$ etcdutl snapshot restore snapshot.db --data-dir /var/lib/etcd-restore
```

Then etcd is ready to be started with this command (again on each host):

```
$ etcd \
  --name m1 \
  --listen-client-urls http://host1:2379 \
  --advertise-client-urls http://host1:2379 \
  --listen-peer-urls http://host1:2380 &
```


© Copyright 2024 Zenika. All rights reserved

BACKUP/RESTORE WITH VELERO (VMWARE)



Velero gives you tools to back up and restore your Kubernetes cluster resources and persistent volumes. Velero lets you:

- Take backups of your cluster and restore in case of loss.
- Copy cluster resources to other clusters.
- Replicate your production environment for development and testing environments.

Velero consists of:

- A server that runs on your cluster
- A command-line client that runs locally

© Copyright 2024 Zenika. All rights reserved

BACKUP/RESTORE - VELERO COMPATIBILITY



Velero will use a storage provider to store the backups:

- AWS S3
- Azure Blob Storage
- Google Cloud Storage

And supports these storage solutions for volume snapshots:

- AWS EBS
- Azure Managed Disks
- Google Compute Engine Disks
- Restic
- Portworx
- DigitalOcean

© Copyright 2024 Zenika. All rights reserved

BACKUP/RESTORE - VELERO INSTALLATION



Get the latest release for your platform. This package includes:

- `velero` cli command
- MinIO deployment `examples/minio/00-minio-deployment.yaml` for local S3-compatible storage

To install velero with minio:

```
$ cat <<EOF > credentials-velero
[default]
aws_access_key_id = minio
aws_secret_access_key = minio123
EOF
$ kubectl apply -f examples/minio/00-minio-deployment.yaml
$ velero install \
    --provider aws \
    --bucket velero \
    --secret-file ./credentials-velero \
    --use-volume-snapshots=false \
    --backup-location-config \
        region=minio,s3ForcePathStyle="true",s3Url=http://minio.velero.svc:9000
```


© Copyright 2024 Zenika. All rights reserved

BACKUP/RESTORE - VELERO USAGE



Create a backup for any object that matches the `app=nginx` label selector:

```
$ velero backup create nginx-backup --selector app=nginx
```

Create regularly scheduled backups based on a cron expression using the `app=nginx` label selector:

```
$ velero schedule create nginx-daily --schedule="0 1 * * *" --selector app=nginx
```

Restore your lost resources:

```
$ velero restore create --from-backup nginx-backup
```


© Copyright 2024 Zenika. All rights reserved



Lab 3.3: Backup/Restore

© Copyright 2024 Zenika. All rights reserved

TOOLS FOR CAPACITY PLANNING



The easiest way to see cluster free and used capacity is to use `kubectl top nodes` or `kubectl top pods` when Metrics server is deployed.

```
NAME       CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
worker-1   39m          4%     1216Mi          46%
worker-2   41m          4%     669Mi           25%
worker-3   55m          5%     1342Mi          50%
```

However, this doesn't help with capacity planning:

- Only shows instant usage
- Doesn't show or enforce threshold on resources used

© Copyright 2024 Zenika. All rights reserved

TOOLS FOR CAPACITY PLANNING - REQUESTS/LIMITS

To manage Capacity planning, Operators need workloads to declare necessary resources (CPU, memory).

To do so, Kubernetes API uses two concepts:

- Requests: Describes the minimum amount of compute resources required
- Limits: Describes the maximum amount of compute resources allowed

© Copyright 2024 Zenika. All rights reserved

TOOLS FOR CAPACITY PLANNING - REQUESTS



- Requests
  - Used by the Scheduler to ensure Pod fits on a Node
  - The Pod remains in the PENDING state as long as the resource request cannot be satisfied

```
$ kubectl get pods
NAME            READY   STATUS    RESTARTS   AGE
too-much-cpu    0/1     Pending   0          4s
```

```
$ kubectl describe pods | tail -n 4
Events:
  Type     Reason            Age                 From               Message
  ----     ------            ----                ----               -------
  Warning  FailedScheduling  11s (x8 over 75s)   default-scheduler  0/3 nodes are available: 3
Insufficient cpu.
```


© Copyright 2024 Zenika. All rights reserved

TOOLS FOR CAPACITY PLANNING - LIMITS



- Limits
  - Used by the Kubelet to enforce thresholds on resources used
  - The Pod might be TERMINATED if memory usage goes over the defined limit
  - The Container CPU use is being throttled when CPU usage goes over the defined limit

```
$ kubectl describe po | grep State -A4
    State:          Waiting
      Reason:       CrashLoopBackOff
    Last State:     Terminated
      Reason:       OOMKilled
      Exit Code:    1
      Started:      Tue, 11 Dec 2022 15:28:34 +0100
      Finished:     Tue, 11 Dec 2022 15:29:10 +0100
```


© Copyright 2024 Zenika. All rights reserved

TOOLS FOR CAPACITY PLANNING - REQUESTS/LIMITS UNITS

For Memory, limits and requests are expressed in bytes as a plain integer or as a fixed-point integer using one of these suffixes: `E, P, T, G, M, K`. Or the power-of-two equivalents: `Ei, Pi, Ti, Gi, Mi, Ki`.

For example, the following represent roughly the same value:

```
128974848, 129e6, 129M, 123Mi
```

For CPU, limits and requests are expressed in CPU units. One CPU unit is equivalent to 1 core.

It can be expressed as a millicpu `100m` or with a decimal `0.1`.

© Copyright 2024 Zenika. All rights reserved

TOOLS FOR CAPACITY PLANNING - REQUESTS/LIMITS EXAMPLE

Example Pod defining CPU/Memory requests and limits:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: too-much-cpu
spec:
  containers:
  - name: too-much-cpu
    image: centos
    resources:
      requests:
        cpu: "500m"
        memory: "128Mi"
      limits:
        cpu: "1.5"
        memory: "1Gi"
```


© Copyright 2024 Zenika. All rights reserved

TOOLS FOR CAPACITY PLANNING - QOS DEFINITION



When available resources on a Node become scarce, Kubernetes might evict workloads to ensure stability of the Node.

To make decisions about which Pods to evict, Kubernetes assigns QoS classes to them at their creation.

There are 3 QoS classes:

- Guaranteed: Every Container in the Pod has equal value for limits and requests (CPU and Memory)
- Burstable: Doesn't meet Guaranteed criteria but at least one Container has a CPU or Memory request
- BestEffort: Containers in the Pod do not have any memory or CPU limits or requests

© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION - NODE CAPACITY



Each node has a finite amount of resources which is shown when using:

```
$ kubectl describe node worker-1
Capacity:
  cpu:                2
  memory:             3969472Ki
  pods:               110
Allocatable:
  cpu:                2
  memory:             3867072Ki
  pods:               110
```

- `Capacity` is the total amount of resources available on the Node
- `Allocatable` is the amount of resources available for Pods

The difference is the amount of resources reserved for system daemons running on the Node.

See https://kubernetes.io/docs/tasks/administer-cluster/reserve-compute-resources/

© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION (1/2)



Node garbage collection is done by the kubelet and occurs on 5 condition types:

- `memory.available`
- `nodefs.available`
- `nodefs.inodesFree`
- `imagefs.available`
- `imagefs.inodesFree`

- `nodefs` is used by the kubelet for volumes, daemon logs, etc
- `imagefs` is used by the container runtime for storing images and container writable layers

© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION (2/2)



There are two kinds of eviction types:

- Soft: no action is taken until a grace period has been exceeded
  - Grace period is configured with flag `eviction-soft-grace-period`
  - When soft evicting Pods, the `eviction-max-pod-grace-period` flag describes the grace period to use when terminating Pods.
- Hard: no grace period, immediate action to reclaim the starved resource is taken

© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION - CONFIGURATION



Eviction thresholds are configurable with a flag on kubelet startup command:

```
--eviction-hard=memory.available<500Mi,
                nodefs.available<1Gi,imagefs.available<10Gi
```

or with the kubelet config file:

```
kind: KubeletConfiguration
apiVersion: kubelet.config.k8s.io/v1beta1
evictionHard:
    memory.available: "500Mi"
    nodefs.available: "1Gi"
    imagefs.available: "10Gi"
```


© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION - DEFAULT VALUES



The kubelet has the following default hard eviction thresholds:

- `memory.available` < 100Mi
- `nodefs.available` < 10%
- `nodefs.inodesFree` < 5%
- `imagefs.available` < 15%

© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION - NODE CONDITION



When an eviction threshold has been met (soft or hard), the kubelet reports the corresponding Node status

Node Condition	Eviction Signal
`MemoryPressure`	`memory.available`
`DiskPressure`	`nodefs.available`, `nodefs.inodesFree`, `imagefs.available`, or `imagefs.inodesFree`

© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION - RECLAIMING RESOURCES



The kubelet then tries to reclaim resources with the following process

1. The kubelet removes dead Pods and theirs containers, and deletes all unused images
2. If insufficient resources were reclaimed, the kubelet will now evict end-user Pods

- Pods are sorted according to their QoS and on their total disk usage: local volumes + logs & writable layer of all their containers.

ℹ️ Having both `nodefs` and `imagefs` on the **Node**, only the actions concerning the right filesystem are taken e.g.: for `imagefs` eviction threshold, it deletes all unused images

© Copyright 2024 Zenika. All rights reserved

GARBAGE COLLECTION - QOS RESULT



Then kubelet ranks and evicts Pods in the following order:

- BestEffort or Burstable Pods whose usage of a starved resource exceeds its request
- Guaranteed pods and Burstable pods whose usage is beneath requests are evicted last

This mechanism only works with already scheduled workloads but doesn't enforce preemption for pending Pods.

© Copyright 2024 Zenika. All rights reserved



Lab 3.4 & 3.5

© Copyright 2024 Zenika. All rights reserved

CONNECTIVITY WITH THE REST OF YOUR INFRASTRUCTURE



To expose services outside the cluster, some interactions with external infrastructure is needed. But Kubernetes provides some tools to help you:

- LoadBalancers: IP/Protocol/Port routing
  - Cloud
  - On premises
- Ingress: HTTP Header routing
- Gateway API: limited to HTTP Header routing and GRPC requests for now (but next: tcp, udp)

© Copyright 2024 Zenika. All rights reserved

LOADBALANCERS - CLOUD



- Many cloud providers propose an integration of Kubernetes with their internal load balancing features (AWS, Azure, GCE)
- Set `.spec.type` to LoadBalancer, then Kubernetes will interact with the Cloud Provider api and provision automatically a LoadBalancer
- Note:
  - This provisioning is asynchrone
  - The external IP will appear as `"Pending"` while the LoadBalancer is being provisioned
  - Technically, the LoadBalancer will redirect the traffic to the service using a NodePort (you don't have to configure the NodePort, it is done implicitly)

© Copyright 2024 Zenika. All rights reserved

LOADBALANCERS - ON PREMISES



There are few projects which aim at providing LoadBalancer service type (with the appropriate controller) for On Premises Kubernetes deployments:

- Metallb
  - Already used by serveral people and companies but still a beta system
- Cilium
- F5
  - Relies on specific networking to route traffic from BIG IP to the cluster
- HAproxy

© Copyright 2024 Zenika. All rights reserved

LOADBALANCERS - LIMITATIONS



- Exposing services through NodePort is not really elegant
- You can manually configure external load balancers not managed by Kubernetes (HAProxy, F5, ...) ... only if you have access to such load balancers
- Kubernetes services work at the *TCP* level, hence cannot provide features relying on cookies and other *HTTP* features
- *Ingress* can! 😀

© Copyright 2024 Zenika. All rights reserved

INGRESS



- What is an Ingress?
- Which Ingress Controller to choose?
- How to deploy/manage them?

© Copyright 2024 Zenika. All rights reserved

INGRESS DESCRIPTION



Ingress is used to manage traffic coming

- from outside the cluster
- to service inside the cluster

It provides SSL Termination, loadbalancing, name-based virtual hosting

- Ingress is the resource describing how the traffic should be handled
- The Ingress Controller watches Ingresses and updates the Reverse Proxy configuration

© Copyright 2024 Zenika. All rights reserved

EXAMPLE INGRESS



The following Ingress handles all requests

- with path `/my-app`
- header `Host: my-app.mydomain.com`
- and sends them to the service named `my-app`

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app
spec:
  rules:
  - host: my-app.mydomain.com
    http:
      paths:
      - path: /my-app
        pathType: Prefix
        backend:
          service:
            name: my-app
            port:
              number: 80
```


© Copyright 2024 Zenika. All rights reserved

WHICH INGRESS CONTROLLER (1/2)



There are many Ingress controllers available for Kubernetes:

- Traefik from Traefik Labs
- Contour from Vmware (now in CNCF)
- Istio from Google and IBM, in partnership with the Envoy team at Lyft
- Ambassador from Datawire
- Nginx from NGINX Inc
- Kong from KongHQ
- HAProxy from HAProxy Technologies
- F5 BIG-IP Controller from F5 Networks

© Copyright 2024 Zenika. All rights reserved

WHICH INGRESS CONTROLLER (2/2)



Name	Company	Opensource	Commercial support	Description
Traefik	Traefik Labs	✅	✅	Let's Encrypt, secrets, http2, websocket
Contour	CNCF	✅	-	Custom Ingress resource, many loadbalancing algorithms
Istio	Istio authors	✅	✅	More than a simple Ingress
Ambassador	Datawire	✅	✅	gRPC, http2, istio integration
Nginx	NGINX Inc	✅	✅	-
HAProxy	HAProxy Technologies	✅	✅	Many loadbalancing algorithms
F5 BIG-IP Controller	F5 Networks	❌	✅	-

A full-blown comparison is available here

© Copyright 2024 Zenika. All rights reserved

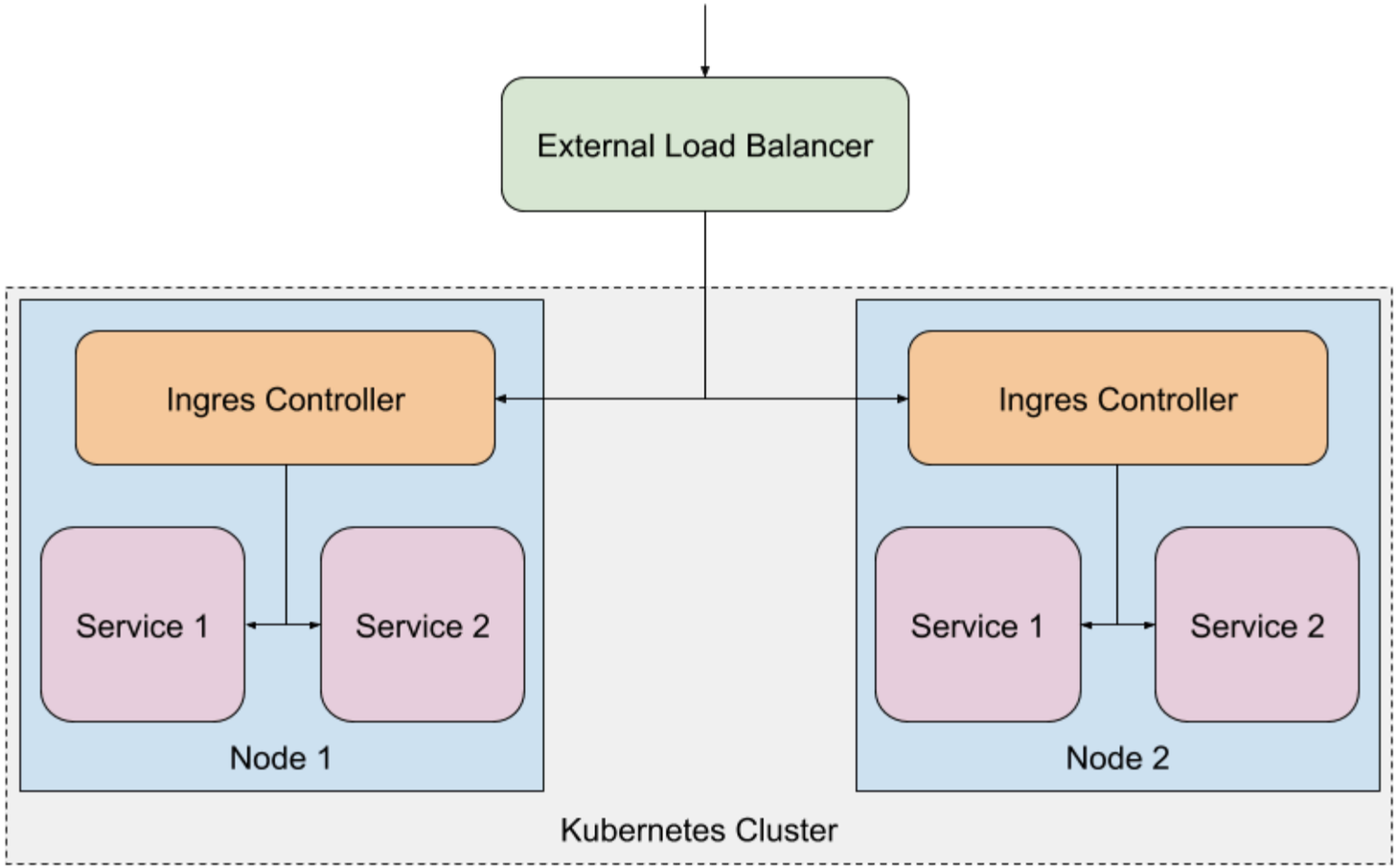
HOW TO CHOOSE



- Do you need specific protocols (http2, gRPC, tcp, ...)?
- Do you need find fine-grained traffic control (For A/B Testing, ...)?
- Is it already proof tested?
- Is it compatible with monitoring (Prometheus metrics export, ...)?
- Do you want to stick to vanilla Ingress resources?

© Copyright 2024 Zenika. All rights reserved

INTEGRATION WITH EXISTING INFRASTRUCTURE (1/3)



© Copyright 2024 Zenika. All rights reserved

INTEGRATION WITH EXISTING INFRASTRUCTURE (2/3)

Reference implementation architectures usually propose:

- A front LoadBalancer outside the cluster
- Distributing traffic to an Ingress Controller
  - Exposed as Deployment + NodePort Service or DaemonSet + HostPort container
- The Ingress Controller is able to distribute traffic across cluster services based on L7 informations (Host, Cookie, ...)
- The front load balancer can be used
  - to distribute traffic among nodes
  - in front of several Kubernetes instances when "cluster upgrade" means "create new and destroy old"

© Copyright 2024 Zenika. All rights reserved

INTEGRATION WITH EXISTING INFRASTRUCTURE (3/3)

To manage efficiently your TLS certificates you can:

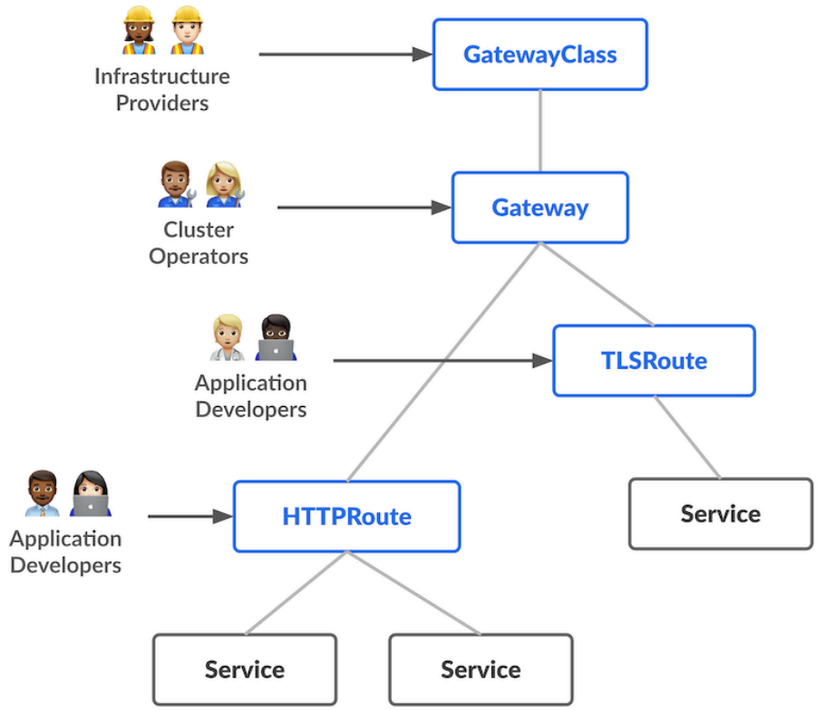
- Use CertManager
- Use a wildcard certificate and configure TLS offloading
  - on your front load balancer (outside the cluster)
  - on your Ingress controller (inside the cluster)
- Use Let's encrypt with your Ingress controller (if compatible)
- Do some tricky implementation like with Vault's PKI secrets engine & dynamic X.509 certificates ♥

© Copyright 2024 Zenika. All rights reserved

GATEWAY API

Due to the universality of *Ingress*, the configuration is limited, hence the use (abuse?) of annotations, or even specific resources (*CRD*) for some *IngressController*. It is also limited to the HTTP protocol.

To overcome these problems and allow a better separation of roles involved in Kubernetes management, a new solution is available: Kubernetes Gateway API



© Copyright 2024 Zenika. All rights reserved



Lab 3.6: Ingress

© Copyright 2024 Zenika. All rights reserved

SCHEDULING



The default Kubernetes scheduler can be aware of the Cluster Topology and spread workloads in order to increase resilience.

This mechanism uses:

- The `SelectorSpreadPriority` priority function
- The `topology.kubernetes.io/zone` and `topology.kubernetes.io/region` labels on the Cluster Nodes

When using a managed Kubernetes the labels should be set by your provider, when deploying your own cluster, you should make sure to set them.

-- See: https://kubernetes.io/docs/reference/kubernetes-api/labels-annotations-taints/#topologykubernetesioregion

© Copyright 2024 Zenika. All rights reserved

SCHEDULING



There are few ways to control how scheduler will assign Pods to Node:

- NodeSelector
  - Pod will be assigned to Nodes with a label matching selector
  - will be deprecated in favor of NodeAffinity
- NodeAffinity: can express requirement and/or preference
- PodAffinity: to colocate Pods
- PodAntiAffinity: to make sure two Pods won't be on the same Node

© Copyright 2024 Zenika. All rights reserved

NODE AFFINITY



There is two kind of affinities:

- `preferredDuringSchedulingIgnoredDuringExecution`
  - if no Node matches the affinity the Pod will still be deployed on another Node
- `requiredDuringSchedulingIgnoredDuringExecution`
  - if no Node matches the affinity the Pod will not be deployed and stay in Pending state

© Copyright 2024 Zenika. All rights reserved

NODE AFFINITY OPERATORS



Node selector example for affinity

```
nodeSelectorTerms:
  - matchExpressions:
      - key: kubernetes.io/env
        operator: In
        values:
          - qa
```

Available operators are: `In, NotIn, Exists, DoesNotExist, Gt, Lt`

© Copyright 2024 Zenika. All rights reserved

NODE AFFINITY EXAMPLE



Pod with Node affinity example

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values:
                  - eu-west1-b
  containers:
    - name: with-node-affinity
      image: k8s.gcr.io/pause:2.0
```


© Copyright 2024 Zenika. All rights reserved

POD AFFINITY/ANTI-AFFINITY



Pod affinity/anti-affinity mechanism is used to make sure Pods:

- Run on the same Node or set of Nodes
- Run on different Node or set of Nodes

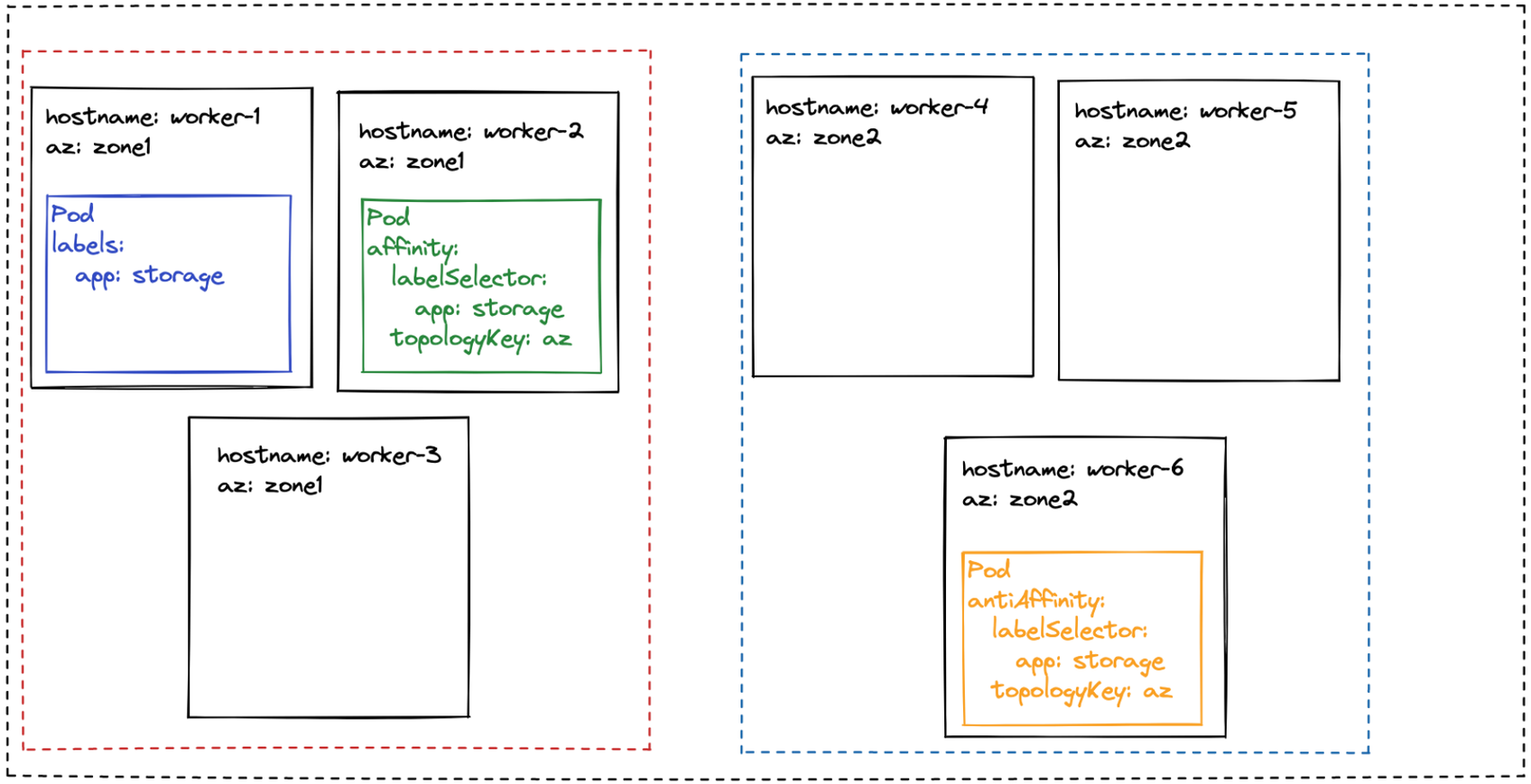
It can be used to:

- Collocate some Pods on the same Node or availability zone to reduce latency
- Spread Pods of a ReplicaSet or StatefulSet on different Nodes and improve resilience

ℹ️ More examples here

© Copyright 2024 Zenika. All rights reserved

POD AFFINITY/ANTI-AFFINITY



© Copyright 2024 Zenika. All rights reserved

POD AFFINITY EXAMPLE



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: with-pod-affinity
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - storage
          topologyKey: az
  containers:
    - name: with-pod-affinity
      image: k8s.gcr.io/pause:2.0
```


© Copyright 2024 Zenika. All rights reserved

POD ANTI-AFFINITY EXAMPLE



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: with-pod-anti-affinity
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchExpressions:
                - key: app
                  operator: In
                  values:
                    - storage
            topologyKey: az
  containers:
    - name: with-pod-anti-affinity
      image: k8s.gcr.io/pause:2.0
```


© Copyright 2024 Zenika. All rights reserved



Lab 3.7: Affinity

© Copyright 2024 Zenika. All rights reserved

DISRUPTIONS



Pods will not disappear until something (controller, person, ...) deletes them, or an inevitable error occurs.

So you have two different disruptions that you will need to manage efficiently:

- Voluntary disruption: cluster upgrade/management, deployment upgrade, ...
- Involuntary disruption: VM/host failure, kernel OOM, network partition, ...

© Copyright 2024 Zenika. All rights reserved

DISRUPTIONS



- To mitigate involuntary disruptions, you will need to:
  - Ensure that your `Pods` have enough resources allocated
  - Ensure that your `Pods` have desired availability
  - Ensure that your `Pods` are spread correctly across your `nodes` and that your `nodes` are not collocated (in the same rack for example).
- To deal with voluntary disruptions you will need to set up `PodDisruptionBudget` resources.
  - On a `classic` kubernetes install, there are no automatic voluntary disruptions, only user-triggered one

© Copyright 2024 Zenika. All rights reserved

POD DISRUPTION BUDGET EXAMPLE



Consider your DNS pod. Your cluster is running several applications that need to perform DNS queries over time ⇒ you want at least 1 instance out of 2 during cluster operation.

First, to mitigate involuntary disruptions, you will need to ensure affinity/anti-affinity like previously explained and high availability using more than one replicas.

And second, you will need to add a `PodDisruptionBudget` to ensure that you can operate your cluster without endangering the availability of your current workload.

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: kube-dns-pdb
  namespace: kube-system
spec:
  minAvailable: 1
  selector:
    matchLabels:
      k8s-app: kube-dns
```


© Copyright 2024 Zenika. All rights reserved

POD DISRUPTION BUDGET



The administrator need to use `kubectl` command which respect PodDisruptionBudget with the eviction API like : `kubectl drain`.

⚠️ Warning

- If you only have 1 **Replica** configured for the matched `PodDisruptionBudget` with a `minAvailable` to 1, it will block your **voluntary** disruption indefinitely.
- Not all **voluntary** disruptions are restricted by `PodDisruptionBudget`. For example, deleting deployments or pods bypasses it.

© Copyright 2024 Zenika. All rights reserved

TAINTS AND TOLERATIONS



Taints are used to mark some Nodes as not usable by Pods without the corresponding Toleration.

It can be used to:

- Dedicate some nodes to some users or usage
- Reserve some nodes with special hardware

© Copyright 2024 Zenika. All rights reserved

TAINTS EXAMPLE



```
$ kubectl taint nodes node1 key=value:NoSchedule
```

The taint has key `key`, value `value`, and taint effect `NoSchedule`.

There are 3 possible effects for Taints:

- NoExecute: Pods without the Toleration will be evicted and won't be scheduled on this Node
- NoSchedule: Pods without the Toleration won't be scheduled on this Node
- PreferNoSchedule: The scheduler will try not to schedule Pods without the Toleration on this Node

ℹ️ **NoExecute**: if the Node is taint after some Pods has been started, Pod without the **Toleration** will be evicted.

© Copyright 2024 Zenika. All rights reserved

TOLERATION EXAMPLE



The corresponding Tolerations can then be added to Pods:

```yaml
tolerations:
  - key: "zenika.com/storage"
    operator: "Equal"
    value: "ssd"   # optional
    effect: "NoExecute"
    tolerationSeconds: 3600
```

The additional `tolerationSeconds` is used to delay the eviction of the Pod

© Copyright 2024 Zenika. All rights reserved

IN SHORT



To reserve a Node for a specific usage you need to combine two mechanisms

1. NodeSelector: Pods are deployed on a subset of Nodes
2. Taints/Tolerations: Nodes only accept a subset of Pods
3. NodeSelector + Taints/Tolerations: Pods are deployed on a subset of Nodes which only accept a subset of Pods

© Copyright 2024 Zenika. All rights reserved



Lab 3.8: Taints & Tolerations

© Copyright 2024 Zenika. All rights reserved

CUSTOM SCHEDULERS



Sometimes the Scheduler default configuration does not fit your needs.

It's possible to use a customize the configuration of the scheduler, however extra care should be taken if it can affect the scheduling of some critical workloads.

You can add a custom profile to the scheduler to adapt its behavior:
https://kubernetes.io/docs/reference/scheduling/config/#profiles

Then in identified Pods, define the property `spec.schedulerName` with the needed profile.

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved

TROUBLESHOOTING A CLUSTER



Common steps:

- Know if components are launched as systemd services or static pods
  - Check systemd services status: `systemctl status <service>`
  - Check static pods status: `kubectl get pods -n kube-system`
  - If api-server is unavailable check container status:
    - `crictl --runtime-endpoint=/run/containerd/containerd.sock ps -a`
    - `ctr container ls -a`
  - View static pods logs: `crictl logs <container ID>`
- Check service logs either
  - On the nodes with: `journalctl -u <service> -f`
  - On your log centralization solution

© Copyright 2024 Zenika. All rights reserved

TROUBLESHOOTING API SERVER



- Symptom: API server unavailable
- How to tell:

```
$ kubectl get pods # For example...
The connection to the server
<api-server-address>:<api-server-port> was refused - did you specify the right
host or port?
```

- Probable causes:
  - kube-apiserver down
  - etcd cluster unhealthy
- Remediation
  - Check api-server logs and fix it
  - Heal the etcd cluster

© Copyright 2024 Zenika. All rights reserved

TROUBLESHOOTING SCHEDULER



- Symptom: Pods stay in PENDING status
- How to tell:

```
$ kubectl get pods
NAME                          READY     STATUS    RESTARTS   AGE
whoami-56d4574f6c-2l25s       0/1       Pending   0          4s
```

- Probable causes:
  - kube-scheduler unavailable
  - not enough resources to satisfy Pod creation
- Remediation
  - Check kube-scheduler logs and fix it
  - Check resources requests

© Copyright 2024 Zenika. All rights reserved

TROUBLESHOOTING CONTROLLER-MANAGER



- Symptom: Pods, ReplicaSets, Endpoints are not created
- How to tell:

```
$ kubectl get deploy
NAME      DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
whoami    1         0         0            0           10s
```

- Probable cause:
  - kube-controller-manager unavailable
- Remediation
  - Check kube-controller-manager logs and fix it

© Copyright 2024 Zenika. All rights reserved

TROUBLESHOOTING NODE



- Symptom: Node not ready
- How to tell:

```
$ kubectl get nodes
NAME      STATUS       ROLES            AGE       VERSION
node1     Ready        control-plane    31m       v1.11.3
node2     NotReady     <none>           27m       v1.11.3
```

- Probable causes:
  - kubelet down on node
  - kubelet cannot reach api-server
- Remediation
  - check kubelet service logs
  - fix kubelet service
  - fix kubelet <-> kube-apiserver connectivity

© Copyright 2024 Zenika. All rights reserved

TROUBLESHOOTING DNS



- Symptom: DNS not available
- How to tell:

```
# Inside a pod, try to resolve a service by name
$ nslookup catalogue-db
;; connection timed out; no servers could be reached
```

- Probable cause:
  - core-dns pods not available
- Remediation
  - check that a cluster dns is deployed (either coredns)
  - fix cluster dns deployment

© Copyright 2024 Zenika. All rights reserved

KUBECTL DEBUG



- To debug running Pod, it is possible to launch an *ephemeral* container in this Pod
- Introduced in beta in 1.23, stable since 1.25
- Example to create a debug container in the `my-app` Pod with the `busybox:1.28` image

```
$ kubectl debug -it my-app --image=busybox:1.28 --target=whoami
Targeting container "whoami". If you don't see processes from this container it may be because
the container runtime doesn't support this feature.
Defaulting debug container name to debugger-98jzs.
If you don't see a command prompt, try pressing enter.
/ # hostname
whoami
/ # ps aux
PID   USER     TIME  COMMAND
    1 root      0:00 /whoami # <== this is the processus of the whoami container!!
    9 root      0:00 sh
   15 root      0:00 ps aux
```


© Copyright 2024 Zenika. All rights reserved



Lab 3.9: Troubleshooting

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



CLUSTER UPGRADE

© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



- Architecture
- Installation
- Configuration and operational maintenance
- *Cluster Upgrade*
- Day to day actions
- Extensibility: Operators, CRD and API Servers
- Service Mesh, Security

© Copyright 2024 Zenika. All rights reserved

CHAPTER CONTENT



- Kubernetes Release Cycle
- Upgrade process
- Upgrade with kubeadm
- Upgrade with kops
- Upgrade with kubespray
- Upgrade with gke
- Upgrade with eks

© Copyright 2024 Zenika. All rights reserved

KUBERNETES RELEASE PROCESS



- No LTS (Long Term Support) version
  - One version is supported for about one year
  - 3 releases per year
  - Plan to upgrade AT LEAST every year
- Driven by SIG (Special Interest Group) Release team

© Copyright 2024 Zenika. All rights reserved

KUBERNETES RELEASE CYCLE



A Kubernetes release cycle has a length of approximately 15 weeks.

- Normal Dev (Weeks 1-8)
- Code Freeze (Weeks 9-11)
- Post Release (Weeks 11+)

Code Freeze: The period of ~4 weeks before the final release date, during which only critical bug fixes are merged into the release.

Some time off is allocated:

- during the end of year holidays
- during KubeCon/CloudNativeCon weeks
- an explicit 2 weeks break for SIG Release between each cycle

-- Source: Kubernetes Release Cadence Change

© Copyright 2024 Zenika. All rights reserved

KUBERNETES VERSIONS SUPPORT



K8s version	Release month	End of life
v1.27.x	April 2023	2024-06-28
v1.28.x	August 2023	2024-10-28
v1.29.x	November 2023	2025-02-28
v1.30.x	April 2024	2025-06-28

Check the release notes online on: https://relnotes.k8s.io/

© Copyright 2024 Zenika. All rights reserved

UPGRADE PATTERNS



- Recreate and destroy the whole cluster
  - Easy for stateless workloads and with a front LB
  - Requires more infrastructure
  - Can be used to validate current cluster workloads on new version before upgrade
- Upgrade existant cluster
  - Highly dependent on the tool used for creating the cluster
  - Well documented

© Copyright 2024 Zenika. All rights reserved

STANDARD UPGRADE PROCESS STEPS (1/2)



- Keep these in mind:
  - You have to keep your cluster up to date
  - You have to read the release notes

Failures happen even with the big one : Reddit, march 2023

© Copyright 2024 Zenika. All rights reserved

STANDARD UPGRADE PROCESS STEPS (2/2)



- Upgrading is done with these steps:
  1. Backup your cluster (etcd snapshot and all)
  2. Upgrade the control plane on each nodes
     - Workloads will still be online
     - Control plane might be unavailable if not deployed in HA mode or all nodes are upgraded at once
  3. Upgrade worker nodes
     - Rolling-update or with double node number
     - It might be a good time to patch/update your nodes if you don't do it on a more regular basis

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH KUBEADM



- Upgrade with kubeadm is well documented
  - Even for high availability setups

Upgrade steps:

- Upgrade the control plane
- Upgrade control-plane and node packages (infrastructure pods)
- Upgrade kubelet on each nodes

Kubernetes is made with a tolerated skew of one version between the API Server and other control-plane components.

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH KUBEADM - CONTROL PLANE (1/2)



- Use `kubeadm upgrade plan <version>` to preview the upgrade steps

```
[preflight] Running pre-flight checks.
[...] Running cluster health checks
Components that must be upgraded manually after you have upgraded the control plane with
'kubeadm upgrade apply':
COMPONENT   CURRENT       TARGET
kubelet     1 x v1.29.0   v1.30.0

Upgrade to the latest version in the v1.30 series:

COMPONENT                 CURRENT    TARGET
kube-apiserver            v1.29.0    v1.30.0
kube-controller-manager   v1.29.0    v1.30.0
kube-scheduler            v1.29.0    v1.30.0
kube-proxy                v1.29.0    v1.30.0
CoreDNS                   v1.11.1    v1.11.1
etcd                      3.5.10-0   3.5.12-0

You can now apply the upgrade by executing the following command:

        kubeadm upgrade apply v1.30.0
```


© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH KUBEADM - CONTROL PLANE (2/2)



When the upgrade has been done with kubeadm you should also upgrade your network CNI plugin.

⚠️ These steps are not covered here as they are dependent on the chosen network CNI

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH KUBEADM - NODE KUBELET (1/2)



On each node:

- mark it as unschedulable:
  - `kubectl cordon $NODE`
- Evict workloads:
  - `kubectl drain $NODE`
  - You might need to add `--ignore-daemonsets`
    - `kubectl drain $NODE --ignore-daemonsets`

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH KUBEADM - NODE KUBELET (2/2)



- Upgrade `kubelet` and `kubeadm`
- Upgrade `kubelet` config
  - `kubeadm upgrade node config --kubelet-version <version>`
- Restart the `kubelet`: `systemctl restart kubelet`
- Mark node schedulable: `kubectl uncordon $NODE`

© Copyright 2024 Zenika. All rights reserved



Lab 4.1: Kubeadm upgrade

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH KOPS



Manual upgrade steps:

- Update the version in the cluster declaration
  - `kops edit cluster` or `kops upgrade cluster $NAME`
- Apply the changes on the infrastructure declaration
  - `kops update cluster --yes`
- Check what needs reboot
  - `kops rolling update cluster`
- Restart the nodes
  - `kops rolling update cluster --yes`

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH KUBESPRAY



- There is a dedicated playbook for safe upgrades. Launch it with:
  - `ansible-playbook upgrade-cluster.yml -b -i <inventory> -e <kubernetes_version>`
- The upgrade does the following steps:
  - container runtime (containerd, crio, ...)
  - etcd
  - kubelet and kube-proxy
  - network_plugin
  - kube-apiserver, kube-scheduler and kube-controller-manager
  - Add-ons (e.g.: core-dns)

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH GKE



Upgrade with gke are taken care off by GCP as two steps

- Upgrade control plane
  - During this phase, the control plane will be unavailable
  - The upgrade take ~5-10 minutes
- Rolling update of the nodes
  - Change the version defined on the node pool
  - GCP will then drain, shutdown and restart the node with the new version

© Copyright 2024 Zenika. All rights reserved

UPGRADE WITH EKS - CONTROL PLANE



- Upgrade from the console or with the cli
  - `eksctl upgrade cluster --name my-cluster --version 1.28 --approve`
- Same steps as GCP
- Follow the documentation for upgrading nodes

ℹ️ Details available here

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



DAY TO DAY ACTIONS

© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



- Architecture
- Installation
- Configuration and operational maintenance
- Cluster Upgrade
- *Day to day actions*
- Extensibility: Operators, CRD and API Servers
- Service Mesh, Security

© Copyright 2024 Zenika. All rights reserved

CHAPTER CONTENT



- Namespaces and Instances
- NetworkPolicies
- Common Admission Controllers
- LimitRanges
- Quotas
- Security contexts
- Pod Security Standards
- Authentication/Authorization/RBAC

© Copyright 2024 Zenika. All rights reserved

NAMESPACES AND CLUSTER INSTANCES (1/2)



- How do I split my environments/apps on kubernetes?
  - Clusters are a mean to mutualise resources (cpu, memory, storage, ...)
  - Namespaces provide isolation within a cluster
- But
  - Namespace isolation isn't that strong with current container runtimes
  - One single mistake could jeopardize the entire cluster

© Copyright 2024 Zenika. All rights reserved

NAMESPACES AND CLUSTER INSTANCES (2/2)



- It depends on the maturity, autonomy and responsibility of the teams
- Some patterns:
  - Clusters reflecting the operator teams separation
  - Separate clusters depending on how users interact with them (manually, with tools)
- Things to consider
  - You need to be able to test changes applied to infrastructure components (service mesh, ingress controller, ...)
  - Creating a new cluster (or replicating an existing one) should be easy
  - Use RBAC/Quotas/... to keep everybody satisfied

© Copyright 2024 Zenika. All rights reserved

CONSTRAINTS AVAILABLE ON A NAMESPACES



By grouping ressources in a Namespace, you will be able to apply:

- NetworkPolicy to filter communications between Pods
- LimitRanges and ResourceQuotas to help capacity planning
- RBAC to secure who can do something
- PodSecurityStandard to restrict the security configuration of a Pod

> We will cover those points in the following slides

© Copyright 2024 Zenika. All rights reserved

SECURE SERVICES INSIDE A CLUSTER



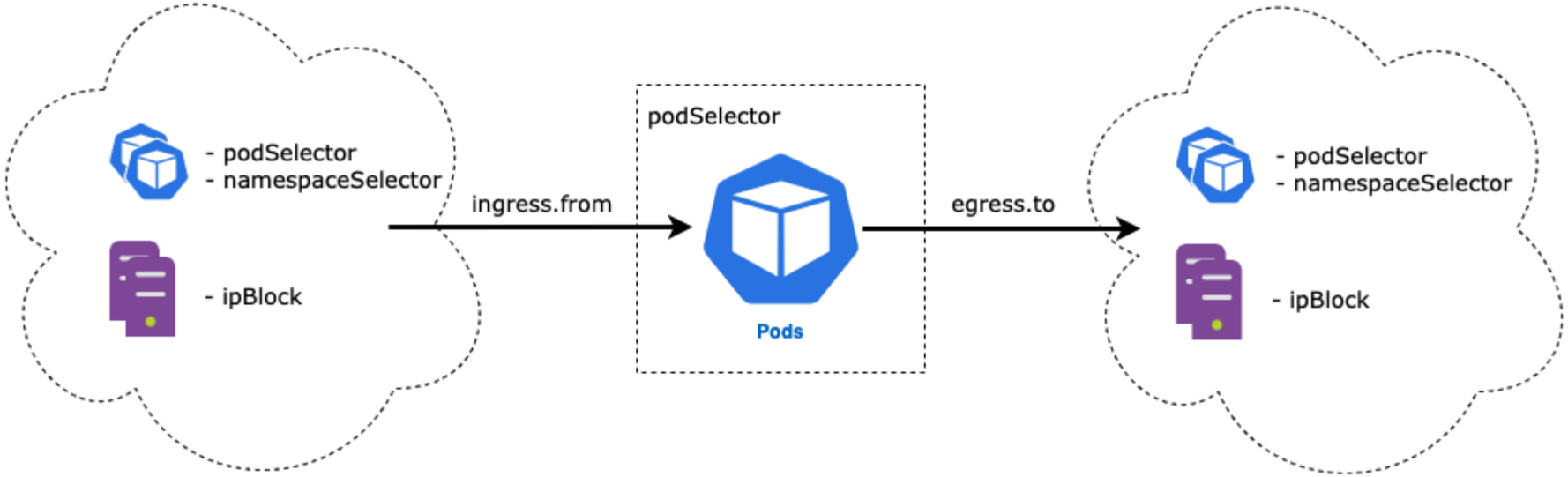
- By default Pods accept traffic from any source
- It may be useful to restrict access to/from some Pods
- To do so you'll use a NetworkPolicy
  - Pods become isolated when they are matched by a NetworkPolicy
  - You must have a network plugin which supports NetworkPolicy
    - Otherwise you'll be able to create the object, but it will have no effect

© Copyright 2024 Zenika. All rights reserved

NETWORK POLICY TYPES

Two types of policy can be created:

- `egress`: traffic originating from the selected Pods
- `ingress`: traffic sent to the selected Pods



© Copyright 2024 Zenika. All rights reserved

NETWORKPOLICY EXAMPLE



```yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: api-allow
spec:
  podSelector:
    matchLabels:
      role: api
      app: review
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
              app: review
```

This policy only allows access to Pods with the label `role=api` from Pods with label `role=frontend`

There are a lot of example of NetworkPolicies on this github repository: https://github.com/ahmetb/kubernetes-network-policy-recipes

© Copyright 2024 Zenika. All rights reserved

ISOLATING NAMESPACES FROM EACH OTHER



```yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  namespace: my-project
  name: deny-from-other-namespaces
spec:
  podSelector:
    matchLabels:
  ingress:
    - from:
        - podSelector: {}
```

- This policy is deployed on Namespace `my-project`
- `podSelector.matchLabels` is empty, so it applies on all Pods in the Namespace
- `ingress.from.podSelector` is empty, so it allows traffic from all Pods in the Namespace

© Copyright 2024 Zenika. All rights reserved

COMPATIBLE/INCOMPATIBLE PLUGINS



- Incompatibles plugins:
  - Flannel
- Compatible plugins:
  - Calico
  - Canal
  - Cilium

© Copyright 2024 Zenika. All rights reserved

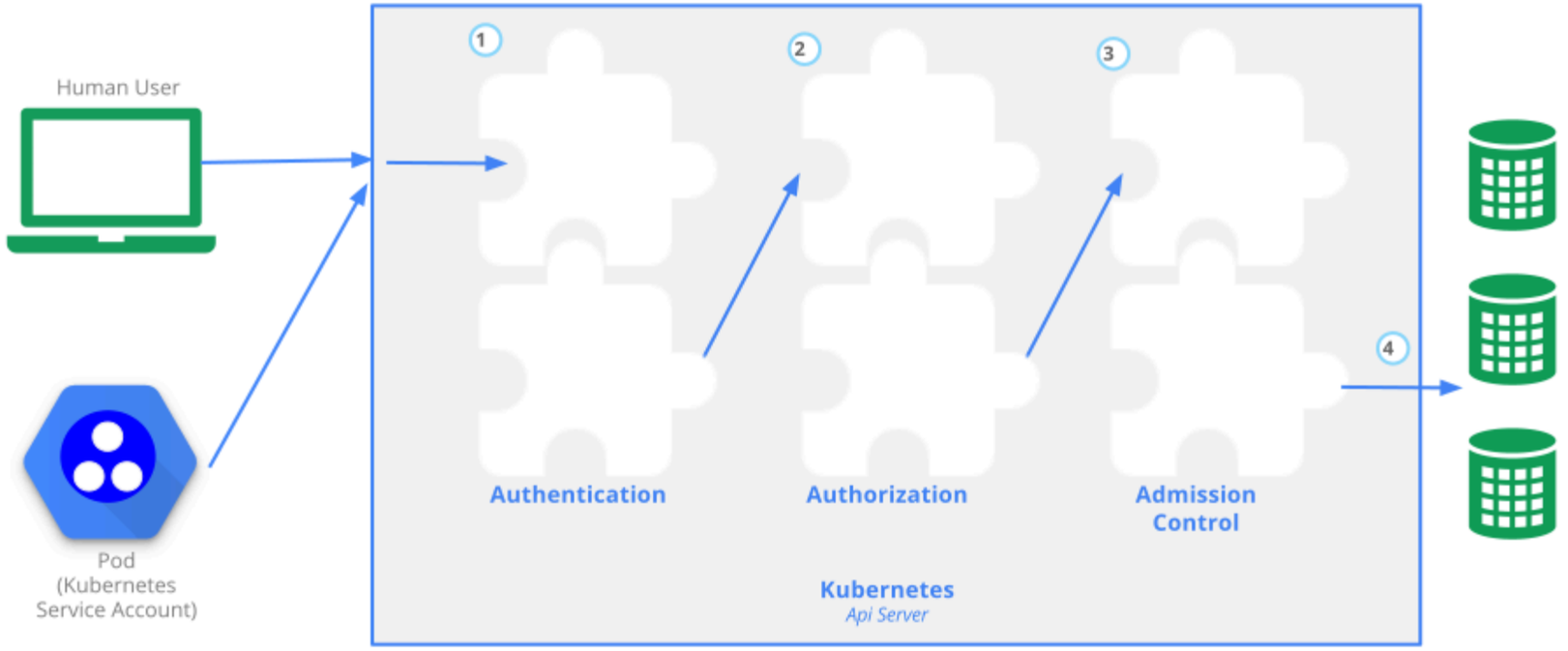


Lab 5.1: Network policies

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS

Objects in requests are not directly persisted.



© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - ALL OF THEM



- AlwaysPullImages: sets imagePullPolicy to Always on every Pods
- CertificateApproval/CertificateSigning: adds more checks on requests for CertificateSigningRequest
- CertificateSubjectRestrictions: prevents creation of CertificateSigningRequest with signerName as system:masters
- DefaultStorageClass: sets DefaultStorageClass on PVC
- DefaultIngressClass: sets IngressClass on Ingress
- DefaultTolerationSeconds: sets the default forgiveness toleration for Pods
- DenyServiceExternalIPs: denies use of ExternalIPs field in Service. This feature is too powerful and can intercept traffic.

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - ALL OF THEM



- ExtendedResourceToleration: adds tolerations to Pods requesting extended resources
- ImagePolicyWebhook: delegate admission decision based on image
- LimitPodHardAntiAffinityTopology: limits AntiAffinity to `kubernetes.io/hostname`
- LimitRanger: applies LimitRange controls
- MutatingAdmissionWebhook: calls any mutating webhooks which match the request
- NamespaceAutoProvision: creates Namespace on demand
- NamespaceExists: checks Namespace existence

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - ALL OF THEM



- NamespaceLifecycle: prevents deletion of reserved Namespace and denies resources creation on a terminating Namespace
- NodeRestriction: limits what a kubelet can modify (owned Pods and Node)
- OwnerReferencesPermissionEnforcement: make `delete` permission necessary to modify `metadata.ownerReferences`
- PodNodeSelector: limits what node selectors may be used within a namespace
- PersistentVolumeClaimResize: adds check on which PVC can be resized
- PodSecurity: replace PodSecurityPolicy resource, enforces security context on Pods
- Priority: applies the priority value to Pods, also rejects Pods without PriorityClass

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - ALL OF THEM



- ResourceQuota: enforces ResourceQuota
- RuntimeClass: add default Pod overhead on Pods with a RuntimeClass configured
- ServiceAccount: automates ServiceAccount management
- StorageObjectInUseProtection: places finalizers on new created PV
- TaintNodesByCondition: taints newly created Nodes as NotReady and NoSchedule
- ValidatingAdmissionPolicy: implements the CEL validation for incoming matched requests
- ValidatingAdmissionWebhook: calls any validating webhooks which match the request

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - DEPRECATED



- AlwaysAdmit / AlwaysDeny
- DenyEscalatingExec / DenyExecOnPrivileged: replaced by PodSecurity
- PersistentVolumeLabel: attaches zone/region labels to PV, replaced by cloud controller manager

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - ALPHA/BETA



- EventRateLimit (alpha): limits requests per scope (Namespace, User, ...)
- PodNodeSelector (alpha): use this Admission Controller to have a default NodeSelector on a Namespace
- PodTolerationRestriction (alpha): checks coherence between Pod and Namespace toleration

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - CONFIGURATION



For each version of Kubernetes there is are default sets of activated/deactivated plugins. Configuration of activated/deactivated controllers is done via command-line parameters to the api server:

- Activated: `kube-apiserver --enable-admission-plugins=...`
- Deactivated: `kube-apiserver --disable-admission-plugins=...`

You can see which admission plugins are activated with:

- `kube-apiserver -h | grep enable-admission-plugins`

© Copyright 2024 Zenika. All rights reserved

COMMON ADMISSION CONTROLLERS - DEFAULT



Here are the default activated ones for the current Kubernetes version:

CertificateApproval, CertificateSigning, CertificateSubjectRestriction, DefaultIngressClass, DefaultStorageClass, DefaultTolerationSeconds, LimitRanger, MutatingAdmissionWebhook, NamespaceLifecycle, PersistentVolumeClaimResize, PodSecurity, Priority, ResourceQuota, RuntimeClass, ServiceAccount, StorageObjectInUseProtection, TaintNodesByCondition, ValidatingAdmissionWebhook

© Copyright 2024 Zenika. All rights reserved



Lab 5.2: Admission controllers

© Copyright 2024 Zenika. All rights reserved

LIMITRANGES



- Kubernetes best practices include defining Request/Limits for workloads
  - It helps the scheduler
  - It's a good way to make resource usage explicit
  - Using Quotas imposes every container to define a memory limit
- LimitRanges are used to ensure default, min, max request/limit values for cpu and memory usage
- How?
  - Create an object LimitRange in the concerned Namespace

© Copyright 2024 Zenika. All rights reserved

LIMITRANGES EXAMPLE



`limit-range.yml`:

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: limit-range
  namespace: limit
spec:
  limits:
    - default:
        cpu: 1
        memory: 512Mi
      defaultRequest:
        cpu: 0.5
        memory: 256Mi
      type: Container
```


© Copyright 2024 Zenika. All rights reserved

LIMITRANGES OTHER PROPERTIES



- Besides default values, LimitRanges can be set for the following:
  - `min`/`max`: applied to CPU or memory
  - `maxLimitRequestRatio`: the max authorized ratio between limit and request
  - `type`: the limit can be applied to individual `Container`

Keep in mind that:

```
min ≤ request ≤ limit ≤ max
```


© Copyright 2024 Zenika. All rights reserved

LIMITRANGES BEHAVIOUR



- If you set only `min` and `max`, `default` and `defaultRequest` values will be automatically set with the `max` value
- If a container defines a limit but no request, the container request is set to the container limit as default
- If a container defines a request but not a limit, the container limit is set to the default limit

⚠️ Warning: If two or more **LimitRange** objects exist in the namespace, it is not deterministic which default value will be applied.

© Copyright 2024 Zenika. All rights reserved

QUOTAS



When sharing a cluster between projects/teams, you should define some resource Quotas. These Quotas apply on:

- Compute resources
- Extended resources
- Storage resources
- Resource count

© Copyright 2024 Zenika. All rights reserved

QUOTAS - HOW



- Quotas are enforced by an admission plugin on resource creation.
- To use quotas with your cluster, you must create a ResourceQuota in the targeted Namespace
- Then the API server will refuse to create the resource if it would result in exceeding the defined Quotas

© Copyright 2024 Zenika. All rights reserved

QUOTA EXAMPLE



quota-example.yml:

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: quota-mem-cpu
  namespace: quotas
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
```


© Copyright 2024 Zenika. All rights reserved

COMPUTE RESOURCE QUOTAS



Compute resources quota can be set on:

- `limits.cpu`
- `limits.memory`
- `requests.cpu`
- `requests.memory`

Keep in mind the default values assigned when only a subset of `limits`, `requests` are defined on Pods or LimitRange

© Copyright 2024 Zenika. All rights reserved

EXTENDED RESOURCE QUOTAS



- It's also possible to define some Extended resources on Nodes.

  For example, you can define `requests.nvidia.com/gpu: 4`

- The definition and attribution of these resources isn't really straightforward at this time, you need to interact directly with the API.
- However it's possible to set Quotas on these extended resources.

© Copyright 2024 Zenika. All rights reserved

STORAGE RESOURCE QUOTAS



Quotas can be set on storage resources:

- `requests.storage`: sum of storage used by the PVC
- `persistentvolumeclaims`: number of PVC
- `<storage-class-name>.storageclass.storage.k8s.io/requests.storage`
- `<storage-class-name>.storageclass.storage.k8s.io/persistentvolumeclaims`
- `requests.ephemeral-storage` (beta)
- `limits.ephemeral-storage` (beta)

© Copyright 2024 Zenika. All rights reserved

OBJECT COUNT QUOTAS



Quotas can be set on object count, with this syntax: `count/<resource>.<group>`

The following resources are supported:

- `configmaps` / `secrets`
- `persistentvolumeclaims`
- `pods`: The total number of pods in a non-terminal state that can exist in the namespace
- `replicationcontrollers`
- `resourcequotas`
- `services` / `services.loadbalancers` / `services.nodeports`

© Copyright 2024 Zenika. All rights reserved

QUOTA SCOPES



Quotas set a scope to restrict their field of application:

- `Terminating` / `NotTerminating`
- `BestEffort` / `NotBestEffort`
- `PriorityClass`
- `CrossNamespacePodAffinity`

© Copyright 2024 Zenika. All rights reserved

QUOTAS AND PRIORITYCLASS



Quotas can also be applied to PriorityClass

Example of ResourceQuota applied to a PriorityClass:

```
...
scopeSelector:
  matchExpressions:
    - operator: In
      scopeName: PriorityClass
      values: ["high"]
...
```


© Copyright 2024 Zenika. All rights reserved

QUOTAS AND CROSSNAMESPACEPODAFFINITY



Quotas can also be applied to CrossNamespacePodAffinity

Example of ResourceQuota applied to a PriorityClass:

```
...
scopeSelector:
  matchExpressions:
    - operator: In
      scopeName: PriorityClass
      values: ["high"]
...
```


© Copyright 2024 Zenika. All rights reserved



Lab 5.3 & 5.4: LimitRanges & Quotas

© Copyright 2024 Zenika. All rights reserved

SECURITYCONTEXT



A security context is used to define some security properties of the process running inside containers.

- Pod and Container properties
  - `runAsUser`: the first process of the Pod containers will run as `uid`
  - `runAsGroup`: default `gid` of files created in volumes
  - `runAsNonRoot`: if set to true, the Kubelet won't allow `root` to run the container
  - `seLinuxOptions`: SELinux context to be applied to all containers
  - `supplementalGroups`: other `gid` associated to the user running containers

© Copyright 2024 Zenika. All rights reserved

SECURITYCONTEXT PROPERTIES (2/2)



- Pod specifics:
  - `fsGroup`: default `gid` of the process and change the group owner of the files for some volume types
- Container specifics:
  - `allowPrivilegeEscalation`
  - `capabilities`: capabilities to add/drop
  - `privileged`: if set to true, the container will run as privileged
  - `readOnlyRootFilesystem`

© Copyright 2024 Zenika. All rights reserved

SECURITYCONTEXT EXAMPLE



pod-with-security-context.yml:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
    runAsUser: 1000
    fsGroup: 2000
  volumes:
    - name: sec-ctx-vol
      emptyDir: {}
  containers:
    - name: sec-ctx-demo
      image: gcr.io/node-hello:1.0
      volumeMounts:
        - name: sec-ctx-vol
          mountPath: /data/demo
      securityContext:
        allowPrivilegeEscalation: false
```


© Copyright 2024 Zenika. All rights reserved

POD SECURITY ADMISSION



The Pod security standards define three levels of security policies which can be applied to validate Pod creation.

- `Privileged`: an unrestricted permissive policy
- `Baseline`: minimal restrictive policy to prevents known privilege escalation
- `Restricted`: heavily restricted policy, following current Pod hardening best practices

These policies define authorized and required values in `SecurityContext` and other PodSpec section regarding security best practices.

Content of these policies: https://kubernetes.io/docs/concepts/security/pod-security-standards/

© Copyright 2024 Zenika. All rights reserved

POD SECURITY ADMISSION



To enable the Pod security admission you must either:

- Enable the PodSecurity admission plugin (for kubernetes 1.22, the feature is behind a feature gate `--feature-gates="...,PodSecurity=true"`)
- Install the PodSecurity admission webhook

One enabled, applied policies are configured on Namespaces with custom labels:

```
# The per-mode level label indicates which policy level to apply for the mode.
#
# MODE must be one of `enforce`, `audit`, or `warn`.
# LEVEL must be one of `privileged`, `baseline`, or `restricted`.
pod-security.kubernetes.io/<MODE>: <LEVEL>
```


© Copyright 2024 Zenika. All rights reserved

OPEN POLICY AGENT

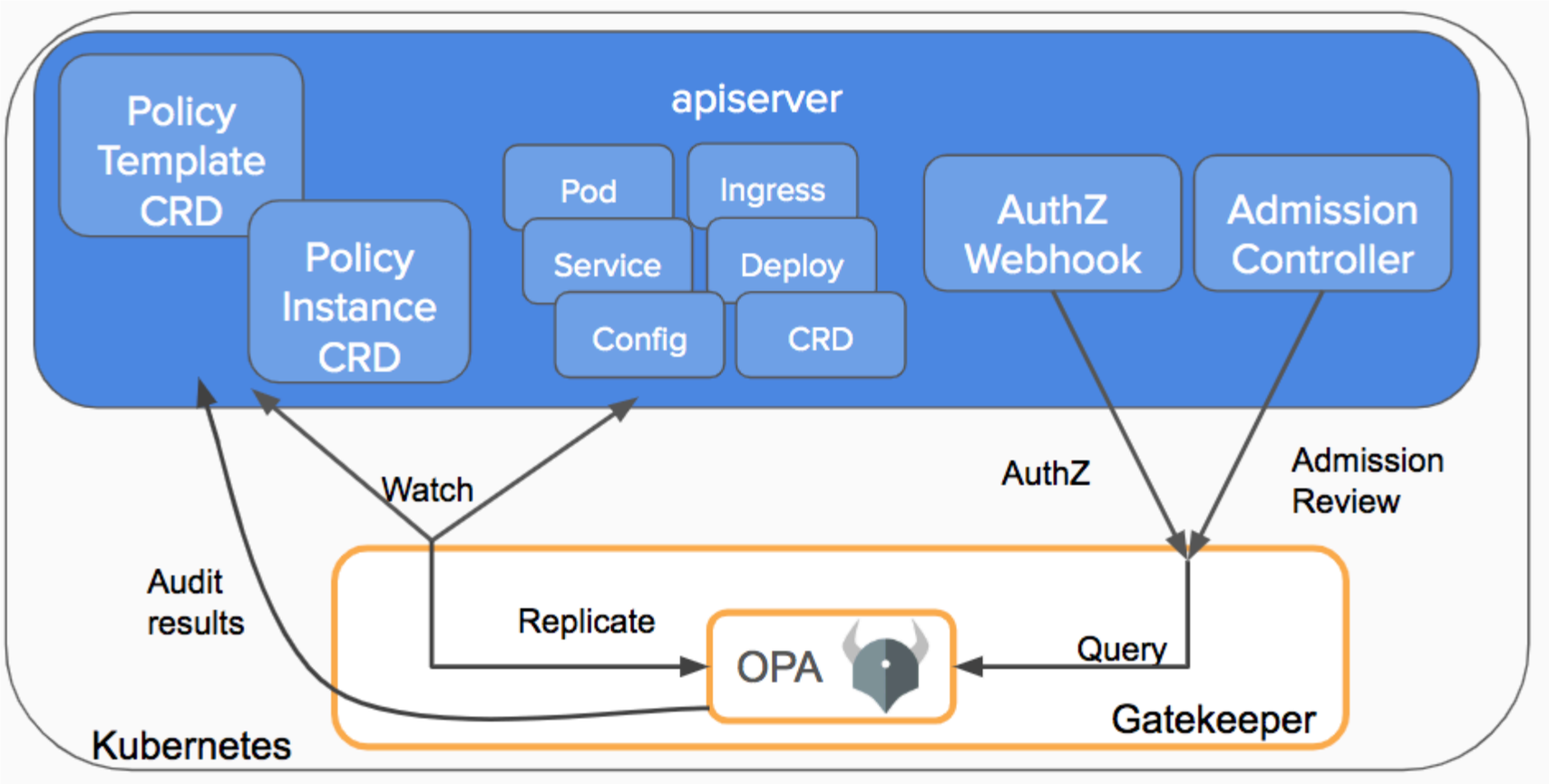


Open Policy Agent is a CNCF project.

- Policy engine
- Audit your stack (not only Kubernetes)
- Enforce policies on your stack
- Define policy as code
  - with a custom language Rego
  - as Custom Resource Definition

© Copyright 2024 Zenika. All rights reserved

OPEN POLICY AGENT - ARCHITECTURE



© Copyright 2024 Zenika. All rights reserved

OPEN POLICY AGENT - POLICY EXAMPLE (1/3)



```yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: ns-must-have-label-gk
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Namespace"]
  parameters:
    labels: ["gatekeeper"]
```


© Copyright 2024 Zenika. All rights reserved

OPEN POLICY AGENT - POLICY EXAMPLE (2/3)



```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8srequiredlabels
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredLabels
      validation:
        # Schema for the `parameters` field
        openAPIV3Schema:
          properties:
            labels:
              type: array
              items: string
[...]
```


© Copyright 2024 Zenika. All rights reserved

OPEN POLICY AGENT - POLICY EXAMPLE (3/3)



```
[...]
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8srequiredlabels

        violation[{"msg": msg, "details": {"missing_labels": missing}}] {
          provided := {label | input.review.object.metadata.labels[label]}
          required := {label | label := input.parameters.labels[_]}
          missing := required - provided
          count(missing) > 0
          msg := sprintf("you must provide labels: %v", [missing])
        }
```


© Copyright 2024 Zenika. All rights reserved



Lab 5.5 & 5.6: Security context & PodSecurityStandard

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION/AUTHORIZATION/RBAC



- Security model:
  - Authentication: know who emitted the request
  - Authorization: is the requests allowed?
- 3 modes:
  - RBAC (Role-Based Access Control)
  - Node
  - Webhook

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION - USER TYPES



- Normal users
  - Can't be created by simple API calls
  - Managed outside the cluster
- Service accounts
  - Bound to a Namespace
  - Created automatically or by API calls
  - Managed inside the cluster with Secret

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION STRATEGIES

Whatever authentication method used, plugins used for authentication will add some of the following information to requests made to the API Server:

- Username
- UID
- Groups
- Extra fields

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION METHODS



- X509 cert
- Static tokens (do not use)
- Bootstrap tokens
- Service Account tokens
- OpenID Connect tokens
- Authentication webhook
- Authentication proxy

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION - X509 CERT



X509 authentication is enabled by passing the argument `--client-ca-file=ca.crt`

- This CA certificate is used to authenticate client certs presented to the API server
- The CN of the certificate is used as the username for the request
- The O fields of the certificates are used as complementary groups

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION - SERVICE ACCOUNTS (1/2)



Service accounts are automatically created and associated with Pods when the ServiceAccount admission controller is enabled.

When a pod starts, a token for the Service Account is generated and mounted in `/var/run/secrets/kubernetes.io/serviceaccount/token`. It expires when the pod is deleted.

Since `1.24`, long-lived token secrets are now longer automatically created. You can still request one using a yaml manifest:

```yaml
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: mysecretname
  annotations:
    kubernetes.io/service-account.name: myserviceaccount
```


© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION - SERVICE ACCOUNTS (2/2)



When authenticating with the previous token, the following informations will be added to the request:

- Username: `system:serviceaccount:(NAMESPACE):(SERVICEACCOUNT)`
- Groups: `system:serviceaccounts` and `system:serviceaccounts:(NAMESPACE)`

⚠️ Warning:

- Access to the secret allows to use the Service Account
- Take care of what Service Accounts are allowed to do

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION - BOOTSTRAP TOKENS



Bootstrap tokens are temporary tokens managed by the cluster which can be used to make a certificate request to the API Server. The request can be approved on the server side with the CSR resource.

- The tokens are of the form `[a-z0-9]{6}.[a-z0-9]{16}`:
  - First is a Token ID
  - Second component is the Token Secret
- Bootstrap token are enabled via `--enable-bootstrap-token-auth` on the API Server
- You *should* enable the TokenCleaner controller on the controller manager with the flag `--controllers=*,tokencleaner`

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION OPENID CONNECT



- OpenID Connect is a flavor of OAuth2 supported by some providers (Azure Active Directory, Salesforce and Google)
- When the user authenticates, it returns an additional field `id_token`
- This `id_token` is sent in requests to the API Server
- To enable this authentication mode, you must add the following flags to the API Server:
  - `--oidc-issuer-url`: URL of the provider which allows the API server to discover public signing keys
  - `--oidc-client-id`: A client id that all tokens must be issued for
  - There are many other options which can be used to configure the token validation

© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION WEBHOOK



Authentication webhook are used to delegate token validation to another server. Add the flag `--authentication-token-webhook-config-file`

```
apiVersion: v1
kind: Config
clusters:
  - name: name-of-remote-authn-service
    cluster:
      certificate-authority: /path/to/remote-ca.pem
      # URL of remote service to query. Must use 'https'.
      server: https://authn.example.com/authenticate
# users refers to the API server's webhook configuration.
users:
  - name: name-of-api-server
    user:
      client-certificate: apiserver-client-cert.pem
      client-key: apiserver-client-key.pem
current-context: webhook
contexts:
  - context:
      cluster: name-of-remote-authn-service
      user: name-of-api-sever
    name: webhook
```


© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION WEBHOOK - REQUEST



When receiving a query, the `kube-apiserver` will build and send a query to the configured server

```
{
  "apiVersion": "authentication.k8s.io/v1",
  "kind": "TokenReview",
  "spec": {
    "token": "(BEARERTOKEN)"
  }
}
```


© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION WEBHOOK - RESPONSE



Success response:

```
{
  "apiVersion": "authentication.k8s.io/v1",
  "kind": "TokenReview",
  "status": {
    "authenticated": true,
    "user": {
      "username": "user@mycompany.com",
      "uid": "1000",
      "groups": [ "developers" ],
      "extra": { "extrafield1": [ "..." ] }
    }
  }
}
```

Failure response:

```
{
  "apiVersion": "authentication.k8s.io/v1",
  "kind": "TokenReview",
  "status": {
    "authenticated": false
  }
}
```


© Copyright 2024 Zenika. All rights reserved

AUTHENTICATION PROXY



With this mode, the API server delegates the authentication to a front server. The authentication server sends user information through the use of HTTP headers.

This authentication method uses these flags:

- `--requestheader-username-headers=X-Remote-User`
- `--requestheader-group-headers=X-Remote-Group`
- `--requestheader-extra-headers-prefix=X-Remote-Extra-`

To prevent the use of forged headers, the authenticating proxy must present a client certificate.

⚠️ Do not reuse this CA in another context and take care when using this authentication method.

© Copyright 2024 Zenika. All rights reserved

AUTHORIZATION (1/3)



- RBAC
- Webhook
- Node

© Copyright 2024 Zenika. All rights reserved

AUTHORIZATION (2/3)



Request attributes:

- user
- group
- extra
- Request Path (if not API resource)
- API
  - Request verb (`get`, `list`, `create`, `update`, `patch`, `watch`, `proxy`, `redirect`, `delete`, `deletecollection`)
  - Resource / Subresource
  - Namespace
  - API Group

© Copyright 2024 Zenika. All rights reserved

AUTHORIZATION (3/3)



Complementary specialized verbs on some resources:

- RBAC: `bind` on `role`, `clusterrole` in `rbac.authorization.k8s.io`
- Authentication layer: `impersonate` on `users`, `group`, `serviceaccounts` in the core API group and `userextras` in the `authentication.k8s.io` API group

© Copyright 2024 Zenika. All rights reserved

RBAC AND ROLE BINDING



Three kind of resources involved:

- User: who?
- Role/ClusterRole: what is allowed?
- RoleBinding/ClusterRoleBinding: Role affectation to a User

© Copyright 2024 Zenika. All rights reserved

RBAC - ROLE



- A Role is a set of permission on some resources
- Permissions are additive (only Allow rules)
- Defines permissions on resources within a Namespace

```yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: my-project
  name: pod-reader
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "watch", "list"]
  - apiGroups: ["apps"]
    resources: ["daemonsets"]
    verbs: ["get", "watch", "list"]
```


© Copyright 2024 Zenika. All rights reserved

RBAC - ROLEBINDING



read-pods-role-binding.yml

```yaml
# This role binding allows "jane" to read pods in the "default" namespace.
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-pods
  namespace: default
subjects:
  - kind: User
    name: jane # Name is case sensitive
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role #this must be Role or ClusterRole
  name: pod-reader # this must match the name of the Role or ClusterRole you wish to bind to
  apiGroup: rbac.authorization.k8s.io
```


© Copyright 2024 Zenika. All rights reserved

RBAC - CLUSTERROLE/CLUSTERROLEBINDING



A ClusterRole is used to grant permissions on resources:

- Cluster scoped resources (See `kubectl api-resources`)
- Namespaced resources accross all Namespaces
- Non-resource endpoints (like `/healthz`)

`secret-reader-cluster-role.yml`

```yaml
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  # "namespace" omitted since ClusterRoles are not namespaced
  name: secret-reader
rules:
  - apiGroups: [""]
    resources: ["secrets"]
    verbs: ["get", "watch", "list"]
```


© Copyright 2024 Zenika. All rights reserved

RBAC - CLUSTERROLE AGGREGATION (1/2)



It is possible to aggregated multiple `ClusterRole` into one :

`cluster-role-monitoring.yaml`

```yaml
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: monitoring-endpoints
  labels:
    rbac.example.com/aggregate-to-monitoring: "true"
rules:
  - ...
```


© Copyright 2024 Zenika. All rights reserved

RBAC - CLUSTERROLE AGGREGATION (2/2)



cluster-role-aggregate.yaml

```yaml
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: monitoring
aggregationRule:
  clusterRoleSelectors:
    - matchLabels:
        rbac.example.com/aggregate-to-monitoring: "true"
rules: [] # Rules are automatically filled in by the controller manager.
```


© Copyright 2024 Zenika. All rights reserved

RBAC - AUDIT RIGHTS



To check the current user rights, you may use the command:

```
$ kubectl auth can-i get pods
yes
$ kubectl auth can-i --list
Resources                                       Non-Resource URLs   Resource Names   Verbs
*.*                                             []                  []               [*]
                                                [*]                 []               [*]
selfsubjectaccessreviews.authorization.k8s.io   []                  []               [create]
selfsubjectrulesreviews.authorization.k8s.io    []                  []               [create]
                                                [/api/*]            []               [get]
                                                [/api]              []               [get]
                                                [/apis/*]           []               [get]
                                                [/apis]             []               [get]
                                                [/version]          []               [get]
                                                [/version]          []

[...]
```


© Copyright 2024 Zenika. All rights reserved

RBAC - BEST PRACTICE



To keep your cluster safe and healthy, you should limit access at least to these resources:

- Role / RoleBinding
- ClusterRole / ClusterRoleBinding
- Namespace (for PodSecurityStandard)
- LimitRanges
- ResourceQuotas
- NetworkPolicy
- Secrets

© Copyright 2024 Zenika. All rights reserved

AUTHORIZATION - WEBHOOK



This authorization mode, makes the API server sending a query to an external service when determining user privileges.

To be activated, the Webhook must be configured with the flag `--authorization-webhook-config-file=webhook-conf.yml`

```
apiVersion: v1
kind: Config
clusters:
  - name: name-of-remote-authz-service
    cluster:
      certificate-authority: ca-remote-svc.pem
      server: https://remote.authz.svc.com/authorize
users:
  - name: name-of-api-server
    user:
      client-certificate: webhook-client-cert.pem
      client-key: webhook-client-key.pem
current-context: webhook
contexts:
  - context:
      cluster: name-of-remote-authz-service
      user: name-of-api-server
    name: webhook
```


© Copyright 2024 Zenika. All rights reserved

AUTHORIZATION - WEBHOOK REQUEST



subject-access-review.json

```
{
  "apiVersion": "authorization.k8s.io/v1beta1",
  "kind": "SubjectAccessReview",
  "spec": {
    "resourceAttributes": {
      "namespace": "projet-a",
      "verb": "get",
      "group": "apps",
      "resource": "deployment"
    },
    "user": "jane",
    "group": [
      "group1",
      "group2"
    ]
  }
}
```


© Copyright 2024 Zenika. All rights reserved

AUTHORIZATION - WEBHOOK RESULT



access-denied.json

```
{
  "apiVersion": "authorization.k8s.io/v1beta1",
  "kind": "SubjectAccessReview",
  "status": {
    "allowed": false,
    "reason": "user does not have read access to the namespace"
  }
}
```

access-allowed.json

```
{
  "apiVersion": "authorization.k8s.io/v1beta1",
  "kind": "SubjectAccessReview",
  "status": {
    "allowed": true
  }
}
```


© Copyright 2024 Zenika. All rights reserved

AUTHORIZATION - NODE



The Node authorizer is used to allow kubelets to perform required API operations:

- Read: services, endpoints, nodes, pods, secrets, configmap, pvc/pv
- Write: nodes, node status, pods, pods status, events
- Read/Write access to CSR API for TLS bootstrapping
- Create: tokenreviews, subjectaccessreviews for delegated auth/authz checks

Used with the NodeRestriction admission plugin, the Node authorizer limits access to owned resources.

© Copyright 2024 Zenika. All rights reserved



Lab 5.7: RBAC

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



EXTENSIBILITY: OPERATORS, CRD AND API SERVERS

© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



- Architecture
- Installation
- Configuration and operational maintenance
- Cluster Upgrade
- Day to day actions
- *Extensibility: Operators, CRD and API Servers*
- Service Mesh, Security

© Copyright 2024 Zenika. All rights reserved

CHAPTER CONTENT



- Controllers
- Dynamic Admission Controllers
- Custom Resource Definition
- Operators
- API Servers/Aggregation layer

© Copyright 2024 Zenika. All rights reserved

SCENARIOS



Imagine that you want to:

- Control user label when a resource is created
- Reject Namespace deletion if not empty
- Inject a container automatically when a Pod is created
- Create a Postgresql object to deploy a database, create a database, a user, etc..

© Copyright 2024 Zenika. All rights reserved

INTRODUCTION



Kubernetes is highly configurable and extensible. It allows you to create:

- your own API objects (CRD)
- your own mecanism to control resources (Admission Controllers)
- and a lot of other things using Operators.

© Copyright 2024 Zenika. All rights reserved

CONTROLLERS (1/2)



In Kubernetes, a controller is a control loop that watches the shared state of the cluster through the apiserver and makes changes attempting to move the current state towards the desired state

- Mainly in Golang
- It's recommended to use a dedicated ServiceAccount (limited access)
- Keep it Simple

© Copyright 2024 Zenika. All rights reserved

CONTROLLERS (2/2)



Example: You want to watch namespace creation...

```
namespaceInformer := cache.NewSharedIndexInformer(
  &cache.ListWatch{
    ListFunc: func(options metav1.ListOptions) (runtime.Object, error) {
      return kclient.CoreV1().Namespaces().List(options)
    },
    WatchFunc: func(options metav1.ListOptions) (watch.Interface, error) {
      return kclient.CoreV1().Namespaces().Watch(options)
    },
  },
  &v1.Namespace{},
  3*time.Minute,
  cache.Indexers{cache.NamespaceIndex: cache.MetaNamespaceIndexFunc},
)


  ...

namespaceInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
  AddFunc: namespaceWatcher.createCustomerRules,
})
```

If you want to deep dive into controllers, please read this article

© Copyright 2024 Zenika. All rights reserved

DYNAMIC ADMISSION CONTROLLERS (1/4)



> Admission webhooks are HTTP callbacks that receive admission requests and do something with them.

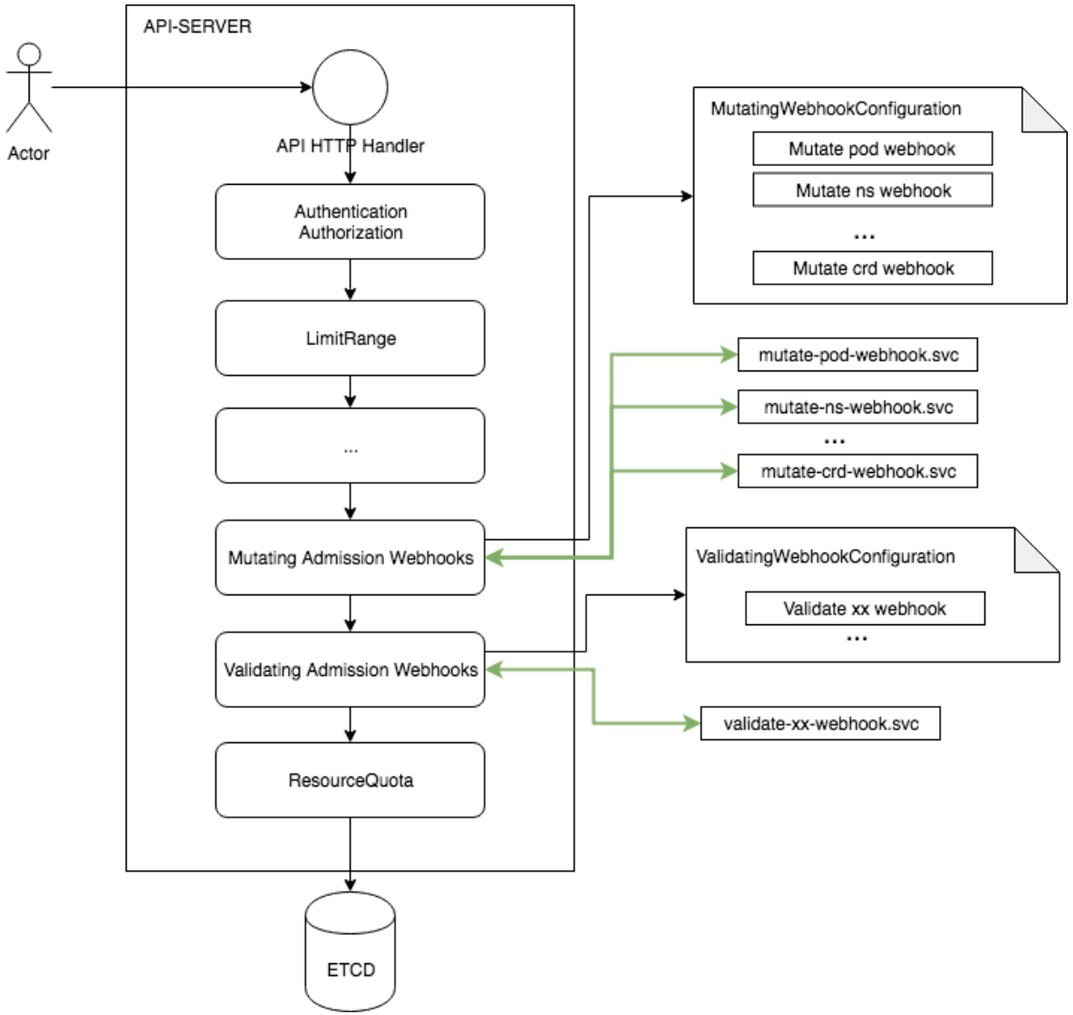
2 native types in `admissionregistration.k8s.io/v1`:

- Mutating admissionwebhook
- Validating admissionwebhook

The principle is simple, each time you have an event, api-server will trigger an internal service in SSL to validate, deny, or modify a resource.

© Copyright 2024 Zenika. All rights reserved

DYNAMIC ADMISSION CONTROLLERS (2/4)



© Copyright 2024 Zenika. All rights reserved

DYNAMIC ADMISSION CONTROLLERS (3/4)



Use cases:

- You can use Validating admission webhook to control if annotations are well defined
- You can use Validating admission webhook to avoid deletion of non empty namespace
- Istio use Mutating admission webhook to inject Envoy container inside each pod
- You can use Mutating admission webhook to inject Vault init container

If you want to deep dive into Dynamic admission controllers, you can find a lot of information on the IBM medium blog

© Copyright 2024 Zenika. All rights reserved

DYNAMIC ADMISSION CONTROLLERS (4/4)



Example of configuration:

```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: pod-mutating-webhook
webhooks:
  - clientConfig:
      caBundle: xxxxxxxxxxxxxx==
      service:
        name: pod-mutating-webhook
        namespace: kube-system
        path: /mutate
    failurePolicy: Ignore
    name: pod-mutating-webhook.custom.com
    namespaceSelector: {}
    rules:
      - apiGroups: [ "" ]
        apiVersions: [ "v1" ]
        operations: [ "CREATE" ]
        resources: [ "pods" ]
```


© Copyright 2024 Zenika. All rights reserved

CUSTOM RESOURCE DEFINITION (1/2)



The CustomResourceDefinition API resource allows you to define custom resources. Defining a CRD object creates a new custom resource with a name and schema that you specify. The Kubernetes API serves and handles the storage of your custom resource.

```yaml
apiVersion: "stable.example.com/v1"
kind: CronTab
metadata:
  name: my-new-cron-object
spec:
  cronSpec: "* * * * */5"
  image: my-awesome-cron-image
```

```
$ kubectl get crontab
NAME                 AGE
my-new-cron-object   6s
```


© Copyright 2024 Zenika. All rights reserved

CUSTOM RESOURCE DEFINITION (2/2)



Resource definition example:

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  # name must match the spec fields below, and be in the form: <plural>.<group>
  name: crontabs.stable.example.com
spec:
  # group name to use for REST API: /apis/<group>/<version>
  group: stable.example.com
  versions:
    - name: v1
      served: true
      storage: true
  # either Namespaced or Cluster
  scope: Namespaced
  names:
    plural: crontabs
    singular: crontab
    kind: CronTab
    shortNames:
      - ct
...
```


© Copyright 2024 Zenika. All rights reserved



Lab 6.1: Custom Resource Definition

© Copyright 2024 Zenika. All rights reserved

OPERATORS (1/2)



Kubernetes Operators is a concept created by CoreOS

The main function of an Operator is to:

- Read from a custom object that represents your application instance
- Make changes to what is running to match this desired state

> Operator = controller pattern + API extension with CRD + single-app focus

© Copyright 2024 Zenika. All rights reserved

OPERATORS (2/2)



How to create your own?

- From scratch: Create your own CRD and controller example
- Using Kubebuilder: example
- Using operator-sdk: example

An awesome list of existing operator is available here: https://operatorhub.io/

© Copyright 2024 Zenika. All rights reserved

API SERVERS/AGGREGATION LAYER



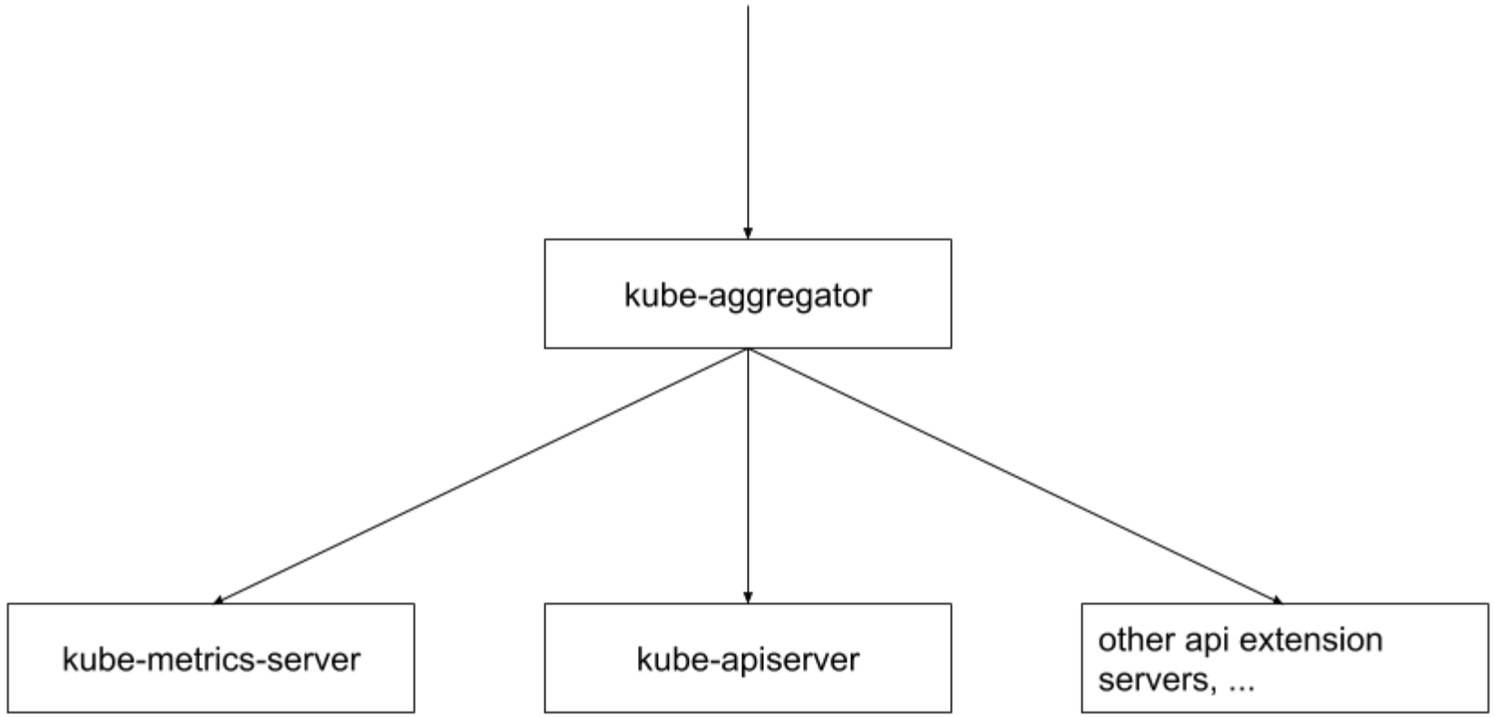
CRD is a really good way to expose new API resources and use Kubernetes to store their state.

However, sometimes you might need to expose new API resources and handle their storage in a specific way.

> e.g: Exposing metrics as API resources, but not using etcd to store them

© Copyright 2024 Zenika. All rights reserved

AGGREGATION LAYER - KUBE-AGGREGATOR



© Copyright 2024 Zenika. All rights reserved

API SERVER CONFIGURATION



To activate the aggregation layer, you must add the following flags to the kube-apiserver:

```
--proxy-client-cert-file=<path to aggregator proxy cert>
--proxy-client-key-file=<path to aggregator proxy key>
```

If `kube-proxy` is not running on host running the API server, add the flag: `--enable-aggregator-routing=true`

© Copyright 2024 Zenika. All rights reserved

API SERVICE EXAMPLE



Then to register new `api-resources` you must declare an APIService to the kube-apiserver

`metrics-apiservice.yaml`

```yaml
apiVersion: apiregistration.k8s.io/v1
kind: APIService
metadata:
  name: v1beta1.metrics.k8s.io
spec:
  service:
    name: metrics-server
    namespace: kube-system
  group: metrics.k8s.io
  version: v1beta1
  insecureSkipTLSVerify: true
  groupPriorityMinimum: 100
  versionPriority: 100
```


© Copyright 2024 Zenika. All rights reserved



Lab 6.2: Aggregation layer

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



SERVICE MESH, SECURITY

© Copyright 2024 Zenika. All rights reserved

TABLE OF CONTENTS



- Architecture
- Installation
- Configuration and operational maintenance
- Cluster Upgrade
- Day to day actions
- Extensibility: Operators, CRD and API Servers
- *Service Mesh, Security*

© Copyright 2024 Zenika. All rights reserved

CHAPTER CONTENT



- Service Mesh
- Security

© Copyright 2024 Zenika. All rights reserved

SERVICE MESH



- Dedicated infrastructure layer for service-to-service communication
- Addresses safety, reliability
- Might also be used for/as:
  - Circuit breaker
  - Traffic splitter/shaper
  - Distributed tracing
  - TLS encryption
  - ...

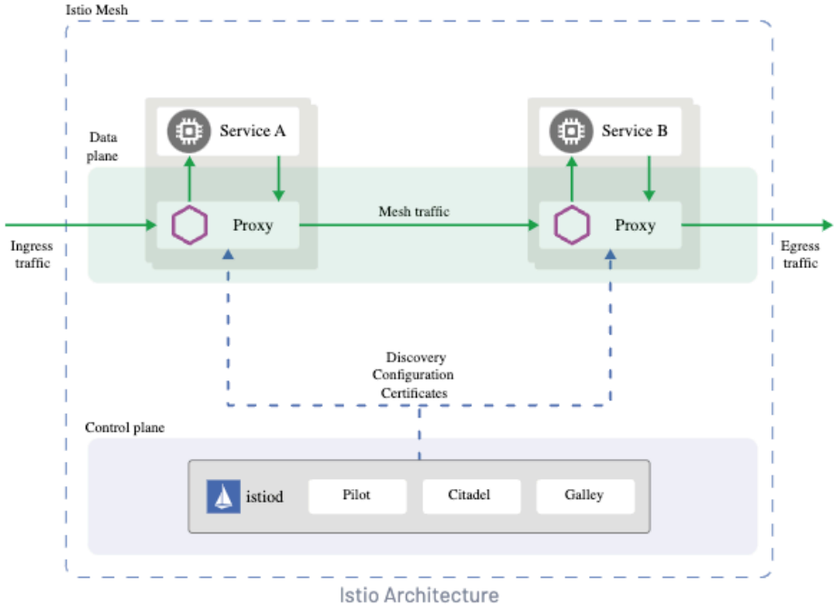
-- Source: https://blog.buoyant.io/2017/04/25/whats-a-service-mesh-and-why-do-i-need-one/

© Copyright 2024 Zenika. All rights reserved

SERVICE MESH - DEPLOYMENT EXAMPLE



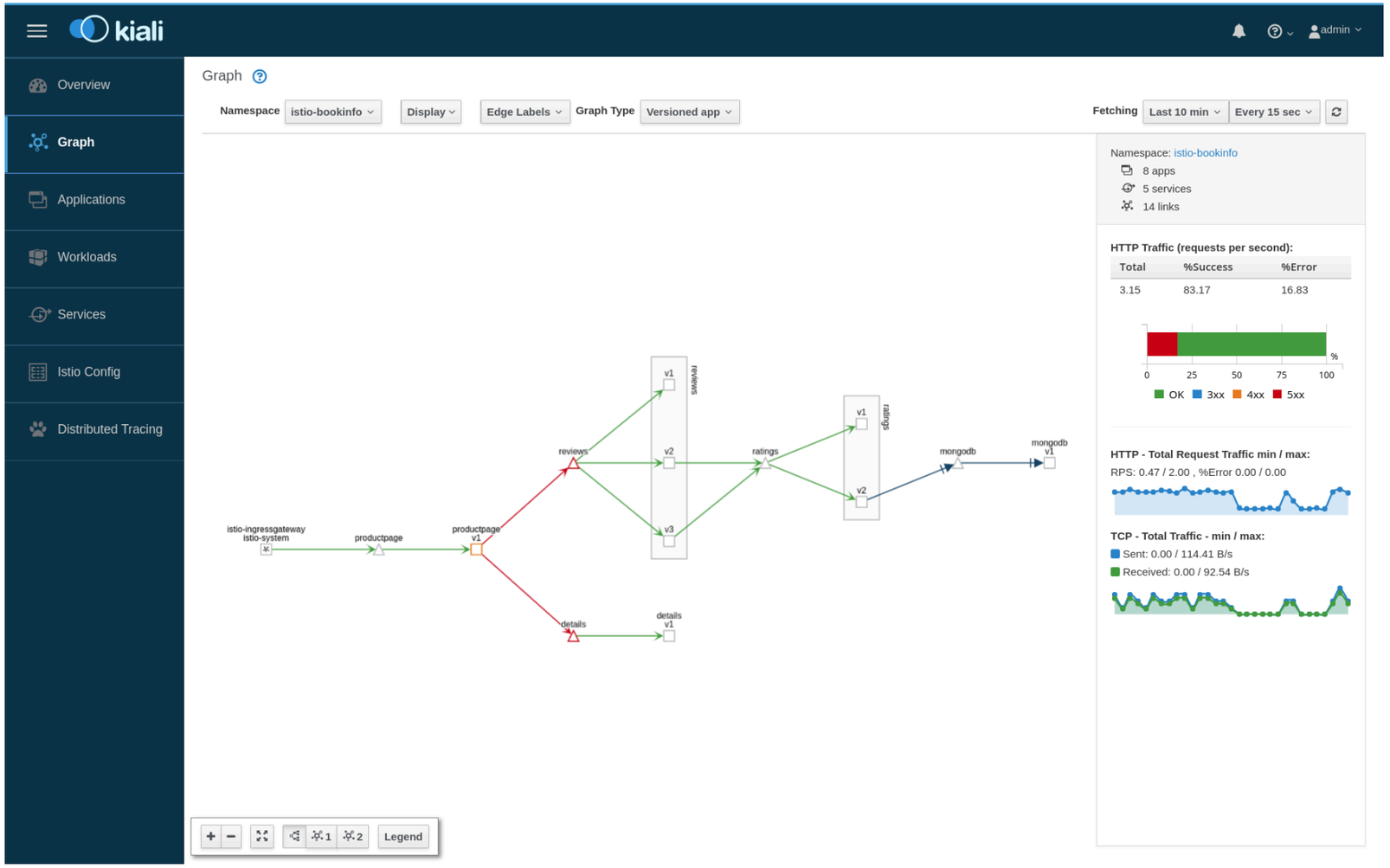
- Service mesh usually relies on the Mutating Webhook admission controller
  - When a Pod is created an Envoy container is added to it
  - All communications go through this proxy
  - The Pilot component of Istio manages Envoy proxies configuration



Istio Architecture

© Copyright 2024 Zenika. All rights reserved

SERVICE MESH - OBSERVABILITY



-- Source: https://www.kiali.io/

© Copyright 2024 Zenika. All rights reserved

SERVICE MESH - OBSERVABILITY - HOW IT WORKS



- Envoy injects a Correlation Id in a coming request if not present
- The application must convey the Correlation Id in output requests
- Traffic metrics are built from the Correlation Ids
- Visualisation is built upon theses metrics

> Correlation Id: Unique identifier value that is attached to requests and messages that allow reference to a particular transaction or event chain.

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved

SECURITY



- There are many tools and articles available on how to secure a Kubernetes cluster:
  - Security
  - Securing Kubernetes
  - kube-bench

© Copyright 2024 Zenika. All rights reserved

SECURITY - CLUSTER



- Use TLS for all communications
- Enable RBAC
  - Limit service account and users rights
- Limit access to trusted registries
- Audit your cluster for vulnerabilities/misconfigurations
- Update your cluster regularly

© Copyright 2024 Zenika. All rights reserved

SECURITY - WORKLOADS (1/2)



- Run containers as NonRoot
- Use PodSecurityAdmission and Linux security features to apply least privilege to Workloads
- Audit manifests with tools like kubesec
  - Generates risk scores for resources
  - Available as `kubectl` plugin
  - Available online: https://kubesec.io/
- Use NetworkPolicies

© Copyright 2024 Zenika. All rights reserved

SECURITY - WORKLOADS (1/2)



Example of best practices on container definition using `SecurityContext`

```yaml
securityContext:
  capabilities:                       # Removing all unnecessary capabilities
    drop:                             # See `man capabilities`
      - ALL
  privileged: false                   # Force a non privileged container
  runAsNonRoot: true                  # Force the container to run with another user than root
  allowPrivilegeEscalation: false     # Forbid privilege escalation
  runAsUser: 5000                     # Force a specific `uid` for the main process
  readOnlyRootFilesystem: true        # Forbid modifications of the root fs
```

See:

- https://docs.docker.com/engine/reference/run/#runtime-privilege-and-linux-capabilities
- https://stackoverflow.com/a/66637144

© Copyright 2024 Zenika. All rights reserved

SECURITY - IMAGES



- Scan images for vulnerabilities
  - Clair
  - Aquasec microscanner
  - Anchore
  - Trivy
- Sign images and manage the whole software supply chain
  - Notary
  - Grafeas

© Copyright 2024 Zenika. All rights reserved

ENCRYPTION ... OR NOT



⚠️ by default, **Secret** are *not encrypted* in the **etcd** database

- why ? Quoted from this

> By creating a threat model that includes the kinds of attacks you want to mitigate, it's clear that managing secrets safely is extremely difficult. The problem is NOT that secrets are just base64 encoded; that was never meant as a security feature. And the problem cannot be simply waved away by software/cloud providers and their flashy documentation.

- Some partial mitigations:
  - setup a `EncryptionConfig` for an encryption-provider
  - Integration of Vault (Hashicorp) to manage Secrets https://github.com/Boostport/kubernetes-vault
  - Store encrypted values directly in Secrets with sealed-secrets

© Copyright 2024 Zenika. All rights reserved

ENCRYPTION AT REST - HOW 1/2



- Generate an encryption key:

```
$ ENCRYPTION_KEY=$(head -c 32 /dev/urandom | base64)
```

- Create an encryption config: `encryption-config.yaml`

```
kind: EncryptionConfig
apiVersion: v1
resources:
  - resources:
      - secrets
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: ${ENCRYPTION_KEY}
      - identity: {}
```


© Copyright 2024 Zenika. All rights reserved

ENCRYPTION AT REST - HOW 2/2



- Configure the `kube-apiserver`
  - Pass the `--encryption-provider-config` flag pointing to the created `encryption-config.yaml`
- Any new Secret will then be encrypted
- To encrypt all existing Secrets, launch:

```
$ kubectl get secrets --all-namespaces -o json | kubectl replace -f -
```

- Several providers exist for encryption: aescbc, kms v2, ...

© Copyright 2024 Zenika. All rights reserved



© Copyright 2024 Zenika. All rights reserved



SUMMARY

© Copyright 2024 Zenika. All rights reserved

INSTALLATION



- Automate cluster installation
- Do not begin with a giant cluster
- Use a distribution which fits your need
- Choose and deploy:
  - A network addon
  - At least one storage provisioner
- Centralize logs and metrics

© Copyright 2024 Zenika. All rights reserved

DAY TO DAY ACTIONS



- Keep up with Kubernetes
  - Automate cluster upgrade
- Keep your Nodes up to date
- Teach users how to deploy on Kubernetes
  - Liveness/Readiness probes
  - Resources limits/requests
  - ...
- Use Quotas/LimitRanges to guide users
- Configure garbage collection

© Copyright 2024 Zenika. All rights reserved

SECURITY



- Apply security at all level
  - Host OS
  - Cluster
  - Docker images
  - Workloads

© Copyright 2024 Zenika. All rights reserved

CKA



- Only official documentation is allowed
- Be comfortable with vim
- Use `kubectl create|run --dry-run=client -o yaml` to save time
- Before starting, don't forget to open the official Cheat Sheet
- Add the following lines in its `~/.vimrc` file to facilitate YAML editing

```
set expandtab
set sw=2
set ts=2
```

There are also many blog articles giving additional information to be well prepared on the day of the exam.

© Copyright 2024 Zenika. All rights reserved

QUESTIONS



© Copyright 2024 Zenika. All rights reserved